Honest Majority Constant-Round MPC with Linear Communication from One-Way Functions

Junru Li
Tsinghua University
and Shanghai Qi Zhi Institute
jr-li24@mails.tsinghua.edu.cn

Yifan Song
Tsinghua University
and Shanghai Qi Zhi Institute
yfsong@mail.tsinghua.edu.cn

Abstract

Secure multiparty computation (MPC) faces a fundamental efficiency trade-off between round complexity and communication complexity: without fully homomorphic encryption, protocols with constant round complexity (e.g., protocols based on garbled circuits) incur high communication cost, while communication-efficient approaches (e.g., protocols based on secret sharing) have round complexity linear in the depth of the circuit. In this work, we focus on reducing the communication complexity of constant-round MPC protocols in the honest majority setting. Existing results either rely on strong assumptions (e.g., random oracles, DDH, LPN) or incur high communication of $\Omega(|C|n^2\kappa)$ bits under one-way functions (OWFs). However, non-constant-round MPC protocols can achieve linear communication in the number of parties even with information-theoretic security.

We resolve this gap by presenting the first *constant-round* honest majority MPC protocol with *linear* communication complexity of $O(|C|n\kappa + n^2\kappa^2 + n^4\kappa)$ only from OWFs. We introduce novel techniques for computing garbled circuits via party virtualization and efficient local computation of virtual parties, which optimize the existing protocols on multiparty garbling. These allow us to overcome the $O(n^2\kappa)$ bit of communication per-gate bottleneck of prior protocols, matching the scalability of the best non-constant-round protocols in the same setting.

Contents

1 Introduction

Secure multiparty computation (MPC) [Yao82, GMW87] enables multiple parties to jointly evaluate a public function while preserving the privacy of their individual inputs. The practical deployment of MPC protocols heavily depends on two efficiency metrics: *round complexity* and *communication complexity*. While significant progress has been made in optimizing these two metrics independently, existing protocols face a fundamental tension: techniques that minimize round complexity (e.g., protocols based on garbled circuits) typically incur high communication overhead, whereas approaches optimizing communication complexity (e.g., protocols based on secret sharing) often require more interaction rounds. However, for large-scale deployments with geographically distributed parties, both metrics must be simultaneously addressed to achieve practical efficiency.

Communication Complexity of Honest Majority MPC. The honest majority setting, where the number of corrupted parties is strictly smaller than the number of honest parties, is important in the research of MPC, as it provides the possibility of constructing MPC protocols only assuming *one-way functions* (OWFs) or even achieving information-theoretic security.

General non-constant-round honest majority MPC protocols in this regime, often based on secret sharing techniques, have achieved remarkable progress in communication efficiency. The DN protocol [DN07] established linear communication complexity $O(|C|n)$ with semi-honest security for computing a circuit of size $|C|$ among n parties, later extended to malicious security [BFO12]. A line of work has made significant progress on reducing the communication cost [CGH+18, GS20, GLO+21, GSY21, EGPS22]. Recent advances have pushed these bounds further, realizing $O(|C| + Dn)$[1] total communication (where D is the circuit depth) under a strong honest majority ($t = (1/2 - \epsilon)n$ for constant $\epsilon < 1/2$) [GPS21]. Despite these communication optimizations, such protocols inherently require round complexity at least linear in the circuit depth due to their gate-by-gate evaluation paradigm, creating significant efficiency challenges for deep circuits in high-latency network environments.

Communication Complexity of Constant-Round MPC. Constant-round MPC protocols are often based on *garbled circuits* [Yao86] or *fully homomorphic encryptions* (FHE) [Gen09]. The FHE-based approaches not only require stronger cryptographic assumptions, but also incur heavy computation, which performs poorly in concrete efficiency. In this work, we focus on constant-round protocols based on garbled circuits, which were first constructed by Beaver et al. [BMR90] (the well-known BMR protocol) and improved by a line of work [LPSY15, BLO16, BLO17, HOSS18a, HOSS18b, BCO+21]. While achieving a constant number of rounds, these protocols historically suffered from high communication costs of $O(|C|n^2\kappa)$ bits, making them impractical for large-scale deployments.

Recent breakthroughs have finally broken this barrier: Beck et al. [BGH+23] achieved $O(|C|\kappa)$ communication under the LPN assumption in the strong honest majority setting ($(1/2 - \epsilon)n$ corruptions), while Goyal et al. [GLM+24] attained $O(|C|n\kappa)$ under DDH and LPN for dishonest majority. More recently, two works [HKN+25, GLOS25] improve the previous works using more lightweight assumptions. In particular, Heath et al. [HKN+25] achieve $O(|C|n\kappa)$ communication in the strong dishonest majority setting ($t = (1 - \epsilon)n$) assuming OT and RO. Goyal et al. [GLOS25] achieve $O(|C|\kappa + D(n + \kappa)^2\kappa)$ in the same setting. In addition, with an honest majority, OTs are not needed in both works.

A natural problem arises: Can we use weaker assumptions than previous works while preserving their communication complexity? For example, can the results of [HKN+25, GLOS25] be extended to honest majority only assuming OWFs? Unfortunately, both works [HKN+25, GLOS25] require RO for OT extensions [IKNP03, KOS15], and [GLOS25] also requires RO for freeXOR garbling [KS08]. It is not clear how to achieve similar results solely from black-box use of OWFs. In contrast, non-constant-round MPC in the same setting can achieve $O(|C|n)$ communication. This leads to our following question:

"Can we construct a constant-round honest majority MPC protocol that only requires communication of $O(|C|n\kappa)$ bits from one-way functions?"

[1] In this section, we drop the $\mathsf{poly}(n, \kappa)$ term in the communication complexity.

1.1 Our Contribution

In this paper, we answer this question affirmatively.

Theorem 1. *Assuming one-way functions, there exists a computationally secure 26-round MPC protocol against a fully malicious static adversary corrupting up to* $(n-1)/2$ *parties with communication of* $O(|C|n\kappa + n^2\kappa^2 + n^4\kappa)$*, where* n *is the number of parties,* $|C|$ *is the circuit size, and* κ *is the computational security parameter.*

If we only ensure semi-honest security, the round complexity can be reduced to 10. Our protocol leverages novel techniques for computing garbled circuits via party virtualization and performing the virtual parties' local computation, which leads to an efficient multiparty garbling process. We refer the readers to Section 2 for more details of our techniques.

Additional Comparison with [GLOS25]. We note that the construction in [GLOS25] can potentially be adapted to achieve linear communication from black-box use of OWFs. We estimate that the resulting protocol would require at least $\Omega(|C|(n+\kappa)\kappa + D \cdot \mathsf{poly}(n,\kappa))$ communication. We provide more details about this estimation in Section 2.1. Our protocol eliminates both the $\Omega(|C|\kappa^2)$ and $\Omega(D \cdot \mathsf{poly}(n,\kappa))$ terms in the communication complexity. In addition, our protocol does not rely on complex network routing techniques utilized in [GLOS25], which leads to better concrete efficiency.

1.2 Other Related Works

A line of works [GS18, BL18, ACGJ18, ACGJ19] focus on minimizing the round complexity of MPC protocols via round collapsing. A subsequent work of them [ACGJ20] considers minimizing the communication complexity of MPCs in the round-optimal cases, and it achieves $\tilde{O}(|C|) = O(|C|\mathsf{poly}(\log n, \kappa))$ in the strong honest majority setting. We estimate that the construction in [ACGJ20] requires at least $\Theta(|C|\kappa^7)$ communication, which is still far from practice.

2 Technical Overview

In this section, we give an overview of our main techniques. Our goal is to construct a *constant-round* MPC protocol for a Boolean circuit of size $|C|$ with linear communication $O(|C|n\kappa)$ bits only assuming one-way functions. We focus on the standard honest-majority setting, where up to t of $n = 2t+1$ parties may be corrupted.

2.1 Background: the BMR Template and the Communication Bottleneck

Before we introduce our main idea, we first briefly review the BMR template [BMR90], which is followed by most of the previous works on constant-round MPC protocols.

Review of the BMR Template. At a high level, the BMR template provides techniques to jointly compute a Yao's garbled circuit [Yao86]. In Yao's garbled circuit, each wire w of a circuit is associated with a tuple $(\lambda_w, k_{w,0}, k_{w,1})$ randomly picked by the garbler, where λ_w is the wire mask for the wire value v_w, and $k_{w,0}, k_{w,1} \in \mathbb{F}_2^\kappa \cong \mathbb{F}_{2^\kappa} = \mathbb{F}$ are the wire labels. The *colored bit* Λ_w of the wire w is defined to be $\Lambda_w = v_w \oplus \lambda_w$. During the evaluation, we will maintain the invariant that the evaluator will learn the colored bit Λ_w and the associated wire label k_{w,Λ_w}. To this end, for each gate g with input wires a, b and output wire c in the circuit, and for each of the 4 possibilities $\Lambda_a, \Lambda_b \in \{0,1\}$,

- the garbler computes $\Lambda_c(\Lambda_a, \Lambda_b) = g(\Lambda_a \oplus \lambda_a, \Lambda_b \oplus \lambda_b) \oplus \lambda_c$ and encrypts the output wire label $k_{c,\Lambda_c(\Lambda_a,\Lambda_b)} \| \Lambda_c(\Lambda_a, \Lambda_b)$ using input wire labels $(k_{a,\Lambda_a}, k_{b,\Lambda_b})$.

In this way, with input labels and colored bits $k_{a,\Lambda_a}\|\Lambda_a, k_{b,\Lambda_b}\|\Lambda_b$, the evaluator can decrypt the output label and colored bit $k_{c,\Lambda_c}\|\Lambda_c$. During the evaluation of the garbled circuit, the evaluator receives the input labels $k_{i,\Lambda_i}\|\Lambda_i$ for each input wire i of the circuit and decrypts the wire label and colored bit of each wire gate by gate.

In the multiparty setting, Beaver et al. [BMR90] and its follow-up work [DI05] extend this idea by having all parties collaboratively act as the garbler, computing the garbled circuit via a generic MPC protocol (referred to as *multiparty garbling*). A naive implementation would require parties to securely evaluate the underlying encryption scheme within the MPC protocol, incurring significant overhead due to the non-black-box use of cryptographic primitives. To mitigate this, a distributed wire label structure is purposed: each party P_i selects its own wire labels $k^{(i)}_{w,0}, k^{(i)}_{w,1}$ for every wire w, and the wire labels for w concatenate all parties' wire labels, i.e.,

$$\boldsymbol{k}_{w,\Lambda} = k^{(1)}_{w,\Lambda}\| \ldots \|k^{(n)}_{w,\Lambda} \quad \text{for } \Lambda \in \{0,1\}.$$

The garbled circuit then consists of ciphertexts encrypting each wire label and the colored bit $\boldsymbol{k}_{w,\Lambda_c}\|\Lambda_c$ under each party's wire labels $(k^{(i)}_{a,\Lambda_a}, k^{(i)}_{b,\Lambda_b})$.

To implement the above idea, a generic solution is to first compute a *secret sharing* of the wire label and colored bit $\boldsymbol{k}_{w,\Lambda_c(\Lambda_a,\Lambda_b)}\|\Lambda_c(\Lambda_a,\Lambda_b)$ to be encrypted for each choice of $\Lambda_a, \Lambda_b \in \{0,1\}$ by a generic MPC protocol following the Yao's garbled circuit. Then each party P_i locally encrypts his own share by $(k^{(i)}_{a,\Lambda_a}, k^{(i)}_{b,\Lambda_b})$ [HKN+25]. Now the encryption of $\boldsymbol{k}_{w,\Lambda_c(\Lambda_a,\Lambda_b)}\|\Lambda_c(\Lambda_a,\Lambda_b)$ contains n cipher-texts, one from each party. However, note that the size of $\boldsymbol{k}_{w,\Lambda_c(\Lambda_a,\Lambda_b)}\|\Lambda_c(\Lambda_a,\Lambda_b)$ is $\Omega(n\cdot\kappa)$ bits, which scales linearly with the number of parties. Then the share size as well as the cipher-text of each party also scales linearly with the number of parties. This results in $\Omega(n^2\kappa)$ bits of communication per gate in the above protocol.

Bottleneck of Adapting Previous Solutions with Honest Majority. As observed in [EGPS22, EGP+23], the standard honest majority setting where $n = 2t+1$ can be viewed as the dishonest majority setting with constant fraction gap in the corruption threshold, i.e., $t = (1-\epsilon)n$ with $\epsilon = 1/2$. In this setting [GPS22, EGP+23], it is possible to reduce a factor of n in the communication relying on *packed secret sharing* and suitable preprocessing data. Following this idea, the work [HKN+25] achieves $O(|C|n\kappa)$ communication in the {OT, RO}-hybrid model. However, their protocol uses OT and RO to prepare $O(|C|n)$ OLE (Oblivious Linear Evaluation) instances in $\mathbb{F}_{2^\kappa}$ with communication $O(|C|n\kappa)$ bits. In the standard honest majority setting, it is not known whether the same result can be achieved only from one-way functions. In particular, the best-known result [GLO+21] still requires at least $\Omega(|C|n^2\kappa)$ bits of communication when instantiating these OLE correlations.

Alternatively, another recent work [GLOS25] achieves a communication complexity of $O(|C|\kappa + D(n+\kappa)^2\kappa)$ bits from RO in the honest majority setting. Instead of following the BMR template, they use the technique of round collapsing to let each party garble his local computation in a general MPC. Their final results are achieved by replacing the parties with virtual parties formed by small committees. However, their result highly relies on the Free-XOR technique. Without Free-XOR, their construction requires garbling $\Omega(|C|(n+\kappa) + D(n+\kappa)^2)$ XOR gates, which requires an additional garbled circuit size of $\Omega(|C|(n+\kappa)\kappa + D(n+\kappa)^2\kappa)$. [GLOS25] also requires $\Omega(|C| + D(n+\kappa)^2)$ OT instances of message length $\Omega(\kappa)$. Again, in the standard honest majority setting, only assuming one-way functions, the best-known result [GLO+21] requires at least $\Omega(|C|n\kappa + D(n+\kappa)^2 n\kappa)$ bits of communication when instantiating these OT correlations. Besides, the work [GLOS25] also requires RO for compact and extractable commitments, which are not known how to achieve from one-way functions.

2.2 Efficient Multiparty Garbling via Party Virtualization

In the following, we introduce our new techniques that resolve the above mentioned difficulties. For simplicity, we first focus on the semi-honest security. We use $[x]_t$ to denote a degree-t Shamir sharing of the secret x.

Starting Point: Identifying the Main Difficulty. Our construction follows the aforementioned BMR template with degree-t Shamir sharings. At a high level, our construction can be separated into three steps. In the first step, all parties together compute degree-t Shamir sharings for the wire masks and colored bits.

1. For each wire w, all parties prepare a degree-t Shamir sharing $[\lambda_w]_t$ where λ_w is a random bit.
2. Then for each gate g with input wires a, b and output wire c, for each choice of $\Lambda_a, \Lambda_b \in \{0,1\}$, all parties compute a degree-t Shamir sharing of $\Lambda_c(\Lambda_a, \Lambda_b) = g(\Lambda_a \oplus \lambda_a, \Lambda_b \oplus \lambda_b) \oplus \lambda_c$.

We note that the above computation can be expressed by a circuit with constant depth and $O(|C|)$ multiplication gates. Thus, we can use a generic MPC protocol in the honest majority setting (such as [DN07, CGH+18]) which requires $O(|C|n\kappa + \mathsf{poly}(n, \kappa))$ communication and constant rounds.

In the second step, each party P_i randomly samples $(k^{(i)}_{w,0}, k^{(i)}_{w,1})$ for each wire w. Now for each gate g with input wires a, b and output wire c, and for each choice of $\Lambda_a, \Lambda_b \in \{0,1\}$, we want to compute a secret sharing of $\boldsymbol{k}_{c,\Lambda_c(\Lambda_a,\Lambda_b)}$ together with the colored bit $\Lambda_c(\Lambda_a, \Lambda_b)$. Note that all parties have already held a degree-t Shamir sharing of $\Lambda_c(\Lambda_a, \Lambda_b)$ in the previous step. To achieve a linear communication complexity, we follow [HKN+25] and try to compute a *packed Shamir sharing* of $\boldsymbol{k}_{c,\Lambda_c(\Lambda_a,\Lambda_b)}$. Here we use degree-$(n-1)$ packed Shamir sharings with privacy threshold t, which allows us to store $n - t = \Omega(n)$ secrets within a single sharing. For simplicity, we denote it by $[\boldsymbol{k}_{c,\Lambda_c(\Lambda_a,\Lambda_b)}]_{n-1}$ (Note that the size of $\boldsymbol{k}_{c,\Lambda_c(\Lambda_a,\Lambda_b)}$ is n, which cannot be stored within a single packed sharing. In our construction, we use 2 packed sharings to store the result).

In the third step, suppose all parties have already computed $[\boldsymbol{k}_{c,\Lambda_c(\Lambda_a,\Lambda_b)}]_{n-1}$. Following [HKN+25], each party P_i locally encrypts his share of $[\boldsymbol{k}_{c,\Lambda_c(\Lambda_a,\Lambda_b)}]_{n-1}$, which is a single field element of size κ, by $(k^{(i)}_{a,\Lambda_a}, k^{(i)}_{b,\Lambda_b})$, and sends the cipher-text to the evaluator. This accomplishes the computation of the garbled circuit. Note that this step only requires $O(|C|n\kappa)$ communication.

We observe that the main difficulty of achieving $O(|C|n\kappa)$ bits of communication is the second step. We first describe our solution with the aid of N additional parties (virtual parties) with corruption threshold $T < N/4$, denoted by $V_1, \ldots, V_N$. Jumping ahead, we will use the technique of party virtualization [Bra87] and each party V_i will be simulated by a pair of real parties.

Achieving the Second Step from Virtual Parties. Recall the computation task of the second step. For each gate g with input wires a, b and output wire c, and for each choice of $\Lambda_a, \Lambda_b \in \{0,1\}$,

- all parties hold a degree-t Shamir sharing $[\Lambda_c(\Lambda_a, \Lambda_b)]_t$,
- each party P_i holds a pair of wire labels $(k^{(i)}_{c,0}, k^{(i)}_{c,1})$.

The goal is to compute a packed Shamir sharing $[\boldsymbol{k}_{c,\Lambda_c(\Lambda_a,\Lambda_b)}]_{n-1}$, where $\boldsymbol{k}_{c,\Lambda_c(\Lambda_a,\Lambda_b)} = (k^{(i)}_{c,\Lambda_c(\Lambda_a,\Lambda_b)})_{i=1}^n$. We take $\Lambda_a = \Lambda_b = 0$ as an example. The other three cases can be handled similarly. We also abuse the notation $\Lambda_c = \Lambda_c(0,0)$ for simplicity.

Our idea is to delegate the computation to the virtual parties. Since the corruption threshold for virtual parties is only $1/4$, we can use packed Shamir secret sharing over virtual parties. Let $D = (N-1)/2$. Then we may use a degree-D packed Shamir secret sharing over virtual parties to store $K = D - T + 1 = \Omega(N)$ secrets within each sharing. Now for a group of K gates $g_1, \ldots, g_K$. Suppose the output wires are $c_1, \ldots, c_K$. All parties perform the following steps.

1. All parties convert $[\Lambda_{c_1}]_t, \ldots, [\Lambda_{c_K}]_t$ to a single packed Shamir sharing over virtual parties, denoted by $[\boldsymbol{\Lambda_c}]^V_D$. Here we use the superscript V to denote Shamir sharings over virtual parties.

   This step is done by letting each party P_i distribute a packed Shamir sharing over virtual parties for his K shares of $[\Lambda_{c_1}]_t, \ldots, [\Lambda_{c_K}]_t$. Since each Λ_{c_j} can be computed from a linear combination of all parties' shares of $[\Lambda_{c_j}]_t$, all virtual parties can locally compute $[\boldsymbol{\Lambda_c}]^V_D$.

2. Each party P_i distributes two packed Shamir sharings over virtual parties for $\boldsymbol{k}^{(i)}_{\boldsymbol{c},\boldsymbol{0}} = (k^{(i)}_{c_1,0}, \ldots, k^{(i)}_{c_K,0})$ and $\boldsymbol{k}^{(i)}_{\boldsymbol{c},\boldsymbol{1}} = (k^{(i)}_{c_1,1}, \ldots, k^{(i)}_{c_K,1})$, denoted by $[\boldsymbol{k}^{(i)}_{\boldsymbol{c},\boldsymbol{0}}]^V_D$ and $[\boldsymbol{k}^{(i)}_{\boldsymbol{c},\boldsymbol{1}}]^V_D$.

3. For each P_i, all virtual parties locally compute

$$[\boldsymbol{k}^{(i)}_{\boldsymbol{c},\boldsymbol{\Lambda_c}}]^V_{2D} = ([\boldsymbol{k}^{(i)}_{\boldsymbol{c},\boldsymbol{1}}]^V_D - [\boldsymbol{k}^{(i)}_{\boldsymbol{c},\boldsymbol{0}}]^V_D) \cdot [\boldsymbol{\Lambda_c}]^V_D + [\boldsymbol{k}^{(i)}_{\boldsymbol{c},\boldsymbol{0}}]^V_D.$$

4. All virtual parties jointly convert $[\boldsymbol{k}^{(1)}_{\boldsymbol{c},\boldsymbol{\Lambda_c}}]^V_{2D}, \ldots, [\boldsymbol{k}^{(n)}_{\boldsymbol{c},\boldsymbol{\Lambda_c}}]^V_{2D}$ to $[\boldsymbol{k}_{c_1,\Lambda_{c_1}}]_{n-1}, \ldots, [\boldsymbol{k}_{c_K,\Lambda_{c_K}}]_{n-1}$ over the real parties.

   This step is done by letting each virtual party V_i distribute a packed Shamir sharing over the real parties for his n shares of $[\boldsymbol{k}^{(1)}_{\boldsymbol{c},\boldsymbol{\Lambda_c}}]^V_{2D}, \ldots, [\boldsymbol{k}^{(n)}_{\boldsymbol{c},\boldsymbol{\Lambda_c}}]^V_{2D}$ (Again, we omit the issue that each packed Shamir sharing over real parties can only store $t+1$ values. In our construction, we use 2 packed sharings to store the result). Now for the ℓ-th entry in $\boldsymbol{k}_{c_j,\Lambda_{c_j}}$, it can be computed by a linear combination of shares of $[\boldsymbol{k}^{(\ell)}_{\boldsymbol{c},\boldsymbol{\Lambda_c}}]^V_{2D}$. In addition, for each j, the coefficients are the same for different ℓ. Thus, all parties can locally compute each of $[\boldsymbol{k}_{c_1,\Lambda_{c_1}}]_{n-1}, \ldots, [\boldsymbol{k}_{c_K,\Lambda_{c_K}}]_{n-1}$.

In the above process, parties (including both real parties and virtual parties) need to communicate $O(Nn\kappa)$ bits for K gates. Recall that $K = D - T + 1 = \Omega(N)$. The amortized communication per gate is $O(n\kappa)$ bits as desired.

Performing Virtual Parties' Computation. To instantiate the above idea, we employ the party virtualization technique [Bra87, DIK+08]. We let each pair of parties $(P_\alpha, P_\beta) \in \mathcal{P}^2$ simulate the computation of one virtual party by a secure two-party computation protocol. Thus, as long as P_α or P_β is honest, the view of this virtual party remains unknown to the adversary. Therefore, there are in total $N = n^2$ virtual parties, and the corruption threshold is $T = t^2 < N/4$.

Now we discuss in more detail how a pair of parties simulate the computation of one virtual party. Suppose a virtual party V_j is simulated by (P_α, P_β). We will maintain the invariant that P_α, P_β hold an additive sharing for each value in the view of V_j. Consider the above process.

- In Step 1 and Step 2, whenever a real party needs to send a message m to V_j, we ask this party to distribute an additive sharing of m to (P_α, P_β). Whenever the virtual party V_j needs to perform linear operations on values in his view, (P_α, P_β) operate on their individual shares locally.

- In Step 3, (P_α, P_β) use a secure two-party multiplication protocol to multiply V_j's shares of $([\boldsymbol{k}^{(i)}_{\boldsymbol{c},\boldsymbol{1}}]^V_D - [\boldsymbol{k}^{(i)}_{\boldsymbol{c},\boldsymbol{0}}]^V_D)$ and $[\boldsymbol{\Lambda_c}]^V_D$, and then perform the linear operations locally.

- In Step 4, when V_j needs to distribute a packed Shamir sharing of n values in his view, each of P_α and P_β distribute a packed Shamir sharing of their individual shares. Then the real parties add up these two packed Shamir sharings and take the result as the packed Shamir sharing distributed by V_j.

Since each virtual party is simulated by 2 real parties, the communication complexity in Steps 1,2,4 only blows up by a factor of 2. On the other hand, to maintain the overall $O(Nn\kappa)$ communication complexity, in Step 3, (P_α, P_β) need to perform n multiplications with $O(n\kappa)$ communication. Unfortunately, we do not know how to achieve this task in the standard honest majority setting only assuming one-way functions.

Our main observation is that in Step 3, V_j is supposed to multiply his share of $([\boldsymbol{k}^{(i)}_{\boldsymbol{c},\boldsymbol{1}}]^V_D - [\boldsymbol{k}^{(i)}_{\boldsymbol{c},\boldsymbol{0}}]^V_D)$ for each $i \in \{1, 2, \ldots, n\}$ with the same $[\boldsymbol{\Lambda_c}]^V_D$. Thus, it is sufficient to prepare random VOLE (Vector Oblivious Linear Evaluation) correlations for P_α, P_β where the vector size is $n\kappa$ bits. By using a PRG, we can prepare a random VOLE correlation with communication independent of the vector size.

VOLE Comes to Aid. To be more clear, suppose u_i denotes V_j's share of $([\boldsymbol{k}^{(i)}_{\boldsymbol{c},\boldsymbol{1}}]^V_D - [\boldsymbol{k}^{(i)}_{\boldsymbol{c},\boldsymbol{0}}]^V_D)$ and v denotes V_j's share of $[\boldsymbol{\Lambda_c}]^V_D$. Here, each u_i is of κ bits, whereas the size of v depends on the field size required by the secret sharing scheme. Then (P_α, P_β) together hold an additive sharing of each u_i and v, denoted by $\langle u_i \rangle = (u_{i,0}, u_{i,1}), \langle v \rangle = (v_0, v_1)$. The goal is to compute $\langle u_i \cdot v \rangle$ for all $i \in \{1, \ldots, n\}$. Since

$$u_i \cdot v = u_{i,0} \cdot v_0 + u_{i,0} \cdot v_1 + u_{i,1} \cdot v_0 + u_{i,1} \cdot v_1,$$

and P_α can locally compute $u_{i,0} \cdot v_0$ while P_β can locally compute $u_{i,1} \cdot v_1$, it is sufficient to let P_α, P_β compute $\langle u_{i,0} \cdot v_1 \rangle$ and $\langle u_{i,1} \cdot v_0 \rangle$.

We focus on the task of computing $\langle \boldsymbol{u}_0 \cdot v_1 \rangle = (\langle u_{i,0} \cdot v_1 \rangle)^n_{i=1}$. For simplicity, we assume that v_1 is just a single bit (In our construction we decompose v_1 bit by bit, and each time we multiply $u_{i,0}$ with a single bit of v_1). Then this can be done efficiently if P_α holds two random vectors $(\boldsymbol{r}_0, \boldsymbol{r}_1)$, each of size $n\kappa$ bits, and P_β holds a random bit b together with $\boldsymbol{r}_b$:

1. P_α sends $\boldsymbol{u}_0 + \boldsymbol{r}_0 + \boldsymbol{r}_1$ to P_β and P_β sends $v_1 + b$ to P_α.
2. P_α sets his share of $\langle \boldsymbol{u}_0 \cdot v_1 \rangle$ as $\boldsymbol{r}_{v_1+b}$. P_β computes his share by $v_1 \cdot (\boldsymbol{u}_0 + \boldsymbol{r}_0 + \boldsymbol{r}_1) + \boldsymbol{r}_b$.

To prepare $((k_0, k_1), (b, k_b))$, we may use a generic MPC protocol in the honest majority setting where all parties prepare random degree-t Shamir sharings $[k_0]_t, [k_1]_t, [b]_t$, then compute $[k_b]_t$ which requires one multiplication operation, and finally reconstruct $[k_0]_t, [k_1]_t$ to P_α and $[b]_t, [k_b]_t$ to P_β. The amortized communication complexity per VOLE correlation is $O(n\kappa)$ bits.

One subtlety in our construction is that the size of v_1 is not a single bit. As a result, we have to prepare $|v_1|$ random VOLE correlations, each of size κ bits. Due to the use of packed Shamir sharings over virtual parties, the size of v_1 needs to be at least $O(\log N) = O(\log n)$ (since the Shamir secret sharing scheme requires the size of the underlying field to be at least the number of shares), which leads the amortized communication complexity per gate to $O(n \log n \cdot \kappa)$ bits. To avoid the $\log n$ overhead, we instantiate the packed secret sharing by AG codes [CC06].

Summary of Our Construction. In summary, our construction starts from the BMR template with degree-t Shamir sharings in the standard honest majority setting. We use a generic MPC protocol with linear communication complexity to compute a degree-t Shamir sharing for each wire mask and colored bit.

In the second step, each party randomly samples a pair of wire labels for each wire in the circuit. Then the main computation task becomes the following. For each gate g with input wires a, b and output wire c, and for each choice of $\Lambda_a, \Lambda_b \in \{0, 1\}$,

- all parties hold a degree-t Shamir sharing $[\Lambda_c(\Lambda_a, \Lambda_b)]_t$,
- each party P_i holds a pair of wire labels $(k^{(i)}_{c,0}, k^{(i)}_{c,1})$.

The goal is to compute a packed Shamir sharing $[\boldsymbol{k}_{c,\Lambda_c(\Lambda_a,\Lambda_b)}]_{n-1}$, where $\boldsymbol{k}_{c,\Lambda_c(\Lambda_a,\Lambda_b)} = (k^{(i)}_{c,\Lambda_c(\Lambda_a,\Lambda_b)})^n_{i=1}$.

To achieve linear communication, we make use of the technique of party virtualization [Bra87, DIK+08] and delegate the computation to the virtual parties. Each virtual party is simulated by two real parties running a secure two-party computation protocol. The main difficulty is to achieve an amortized $O(\kappa)$ bits of communication between the two real parties for each multiplication operation when simulating this virtual party's local computation. Our main observation is that for each gate, the n multiplication operations in each virtual party's local computation share the same multiplicand. This allows us to reduce the computation task to preparing random VOLE correlations. Relying on PRG, we can prepare random VOLE correlations with communication complexity independent of the vector size.

After computing $[\boldsymbol{k}_{c,\Lambda_c(\Lambda_a,\Lambda_b)}]_{n-1}$, we follow [HKN+25] to let each party locally encrypt his share and send the ciphertext to the evaluator. This accomplishes the computation for the garbled circuit with linear communication.

2.3 Towards Malicious Security

To achieve malicious security, we first use a maliciously secure MPC protocol (such as [CGH+18]) in the first step to compute a degree-t Shamir sharing for each wire mask and colored bit. In the second step, we adapt the approach in [GLOS25] to verify the computation of virtual parties. Finally, the third step only involves local computation and sending ciphertexts to the evaluator. Without loss of generality, we may assume that the evaluator is always malicious. Thus, given the first two steps are correctly computed, the malicious security can be achieved following [HKN+25].

Below, we elaborate on how we adapt the approach in [GLOS25] to verify the computation in the second step. Recall that the second step can be further reduced into three sub-steps.

- First, all parties convert $[\Lambda_{c_1}]_t, \ldots, [\Lambda_{c_K}]_t$ to a single packed Shamir sharing over virtual parties, denoted by $[\boldsymbol{\Lambda_c}]_D^V$. Each party P_i also distributes two packed Shamir sharings of his wire labels, denoted by $[\boldsymbol{k}_{\boldsymbol{c},\boldsymbol{0}}^{(i)}]_D^V$ and $[\boldsymbol{k}_{\boldsymbol{c},\boldsymbol{1}}^{(i)}]_D^V$.

  In the meantime, all parties also prepare random VOLE correlations for each virtual party.

- Second, virtual parties locally compute

$$[\boldsymbol{k}_{\boldsymbol{c},\boldsymbol{\Lambda_c}}^{(i)}]_{2D}^V = ([\boldsymbol{k}_{\boldsymbol{c},\boldsymbol{1}}^{(i)}]_D^V - [\boldsymbol{k}_{\boldsymbol{c},\boldsymbol{0}}^{(i)}]_D^V) \cdot [\boldsymbol{\Lambda_c}]_D^V + [\boldsymbol{k}_{\boldsymbol{c},\boldsymbol{0}}^{(i)}]_D^V$$

  for all $i \in \{1, \ldots, n\}$. Here each virtual party's computation is simulated by a pair of real parties (P_α, P_β).

- Finally, virtual parties convert $[\boldsymbol{k}_{\boldsymbol{c},\boldsymbol{\Lambda_c}}^{(1)}]_{2D}^V, \ldots, [\boldsymbol{k}_{\boldsymbol{c},\boldsymbol{\Lambda_c}}^{(n)}]_{2D}^V$ to $[\boldsymbol{k}_{c_1,\Lambda_{c_1}}]_{n-1}, \ldots, [\boldsymbol{k}_{c_K,\Lambda_{c_K}}]_{n-1}$ over the real parties.

Adapting the Verification in [GLOS25]. When the adversary is malicious, corrupted parties may not share valid degree-D packed Shamir sharings over virtual parties in Step 1. Thus, in the second step, all virtual parties will in addition check the consistency of all degree-D packed Shamir sharings. This can be done by revealing a random linear combination of all degree-D packed Shamir sharings (To protect the secrecy, the revealed packed Shamir sharing should be masked by a random degree-D packed Shamir sharing. Here we omit it for simplicity).

However, the above is not sufficient for malicious security. This is because in our construction, we only use a semi-honest two-party computation protocol to simulate each virtual party's computation. If either P_α or P_β is corrupted, the security of V_j's computation is not guaranteed. In [GLOS25], a similar problem is addressed as follows.

- First, for each V_j, (P_α, P_β) commit their inputs and randomness used in the semi-honest two-party computation protocol.

- After simulating virtual parties' computation, each real party P_v checks a random subset of $O(N/n)$ virtual parties' computation. To check a virtual party V_j's computation, (P_α, P_β) reveal their view and open the commitments of their inputs and randomness to P_v.

Since there are $n/2$ honest parties, a constant fraction of the virtual parties' computation will be checked by honest parties. Intuitively, if for a constant fraction of virtual parties, corrupted parties deviate from the semi-honest two-party computation protocol, then it would be caught by one honest party with probability $1-O(2^{-N})$. To maintain the communication efficiency, in [GLOS25], they need to use a compact commitment scheme which is built from random oracles. Unfortunately, we do not know how to construct such a compact commitment scheme only from one-way functions.

On the other hand, in the honest majority setting, the degree-t Shamir secret sharing can be used as a *non-compact* commitment scheme:

- To commit a value m, the dealer distributes a degree-t Shamir secret sharing $[m]_t$.

- To open the commitment of m, parties send their shares of $[m]_t$ to the receiver.

Note that the secret of a degree-t Shamir sharing is determined by the shares of honest parties. Thus, corrupted parties cannot alter the secret without being detected. In our construction, we will ask all parties to compute degree-t Shamir sharings for the random VOLE correlations and virtual parties' shares of $[\boldsymbol{\Lambda_c}]_D^V$. Note that these are in total $O(N\kappa)$ bits. The communication complexity for committing those values is bounded by $O(Nn\kappa)$ bits.

However, we cannot do the same thing to the shares of $\{[\boldsymbol{k}_{\boldsymbol{c},\boldsymbol{0}}^{(i)}]_D^V, [\boldsymbol{k}_{\boldsymbol{c},\boldsymbol{1}}^{(i)}]_D^V\}_{i=1}^n$ for the virtual parties since it would lead to quadratic communication. For the shares generated by honest parties, we can additionally let the generator send these shares to the verifier. However, a corrupted generator may send wrong shares, so the shares generated from one corrupted party to another cannot be fixed before the check in this way. As a result, the adversary can distribute invalid degree-D sharings and modify corrupted parties' shares during verification to pass the check without being detected.

To resolve this issue, our main observation is that when simulating V_j's computation, given the VOLE correlation, P_α's input can be extracted from P_β's view, and vice versa. Thus, we may use P_β's view as a commitment of P_α's input, and vice versa. This allows us to avoid the use of a compact commitment scheme. After the local multiplications of virtual parties are performed, all the inputs for the local computations of honest virtual parties (who are not simulated by two corrupted parties) are fixed by the honest parties' view.

When P_v verifies V_j's computation, P_v will receive the inputs generated for them and the views of P_α and P_β, and then check whether both parties follow the computation correctly. We refer the readers to Section 6.4 for more details.

3 Preliminaries

Notations. We use $\boldsymbol{u} * \boldsymbol{v}$ to denote the coordinate-wise multiplication of two vectors $\boldsymbol{u}, \boldsymbol{v}$ of the same length, and we use $\boldsymbol{u} \otimes \boldsymbol{v}$ to denote the tensor product of two vectors, defined by

$$\boldsymbol{u} \otimes \boldsymbol{v} = (u_i \cdot v_j)_{i\in\{1,\ldots,k\},j\in\{1,\ldots,\ell\}} = (u_1v_1, \ldots, u_1v_\ell, \ldots, u_kv_1, \ldots, u_kv_\ell)$$

for $\boldsymbol{u} = (u_1, \ldots, n_k), \boldsymbol{v} = (v_1, \ldots, v_\ell)$. For two distributions A and B, we use $A \equiv B$ to denote that A and B have the same distribution. When $X = X_\kappa$ and $Y = Y_\kappa$ are two families of distributions indexed by a security parameter κ, we say that X and Y are computationally indistinguishable, denoted $X \equiv_c Y$, if for every polynomial $t(\cdot)$, $\max_{D\in\mathcal{D}_{t(\kappa)}} \Delta_D(X, Y) = \mathsf{negl}(\kappa)$. Here $\Delta_D(X, Y)$ is the advantage of a circuit D in distinguishing X and Y, defined by

$$\Delta_D(X, Y) = |\Pr[D(X) = 1] - \Pr[D(Y) = 1]|,$$

and $\mathcal{D}_t$ is the set of all probabilistic circuits of size t. We use U_m to denote the uniform distribution from $\{0, 1\}^m$.

3.1 The Security Model

We define the security of multiparty computation in the *real and ideal world paradigm* [Can00]. Informally, we consider a protocol Π to be secure if any adversary's view in its execution in the real world can also be simulated in the ideal world.

Real-World Execution. In the real world, there are n parties $P_1, \ldots, P_n$ and an adversary $\mathcal{A}$. We assume that every two parties are connected via a secure synchronous channel so that they can directly send messages to each other. At the beginning of the protocol, $\mathcal{A}$ chooses a set of at most t parties to be corrupted. During the execution of the protocol, an adversary $\mathcal{A}$ controlling corrupted parties interacts with honest parties. At the end of the protocol, the output of the real-world execution includes the inputs and outputs of honest parties and the view of the adversary.

Ideal-World Execution. In the ideal world, the parties interact with a trusted third party $\mathcal{F}$ that receives inputs from the parties, does the computation locally, and sends the outputs to the parties. There is an ideal-world adversary Sim who has one-time access to $\mathcal{F}$. Sim can send inputs and receive outputs for corrupt parties. Sim can also send abort_i to $\mathcal{F}$ to ask the trusted party to abort the output delivery to an honest party P_i. Sim emulates the ideal functionality, simulates the honest parties, and interacts with $\mathcal{A}$. The output of the ideal-world execution includes the inputs and outputs of honest parties, and the view of the adversary.

We say a protocol Π t-securely realizes $\mathcal{F}$ if there exists an ideal adversary Sim, such that for all adversaries $\mathcal{A}$, the distribution of the output of the real-world execution is computationally indistinguishable from the distribution of the output of the ideal-world execution.

3.2 Pseudorandom Generators and Pseudorandom Functions

We give the basic definition of *pseudorandom generators* (PRGs) that can be achieved with one-way functions.

Definition 1. Pseudorandom Generator (PRG) *A pseudorandom generator (PRG)* $\mathsf{PRG} : \{0,1\}^\kappa \to \{0,1\}^m$ *is a deterministic polynomial-time algorithm such that*

$$\{G(x) : x \xleftarrow{\$} \{0,1\}^\kappa\} \equiv_c U_m.$$

We also give the definition of pseudorandom functions (PRFs). A PRF can be constructed from a PRG $\mathsf{PRG} : \{0,1\}^m \to \{0,1\}^{2m}$.

Definition 2. Pseudorandom Function (PRF) *A pseudorandom function (PRF)* $F : \{0,1\}^\kappa \times \{0,1\}^s \to \{0,1\}^m$ *is a deterministic function such that*

- *Given a key* $k \in \{0,1\}^\kappa$, $F(k,\cdot)$ *can be computed in polynomial time.*
- $$\left\{F(k,\cdot) \mid k \xleftarrow{\$} \{0,1\}^\lambda\right\} \equiv_c \left\{f(\cdot) \mid f \xleftarrow{\$} \mathsf{Func}_s^m\right\},$$
  where Func_s^m *denotes the set of all functions from* $\{0,1\}^s$ *to* $\{0,1\}^m$.

3.3 Secret Sharings

In this section, we introduce the basic definitions of linear secret sharing schemes (LSSS).

Definition 3. (Projection Maps). *Let* $\boldsymbol{x} = (\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n) \in (\mathbb{F}_q^\ell)^n$. *Let* $A \subset \{1, \ldots, n\}$ *be a non-empty set. The projection map* $\pi_A : (\mathbb{F}_q^\ell)^n \to (\mathbb{F}_q^\ell)^{|A|}$ *is defined by* $\pi_A(\boldsymbol{x}) = (\boldsymbol{x}_i)_{i \in A}$.

Definition 4. (Linear Secret Sharing Schemes) *Let* $\mathbb{F}_q$ *be a finite field, and let* k, ℓ, *and* $t < n$ *be positive integers. An* (n, t, k, ℓ)*-linear secret sharing scheme (LSSS)* Σ *over* $\mathbb{F}_q$ *consists of two deterministic algorithms* $\Sigma.\mathsf{Sh}(\cdot,\cdot) : \mathbb{F}_q^k \times \mathbb{F}_q^{n\ell} \to (\mathbb{F}_q^\ell)^n$ *and* $\Sigma.\mathsf{Rec}(\cdot) : (\mathbb{F}_q^\ell)^n \to \mathbb{F}_q^k$. *For every* $\boldsymbol{s} \in \mathbb{F}_q^k$ *and* $\boldsymbol{r} \in \mathbb{F}_q^{n\ell}$, $\Sigma.\mathsf{Sh}(\boldsymbol{s}, \boldsymbol{r})$ *is a linear function that outputs a vector of shares* $(\boldsymbol{c}_1, \ldots, \boldsymbol{c}_n) \in (\mathbb{F}_q^\ell)^n$. *For any* $\boldsymbol{c} \in (\mathbb{F}_q^\ell)^n$ *which can be outputted by* $\Sigma.\mathsf{Sh}(\boldsymbol{s}, \boldsymbol{r})$ *for some* $\boldsymbol{r} \in \mathbb{F}_q^{n\ell}$, *we call* $\boldsymbol{c}$ *a* Σ*-sharing of* $\boldsymbol{s}$. *We require the following three properties.*

- *t-privacy: For all* $\boldsymbol{s}, \boldsymbol{s}' \in \mathbb{F}_q^k$ *and* $A \subset \{1, \ldots, n\}$ *of size* $\leq t$,
  $$\{\boldsymbol{r} \xleftarrow{\$} \mathbb{F}_q^{n\ell}, \boldsymbol{c} \leftarrow \Sigma.\mathsf{Sh}(\boldsymbol{s}, \boldsymbol{r}) : \pi_A(\boldsymbol{c})\} \equiv \{\boldsymbol{r}' \xleftarrow{\$} \mathbb{F}_q^{n\ell}, \boldsymbol{c}' \leftarrow \Sigma.\mathsf{Sh}(\boldsymbol{s}', \boldsymbol{r}') : \pi_A(\boldsymbol{c}')\}.$$
- *Reconstruction: For every* $\boldsymbol{s} \in \mathbb{F}_q^k$, *it holds that for any* Σ*-sharing* $\boldsymbol{c}$ *of* $\boldsymbol{s}$, $\Sigma.\mathsf{Rec}(\boldsymbol{c}) = \boldsymbol{s}$.
- *Linearity: The two algorithms* $\Sigma.\mathsf{Sh}(\cdot,\cdot) : \mathbb{F}_q^k \times \mathbb{F}_q^{n\ell} \to (\mathbb{F}_q^\ell)^n$ *and* $\Sigma.\mathsf{Rec}(\cdot) : \mathbb{F}_q^{n\ell} \to \mathbb{F}_q^k$ *are both* $\mathbb{F}_q$*-linear.*

In this work, we use two types of LSSSs, one is the well-known Shamir secret sharing scheme [Sha79], and the other is the sharing scheme based on AG codes [CC06]. We give the theorem of the latter one we used below.

Lemma 1. ([CC06]) *There exist constants ℓ, c such that for any integer n, $k \leq cn$ and $t \leq n/3$, there exists an (n, t, k, ℓ)-LSSS Σ and an (n, t, k, ℓ^2)-LSSS $\Sigma^{(2)}$ over $\mathbb{F}_2$ such that for any two Σ-sharings $[\![\boldsymbol{u}]\!], [\![\boldsymbol{v}]\!]$,*

$$[\![\boldsymbol{u} * \boldsymbol{v}]\!]^{(2)} = [\![\boldsymbol{u}]\!] \otimes [\![\boldsymbol{v}]\!]$$

is a valid $\Sigma^{(2)}$-sharing.

We use $[\cdot]_t$ to denote a degree-t Shamir sharing over $\mathbb{F}_{2^\kappa}$, and we use $[\![\cdot]\!]$ to denote a Σ-sharing for the LSSS Σ from Lemma 1. Moreover, we use $\langle\cdot\rangle$ to denote an additive sharing between 2 parties. Additionally, for an LSSS Σ (resp. $\Sigma^{(2)}$) over $\mathbb{F}_q$, a vector of m Σ-sharings $([\![\boldsymbol{s}_1]\!], \ldots, [\![\boldsymbol{s}_m]\!])$ (resp. $\Sigma^{(2)}$-sharings $([\![\boldsymbol{s}_1]\!]^{(2)}, \ldots, [\![\boldsymbol{s}_m]\!]^{(2)})$) can naturally be regarded as an LSSS $\Sigma_{\times m}$ (resp. $\Sigma^{(2)}_{\times m}$) over $\mathbb{F}_{q^m}$. $\Sigma_{\times m}$ (resp. $\Sigma^{(2)}_{\times m}$) is called an m-fold interleaved secret sharing scheme of Σ (resp. $\Sigma^{(2)}$) [CCXY18], we denote it by $[\![\cdot]\!]_{\times m}$ (resp. $[\![\cdot]\!]^{(2)}_{\times m}$).

Remark 1. *As noted in [GLOS25], for an (n, t, k, ℓ)-LSSS Σ, we have the following algorithms. There are algorithms that can efficiently determine whether a set S of shares comes from a valid Σ-sharing and give such a sharing (if it is valid). There are also algorithms that can efficiently sample a set of at most t_1 shares from another t_2 share ($t_1 + t_2 \leq T$) of a Σ-sharing without the secret. We refer the readers to [GLOS25] for more details.*

4 Subprotocols

General MPC. We give the functionality $\mathcal{F}_{\mathsf{MPC}}$ of a general (non-constant-round) MPC protocol that takes as inputs a circuit C and its inputs, and the functionality outputs degree-t Shamir sharings of the circuit outputs. The functionality can be realized with communication of $O(|C|n\kappa + n^2\kappa)$ bits in $O(\mathsf{depth}(C))$ rounds [CGH+18]. $\mathcal{F}_{\mathsf{MPC}}$ is presented in Figure 1.

Functionality $\mathcal{F}_{\mathsf{MPC}}$

Let $\mathcal{C}$ be the set of corrupted parties. Upon receiving (MPC, C) (where C is an arithmetic circuit over $\mathbb{F}_{2^\kappa}$) from all the n parties:

1. For each input wire w of C attached to P_i, the trusted party receives the input v_w on this wire from P_i.
2. Sim evaluates the circuit C with the inputs and obtains the wire value v_w for each output wire w of C. Then, the trusted party receives a set of corrupted parties' shares of $[v_w]_t$ from Sim and samples the degree-t Shamir sharing $[v_w]_t$ based on the secret and corrupted parties' shares. For each honest party P_i, if abort_i is received from $\mathcal{A}$, Sim sends abort to P_i. Otherwise, Sim sends P_i's share of $[v_w]_t$ for each output wire w to P_i.

Figure 1: Functionality for a General MPC.

Lemma 2. ([CGH+18]) *$\mathcal{F}_{\mathsf{MPC}}$ can be t-securely realized with statistical security against a fully malicious adversary with communication of $O(|C|n\kappa + n^2\kappa)$ bits in $O(\mathsf{depth}(C))$ rounds, where $|C|$ is the number of multiplication gates in C.*

Random Beaver Triples. We also give the functionality $\mathcal{F}_{\mathsf{triple}}$ of preparing random Beaver triples of the form $([a]_t, [b]_t, [c]_t)$ among all parties in $\mathcal{P}$. The functionality is presented in Figure 2.

Functionality $\mathcal{F}_{\mathsf{triple}}$

On receiving (Triple, L) from all the n parties, the trusted party repeats the following L times:

1. The trusted party randomly samples $a, b \in \mathbb{F}_{2^\kappa}$ and computes $c = a \cdot b$.
2. The trusted party receives corrupted parties' shares of $[a]_t, [b]_t, [c]_t$ from Sim and samples $[a]_t, [b]_t, [c]_t$ randomly based on corrupted parties' shares and the secret.

For each honest party P_i, if abort_i is received from $\mathcal{A}$, Sim sends abort to P_i. Otherwise, Sim sends P_i's shares of each triple $([a]_t, [b]_t, [c]_t)$ for each output wire w to P_i.

Figure 2: Functionality for preparing random Beaver triples.

The random triples can be prepared using $\mathcal{F}_{\mathsf{MPC}}$. More concretely, to compute each group of $n-t$ triples, each party inputs 2 random elements to $\mathcal{F}_{\mathsf{MPC}}$. The elements form 2 vectors in $\mathbb{F}_{2^\kappa}^n$, where the i-th entry is inputted by P_i. Then, by multiplying the 2 vectors with a $(n-t) \times n$ Vandermonde matrix, we will get $2(n-t)$ truly random elements that serve as a, b for $n-t$ triples. Then, c can be obtained by multiplying a, b. The above circuit of computing $n-t$ groups of (a, b, c) from the inputs only contains $n-t$ multiplication gates, so computing them via $\mathcal{F}_{\mathsf{MPC}}$ requires an amortized cost of $n\kappa$ bits. As a result, we give the following lemma.

Lemma 3. *$\mathcal{F}_{\mathsf{triple}}$ can be t-securely realized with statistical security against a fully malicious adversary with communication of $O(n^2\kappa + nL\kappa)$ bits in a constant number of rounds.*

Random Coin. We also need the standard functionality $\mathcal{F}_{\mathsf{Coin}}$ of generating a common coin in $\mathbb{F}_{2^\kappa}$, which is provided in Figure 3.

Functionality $\mathcal{F}_{\mathsf{Coin}}$

1. On receiving $\mathsf{RandCoin}$ from all the parties, the trusted party samples $s \in \mathbb{F}_{2^\kappa}$.
2. The trusted party sends s to Sim. For each honest party P_i, if abort_i is received from $\mathcal{A}$, Sim sends abort to P_i. Otherwise, the trusted party sends s to P_i.

Figure 3: Functionality for generating a common coin.

$\mathcal{F}_{\mathsf{Coin}}$ can be computed by letting each party generate a random degree-t Shamir sharing and then reconstruct the sum of them. We state the lemma below.

Lemma 4. *$\mathcal{F}_{\mathsf{Coin}}$ can be t-securely realized with perfect security against a fully malicious adversary with communication of $O(n^2\kappa)$ bits in 2 rounds.*

5 The Preprocessing for Our Protocol

Our main protocol is provided in 3 stages. We will first introduce the preprocessing protocol. Then, we will show how the virtual parties' local computations are performed. Finally, we will show how to generate the garbled circuit and how it is evaluated. In this section, we present the preprocessing for our main protocol.

Before describing the protocol, we first identify the public values for the protocol. All parties agree on $P_{j,1}, P_{j,2}$ who emulate the j-th virtual party V_j for $j = 1, \ldots, N$, where $N = n^2 \cdot m$. Where each pair of parties $(P_i, P_j) \in \mathcal{P}^2$ emulate $m = \Theta(\kappa/n^2)$ different virtual parties (such that $N = \Theta(\kappa + n^2)$). The parties agree on an evaluator P_{king}. Let Σ be an (N, T, K, ℓ)-LSSS over $\mathbb{F}_2$, where $K = \Theta(N)$, $T = N/3$, and ℓ is a constant (given by Lemma 1).

5.1 The Preprocessing Functionality

In the preprocessing phase of our protocol, we let the parties do the following:

1. The parties prepare a random degree-t Shamir sharing for a random bit for each wire. The secrets of the sharings serve as the random masking bits for the wire values. We also let the parties prepare $8N\ell$ auxiliary random degree-t bit sharings for each group of K gates. These sharings will serve as the randomness in generating Σ-sharings for the colored bits to the virtual parties.

2. The parties compute the degree-t sharings for the colored bits of each output wire of a gate.

We now give the preprocessing functionality.

Functionality $\mathcal{F}_{\mathsf{prep}}$

Upon receiving (Prep, C) from all the parties, the trusted party does the following:

1. **Generating Random Bit Sharings.** Let G be the number of gates in C and W be the number of wires in C. The trusted party receives a set of corrupted parties' shares of $W + 8GN\ell/K$ random degree-t bit sharings from Sim and samples the bit sharings based on these shares. Among them:
   - For each wire w of the circuit C, a random bit sharing serves as its mask sharing $[\lambda_w]_t$.
   - The other $8GN\ell/K$ random degree-t bit sharings serve as the auxiliary bit sharings to fix the randomness of generating Σ-sharings for the colored bits (see Step 3 of Π_{VPinput} in Figure 6). These sharings are grouped into $8G/K$ groups of $N\ell$ random bit sharings.
2. **Generating Sharings for Colored Bits.** For each gate g with input wires a, b and output wire c of C, suppose that the gate computes function $f_g : \{0,1\}^2 \to \{0,1\}$. The trusted party computes $\Lambda_{c,i}$ for $i = 1, 2, 3, 4$ where

$$\Lambda_{c,1} = f_g(0 \oplus \lambda_a, 0 \oplus \lambda_b) \oplus \lambda_c, \qquad \Lambda_{c,2} = f_g(0 \oplus \lambda_a, 1 \oplus \lambda_b) \oplus \lambda_c,$$

$$\Lambda_{c,3} = f_g(1 \oplus \lambda_a, 0 \oplus \lambda_b) \oplus \lambda_c, \qquad \Lambda_{c,4} = f_g(1 \oplus \lambda_a, 1 \oplus \lambda_b) \oplus \lambda_c.$$

   Then, the trusted party receives a set of corrupted parties' shares of $[\Lambda_{c,i}]_t$ for each $i = 1, 2, 3, 4$ from Sim. The trusted party samples the whole sharing $[\Lambda_{c,i}]_t$ based on the secret and the corrupted parties' sharing.

For each honest party P_i, if abort_i is received from $\mathcal{A}$, Sim sends abort to P_i. Otherwise, Sim sends P_i's shares of all the prepared sharings to P_i.

Figure 4: Functionality for preprocessing.

5.2 The Preprocessing Protocol

Now we present the preprocessing protocol Π_{Prep} which realizes $\mathcal{F}_{\mathsf{prep}}$ in Figure 5.

Protocol Π_{Prep}

The protocol is run between all the parties $P_1, \ldots, P_n$.

1. **Generating Random Bit Sharings.** Let W be the number of wires in C and G be the number of gates in C. The parties prepare $m_B = W + 8GN\ell/K$ random degree-t Shamir sharings of random bits as follows:
   (a) Each party P_i generates $m_B/(n-t) + \kappa$ random bit sharings $[b_1^{(i)}]_t, \ldots, [b_{m_B/(n-t)}^{(i)}]_t, [b^{(i,1)}]_t, \ldots, [b^{(i,\kappa)}]_t$ and distributes them among all the parties.
   (b) All the parties invoke $\mathcal{F}_{\mathsf{Coin}}$ by sending $\mathsf{RandCoin}$ (same for all the invocations of $\mathcal{F}_{\mathsf{Coin}}$ below) to get a random coin $r \in \mathbb{F}_{2^\kappa}$. The parties expand the coin using a PRG and get a common string in $\mathbb{F}_2^{m_B n\kappa/(n-t)}$. The parties split the string into $m_B n\kappa/(n-t)$ random bits $s_{\alpha,j}^{(\beta)}$ for $j = 1, \ldots, \kappa$, $\alpha = 1, \ldots, m_B/(n-t)$, and $\beta = 1, \ldots, n$.
   (c) All the parties compute

$$[\sigma_j]_t = \sum_{\beta=1}^{n} \sum_{\alpha=1}^{m_B/(n-t)} s_{\alpha,j}^{(\beta)} [b_\alpha^{(\beta)}]_t + \sum_{\beta=1}^{n} [b^{(\beta,j)}]_t$$

   for each $j = 1, \ldots, \kappa$. The parties then send their shares of $[\sigma_1]_t, \ldots, [\sigma_\kappa]_t$ to all the parties.

(d) The parties check whether $[\sigma_j]_t$ is a valid degree-t Shamir sharings and check whether each $\sigma_j \in \mathbb{F}_2 \subseteq \mathbb{F}_{2^\kappa}$ for $j = 1, \ldots, \kappa$. If not, the parties abort the protocol.

(e) Note that a Shamir sharing over $\mathbb{F}_{2^\kappa}$ is also $\mathbb{F}_2$ linear if we regard each share as a vector in $\mathbb{F}_2^\kappa \cong \mathbb{F}_{2^\kappa}$. Then, each bit sharing $[b]_t$ is regarded as an LSSS over $\mathbb{F}_2$, where each party's share is in $\mathbb{F}_2^\kappa$. Let $a = \lfloor \log n \rfloor + 1$. Then, a bit sharings can be regarded as an a-fold interleaved sharing, where each share is a vector over $\mathbb{F}_{2^a}$. We denote such a sharing by $[\cdot]^b_{\times a}$. To prepare each batch of $a(n-t)$ random bit sharings:

i. Each party P_i consumes a random bit sharings $[u_1^{(i)}]_t, \ldots, [u_a^{(i)}]_t$ generated by himself and regards them as an interleaved sharing $[u^{(i)}]^b_{\times a}$.

ii. Let $\mathcal{N}$ be the matrix

$$\mathcal{N} = \begin{pmatrix} 1 & 1 & \cdots & 1 \\ 1 & e_1 & \cdots & e_{n-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & e_1^{n-t-1} & \cdots & e_{n-1}^{n-t-1} \end{pmatrix},$$

where $1, e_1, \ldots, e_{n-1}$ are n different elements in $\mathbb{F}_{2^a}$. The parties locally compute their shares of

$$\begin{pmatrix} [u_1]^b_{\times a} \\ [u_2]^b_{\times a} \\ \vdots \\ [u_{n-t}]^b_{\times a} \end{pmatrix} = \mathcal{N} \cdot \begin{pmatrix} [u^{(1)}]^b_{\times a} \\ [u^{(2)}]^b_{\times a} \\ \vdots \\ [u^{(n)}]^b_{\times a} \end{pmatrix}.$$

iii. The parties locally transform $[u_1]^b_{\times a}, [u_2]^b_{\times a}, \ldots, [u_{n-t}]^b_{\times a}$ to $a(n-t)$ random bit sharings.

The first W random bit sharings serve as $[\lambda_w]_t$ for each wire w in C. The parties group the other $8GN\ell/K$ random bit sharings as the functionality does.

2. **Computing Sharings for Colored Bits.** The parties send (Triple, G_A) to $\mathcal{F}_{\mathsf{triple}}$, where G_A is the number of AND gates in C. If abort is received, all the parties abort the protocol. Otherwise, for each gate g with input wires a, b and output wire c, if the gate is an XOR gate, the parties locally compute $[\Lambda_{c,i}]_t, i = 1, 2, 3, 4$ from $[\lambda_a]_t, [\lambda_b]_t, [\lambda_c]_t$. If the gate is an AND gate:

(a) The parties consume a triple $([\alpha]_t, [\beta]_t, [\gamma]_t)$ prepared by $\mathcal{F}_{\mathsf{triple}}$ and compute $[\lambda_a - \alpha]_t, [\lambda_b - \beta]_t$ locally. Then, the parties sends their shares of $[\lambda_a - \alpha]_t, [\lambda_b - \beta]_t$ to P_{king}.

(b) P_{king} reconstructs $e = \lambda_a - \alpha, d = \lambda_b - \beta$ and sends them to all the parties. Then, all the parties locally compute

$$[\lambda_a \cdot \lambda_b]_t = e \cdot d + e \cdot [\beta]_t + d \cdot [\alpha]_t + [\gamma]_t.$$

After $[\lambda_a \cdot \lambda_b]_t$ for all the gates have been computed, the parties take $[e]_t, [d]_t$ reconstructed for all the AND gates, denoted by $\{[e_j]_t, [d_j]_t\}_{j=1}^{G_A}$. The parties invoke $\mathcal{F}_{\mathsf{Coin}}$ to get a random coin s and expand the coin using a PRG, obtaining $(s_1, \ldots, s_{2G_A}) \in \mathbb{F}_{2^\kappa}^{2G_A}$. Then the parties locally compute

$$[\tau]_t = \sum_{i=1}^{G_A} (s_i \cdot [e_i]_t + s_{G_A+i} \cdot [d_i]_t)$$

and send their shares to each party for reconstruction. The parties then check whether the sharing is valid and whether the secret matches $\sum_{i=1}^{G_A}(s_i \cdot e_i + s_{G_A+i} \cdot d_i)$ computed by each e_i, d_i sent from P_{king}. If not, the parties abort the protocol. Otherwise, the parties locally compute $[\Lambda_{c,i}]_t, i = 1, 2, 3, 4$ from $[\lambda_a]_t, [\lambda_b]_t, [\lambda_c]_t, [\lambda_a \cdot \lambda_b]_t$.

Figure 5: The protocol for preprocessing.

Lemma 5. *Π_{Prep} t-securely realizes $\mathcal{F}_{\mathsf{prep}}$ with computational security in the $\{\mathcal{F}_{\mathsf{Coin}}, \mathcal{F}_{\mathsf{triple}}\}$-hybrid model in a constant number of rounds with communication of $O(|C|n\kappa + n^2\kappa^2 + n^3\kappa)$ bits.*

From Lemma 3 and Lemma 4, $\mathcal{F}_{\mathsf{prep}}$ can be realized in the plain model in a constant number of rounds

with communication of $O(|C|n\kappa + n^2\kappa^2 + n^3\kappa)$ bits. We give the security proof in Section A and a detailed cost analysis in Section B.

6 Virtual Parties' Local Computations

In this section, we identify the local computation of each virtual party and show how to let all the parties help the virtual parties to perform their local computations.

6.1 Generating the Inputs for Virtual Parties

The virtual parties' inputs contain $\Sigma/\Sigma^{(2)}$-sharings of colored bits and wire labels. The parties also generate random Σ-sharings for future checks on the Σ-sharings' validity. In addition, we let the parties commit their "Watchlists" for verification by sharing them in this step. The generation process Π_{VPinput} for these inputs is given in Figure 6. This process is run after invoking $\mathcal{F}_{\mathsf{prep}}$.

Protocol Π_{VPinput}

The parties do the following:

1. **Committing Watchlists.** Each party P_i samples a random set Watch_i of $N/12n$ virtual parties and distributes a random degree-t Shamir secret sharing $[\mathsf{Watch}_i]_t$ to all the parties.
2. **Sharing the Wire Labels.** For each wire w, each party P_i samples two random wire labels $k_{w,0}^{(i)}, k_{w,1}^{(i)} \in \mathbb{F}_2^\kappa$. For each group of K gates (for simplicity, we denote them by $g_1, \ldots, g_K$ with output wires $c_1, \ldots, c_K$), let $k_{c_j,\beta}^{(i)} = (k_{c_j,\beta}^{(i,1)}, \ldots, k_{c_j,\beta}^{(i,\kappa)}) \in \mathbb{F}_2^\kappa$ for each $j = 1, \ldots, K$ and $\beta = 0, 1$, and let $\boldsymbol{k}_{\boldsymbol{\beta}}^{(i,\alpha)} = (k_{c_1,\beta}^{(i,\alpha)}, \ldots, k_{c_K,\beta}^{(i,\alpha)})$ for each $\alpha = 1, \ldots, \kappa$. Then, each party P_i generates a random Σ-sharing $[\![\boldsymbol{k}_{\boldsymbol{1}}^{(i,\alpha)} - \boldsymbol{k}_{\boldsymbol{0}}^{(i,\alpha)}]\!]$ and a random $\Sigma^{(2)}$-sharing $[\![\boldsymbol{k}_{\boldsymbol{0}}^{(i,\alpha)}]\!]^{(2)}$ for each $\alpha = 1, \ldots, \kappa$ based on the secrets. Finally, the parties distribute the sharings to the virtual parties (where each share of these sharings is shared by two members of a virtual party via a random additive sharing).
3. **Sharing Colored Bits to Virtual Parties.** For each group of K gates $g_1, \ldots, g_K$ with output wires $c_1, \ldots, c_K$, recall that each party holds his shares of $[\Lambda_{c_j,i}]_t$ for $j = 1, \ldots, K$ and $i = 1, 2, 3, 4$ after invoking $\mathcal{F}_{\mathsf{prep}}$. For each $i = 1, 2, 3, 4$:
   (a) The parties consume 2 groups of random bit sharings prepared in Step 1 of the preprocessing phase. Let the 2 groups of sharings be $[b_1]_t, \ldots, [b_{N\ell}]_t$ and $[b'_1]_t, \ldots, [b'_{N\ell}]_t$.
   (b) The parties jointly compute the degree-t sharings for the Σ-sharing $[\![\boldsymbol{\Lambda}_i]\!]$ where $\boldsymbol{\Lambda}_i = (\Lambda_{c_1,i}, \ldots, \Lambda_{c_K,i})$ generated with randomness $b_1, \ldots, b_{N\ell}$. More concretely, the parties hold $[\Lambda_{c_j,i}]_t$ for $j = 1, \ldots, K$ and $[b_1]_t, \ldots, [b_{N\ell}]_t$, they compute the linear function $\Sigma.\mathsf{Sh}$ with these sharings and get the degree-t Shamir sharing for $\Sigma.\mathsf{Sh}(\boldsymbol{\Lambda}_i, (b_1, \ldots, b_{N\ell}))$.
   (c) For each virtual party V_j's share $\boldsymbol{\Lambda}_i^{V_j}$ of $[\![\boldsymbol{\Lambda}_i]\!]$, $P_{j,1}$'s share of $\langle \boldsymbol{\Lambda}_i^{V_j} \rangle$ is set to be b'_j and $P_{j,2}$'s share is computed accordingly. Specifically, the parties send their shares of $[b'_j]_t$ to $P_{j,1}$ and compute the degree-t sharing of $P_{j,2}$'s share by $[\boldsymbol{\Lambda}_i^{V_j}]_t - [b'_j]_t$. Each member then checks whether he received a valid degree-t Shamir sharing. If not, he aborts the protocol.
4. **Preparing the Verification of Sharings.** The parties do the following to prepare for the verification of the Σ-sharings $\{[\![\boldsymbol{k}_{\boldsymbol{1}}^{(i,\alpha)} - \boldsymbol{k}_{\boldsymbol{0}}^{(i,\alpha)}]\!]\}_{i\in\{1,\ldots,n\},\alpha\in\{1,\ldots,\kappa\}}$ for each group of gates. The parties view the sharings to be checked as $\Sigma_{\times\kappa}$-sharings $[\![\boldsymbol{x}_1]\!]_{\times\kappa}, \ldots, [\![\boldsymbol{x}_{k_1}]\!]_{\times\kappa}$ (by simply embedding each Σ-sharing into a $\Sigma_{\times\kappa}$-sharing). Each party P_i generates a random $\Sigma_{\times\kappa}$-sharing $[\![\boldsymbol{r}^{(i)}]\!]_{\times\kappa}$ (including the additive sharings for the members of its shares) and distributes a degree-t Shamir sharing for each share of this $\Sigma_{\times\kappa}$-sharing among all the parties.

Figure 6: The protocol for generating virtual parties' input.

6.2 The Local Computations of Virtual Parties

Now we identify the local computation process of the virtual parties that needs to be checked. More concretely, the virtual parties first do local computations to obtain their shares of each $[\![\boldsymbol{k}_{\boldsymbol{\Lambda}_\beta}^{(i,\alpha)}]\!]^{(2)}$ for each group of K gates and each $\beta = 1, 2, 3, 4$. Then, the virtual parties compute a random linear combination of the Σ-sharings of the form $[\![\boldsymbol{k}_{\boldsymbol{1}}^{(i,\alpha)} - \boldsymbol{k}_{\boldsymbol{0}}^{(i,\alpha)}]\!]$ to let all the parties check whether these Σ-sharings are correctly distributed.

Protocol Π_{VPcomp}

For each group of K gates and each $\beta = 1, 2, 3, 4$, the parties and virtual parties do the following:

1. **Computing Multiplications.** Each virtual party V_j computes its share of
$$[\![\boldsymbol{k}_{\boldsymbol{\Lambda}_\beta}^{(i,\alpha)}]\!]^{(2)} = [\![\boldsymbol{\Lambda}_\beta]\!] \otimes [\![\boldsymbol{k}_{\boldsymbol{1}}^{(i,\alpha)} - \boldsymbol{k}_{\boldsymbol{0}}^{(i,\alpha)}]\!] + [\![\boldsymbol{k}_{\boldsymbol{0}}^{(i,\alpha)}]\!]^{(2)}$$
for $\alpha = 1, \ldots, \kappa$ and $i = 1, \ldots, n$.
2. **Verification of the Sharings.**
   (a) The parties invoke $\mathcal{F}_{\mathsf{Coin}}$ to get $r \in \mathbb{F}_{2^\kappa}$. If abort is received, the parties abort the protocol. Then, the parties expand r to a vector $(r_1, \ldots, r_{k_1}) \in \mathbb{F}_{2^\kappa}^{k_1}$ using a PRG. The parties then send the degree-t sharings of each virtual party's shares of $[\![\boldsymbol{r}^{(1)}]\!]_{\times\kappa}, \ldots, [\![\boldsymbol{r}^{(n)}]\!]_{\times\kappa}$ (prepared in Step 4 of Π_{VPinput}) to the virtual party. The members of virtual parties check whether they received valid degree-t sharings. If not, the members abort the protocol. Otherwise, the virtual party reconstructs its shares.
   (b) Each virtual party V_α locally computes its share of $[\![\sigma]\!]_{\times\kappa} = \sum_{j=1}^{k_1} r_j \cdot [\![\boldsymbol{x}_j]\!]_{\times\kappa} + \sum_{i=1}^{n} [\![\boldsymbol{r}^{(i)}]\!]_{\times\kappa}$. The virtual party V_α then sends V_α's share of $[\![\sigma]\!]_{\times\kappa}$ to all the parties.

Figure 7: The protocol for virtual parties' local computations.

6.3 Performing Local Computations

We now introduce how the local computations of the virtual parties can be performed. Note that during the local computation process of the virtual parties, except the computation of $[\![\boldsymbol{\Lambda}_\beta]\!] \otimes [\![\boldsymbol{k}_{\boldsymbol{1}}^{(i,\alpha)} - \boldsymbol{k}_{\boldsymbol{0}}^{(i,\alpha)}]\!]$, all the local computations are linear, which can be computed locally by the members (by performing the linear operations on their additive sharings of virtual parties' data). To compute $[\![\boldsymbol{\Lambda}_\beta]\!] \otimes [\![\boldsymbol{k}_{\boldsymbol{1}}^{(i,\alpha)} - \boldsymbol{k}_{\boldsymbol{0}}^{(i,\alpha)}]\!]$, for each group of gates and $i_1, i_2 \in \{1, \ldots, \ell\}$, each virtual party needs to compute a multiplication of the i_1-th bit of V_j's share of $[\![\boldsymbol{\Lambda}_\beta]\!]$ and the vector formed by the i_2-th bits of V_j's shares of $[\![\boldsymbol{k}_{\boldsymbol{1}}^{(i,\alpha)} - \boldsymbol{k}_{\boldsymbol{0}}^{(i,\alpha)}]\!]$ for $\alpha = 1, \ldots, \kappa$ and $i = 1, \ldots, n$. In total, each virtual party needs to compute $4\ell^2 = O(1)$ multiplications of a bit and a vector in $\mathbb{F}_2^{n\kappa}$. Such multiplications are performed with the help of oblivious transfers (OTs) for long strings. The random string OTs are expanded from seeds prepared by random oblivious transfer (ROT) correlations using a PRG. The ROT correlations (together with degree-t sharings of the ROT correlations that are used for verification) for all party pairs are prepared by the functionality $\mathcal{F}_{\mathsf{ROT}}$ given as follows in Figure 8.

Functionality $\mathcal{F}_{\mathsf{ROT}}$

Upon receiving (ROT, L) from all the parties, the trusted party does the following L times for each $(P_i, P_j) \in \mathcal{P}^2$:

1. The trusted party randomly samples two messages $k_0, k_1 \in \mathbb{F}_{2^\kappa}$ and a bit $b \in \{0, 1\}$. If P_i is corrupted, k_0, k_1 are chosen by Sim. If P_j is corrupted, b, k_b are chosen by Sim.
2. The trusted party receives a set of corrupted parties' shares of $[k_0]_t, [k_0]_t, [b]_t, [k_b]_t$ from Sim. Then, the trusted party samples the whole sharings $[k_0]_t, [k_0]_t, [b]_t, [k_b]_t$ based on the secrets and the corrupted parties' shares.
3. The trusted party sets k_0, k_1 as outputs to P_i and sets b, k_b as outputs to P_j. In addition, each party's shares of $[k_0]_t, [k_0]_t, [b]_t, [k_b]_t$ are also added into this party's output.

For each honest party P_i, if abort_i is received from Sim, the trusted party sends abort to P_i. Otherwise, the trusted party sends P_i's output to P_i.

Figure 8: Functionality for generating random oblivious transfers.

We follow [SY25] and use a general MPC to compute the functionality $\mathcal{F}_{\mathsf{ROT}}$. We give the instantiation Π_{ROT} as follows.

Protocol Π_{ROT}

The protocol is run between all the parties $P_1, \ldots, P_n$. The parties run the following process L times in parallel:

1. For each $(P_i, P_j) \in \mathcal{P}^2$, P_j sends a random bit $b^{(i,j)}$ to $\mathcal{F}_{\mathsf{MPC}}$ as an input.
2. All parties invoke $\mathcal{F}_{\mathsf{MPC}}$ to compute their shares of $[b^{(i,j)} \cdot (b^{(i,j)} - 1)]_t$ for all $(P_i, P_j) \in \mathcal{P}^2$ and reconstruct the output to P_i who checks whether the reconstructed result equals 0. If not, P_i aborts the protocol.
3. For each $(P_i, P_j) \in \mathcal{P}^2$, P_i randomly samples $k_0^{(i,j)}, k_1^{(i,j)} \in \mathbb{F}_{2^\kappa}$ and sends them to $\mathcal{F}_{\mathsf{MPC}}$ as inputs.
4. All the parties invoke $\mathcal{F}_{\mathsf{MPC}}$ to compute their shares of $[k_{b^{(i,j)}}^{(i,j)}]_t$ with $k_{b^{(i,j)}}^{(i,j)} = k_0^{(i,j)} \cdot (1 - b^{(i,j)}) + k_1^{(i,j)} \cdot b^{(i,j)}$ for all $(P_i, P_j) \in \mathcal{P}^2$. Finally, the parties hold $[k_0^{(i,j)}]_t, [k_1^{(i,j)}]_t, [b^{(i,j)}]_t, [k_{b^{(i,j)}}^{(i,j)}]_t$.
5. For each $(P_i, P_j) \in \mathcal{P}^2$, the parties send their shares of $[k_0^{(i,j)}]_t, [k_1^{(i,j)}]_t$ to P_i and send their shares of $[b^{(i,j)}]_t, [k_{b^{(i,j)}}^{(i,j)}]_t$ to P_j. Finally, P_i, P_j reconstruct the secrets.

Figure 9: The protocol for preparing ROTs.

Π_{ROT} realizes $\mathcal{F}_{\mathsf{ROT}}$ in the $\mathcal{F}_{\mathsf{MPC}}$-hybrid model with communication of $O(Ln^3\kappa)$ bits, where the size of the circuits inputted to $\mathcal{F}_{\mathsf{MPC}}$ is $O(Ln^2\kappa)$ and the depth of the circuit inputted to $\mathcal{F}_{\mathsf{MPC}}$ is a constant. From Lemma 2, we obtain the following lemma.

Lemma 6. *$\mathcal{F}_{\mathsf{ROT}}$ can be realized with statistical security in constant rounds against a fully malicious adversary that corrupts at most t parties with communication of $O(Ln^3\kappa)$ bits.*

For each ROT correlation $((k_0, k_1), (b, k_b))$, the parties expand the seeds to vectors in $\mathbb{F}_2^{n\kappa}$ and obtain a random string OT correlations $((\boldsymbol{u}_0, \boldsymbol{u}_1), (b, \boldsymbol{u}_b))$. During each local multiplication of a virtual party, the members consume two string OTs to compute $\langle \boldsymbol{x} \cdot y \rangle$ from $\langle \boldsymbol{x} \rangle$ and $\langle y \rangle$. We now present the protocol Π_{Mult} to let each virtual party perform a local multiplication in Figure 10.

Protocol Π_{Mult}

The protocol runs between two parties. For simplicity, we assume the two parties are P_1, P_2.

Inputs: P_1, P_2 input additive sharings $\langle \boldsymbol{x} \rangle = (\boldsymbol{x}_1, \boldsymbol{x}_2), \langle y \rangle = (y_1, y_2)$, where $\boldsymbol{x} \in \mathbb{F}_2^{n\kappa}$ and $y \in \mathbb{F}_2$. P_1, P_2 also input two ROT relations received from $\mathcal{F}_{\mathsf{ROT}}$. For the first one, P_1 inputs $k_0^{(1)}, k_1^{(1)} \in \mathbb{F}_{2^\kappa}$ and P_2 inputs $b^{(1)} \in \mathbb{F}_2, k_{b^{(1)}}^{(1)}$. For the other one, P_2 inputs $k_0^{(2)}, k_1^{(2)} \in \mathbb{F}_{2^\kappa}$ and P_1 inputs $b^{(2)} \in \mathbb{F}_2, k_{b^{(2)}}^{(2)}$.

1. P_1 expands $k_0^{(1)}, k_1^{(1)}, k_{b^{(2)}}^{(2)}$ and obtains $\boldsymbol{u}_0^{(1)}, \boldsymbol{u}_1^{(1)}, \boldsymbol{u}_{b^{(2)}}^{(2)}$ respectively. P_2 expands $k_0^{(2)}, k_1^{(2)}, k_{b^{(1)}}^{(1)}$ and obtains $\boldsymbol{u}_0^{(2)}, \boldsymbol{u}_1^{(2)}, \boldsymbol{u}_{b^{(1)}}^{(1)}$ respectively.
2. P_1 sends $\boldsymbol{u}_1^{(1)} + \boldsymbol{u}_0^{(1)} + \boldsymbol{x}_1$ and $y_1 + b^{(2)}$ to P_2. P_2 sends $y_2 + b^{(1)}$ and $\boldsymbol{u}_1^{(2)} + \boldsymbol{u}_0^{(2)} + \boldsymbol{x}_2$ to P_1.
3. P_1 locally computes $\boldsymbol{z}_1^{(1)} = \boldsymbol{u}_{y_2+b^{(1)}}^{(1)}$ and $\boldsymbol{z}_1^{(2)} = (\boldsymbol{u}_1^{(2)} + \boldsymbol{u}_0^{(2)} + \boldsymbol{x}_2) \cdot y_1 + \boldsymbol{u}_{b^{(2)}}^{(2)}$. P_2 locally computes $\boldsymbol{z}_2^{(1)} = (\boldsymbol{u}_1^{(1)} + \boldsymbol{u}_0^{(1)} + \boldsymbol{x}_1) \cdot y_2 + \boldsymbol{u}_{b^{(1)}}^{(1)}$ and $\boldsymbol{z}_2^{(2)} = \boldsymbol{u}_{y_1+b^{(2)}}^{(2)}$. Then we have $\boldsymbol{z}_1^{(1)} + \boldsymbol{z}_2^{(1)} = \boldsymbol{x}_1 \cdot y_2$ and $\boldsymbol{z}_1^{(2)} + \boldsymbol{z}_2^{(2)} = \boldsymbol{x}_2 \cdot y_1$.
4. P_1 computes $\boldsymbol{z}_1 = \boldsymbol{x}_1 \cdot y_1 + \boldsymbol{z}_1^{(1)} + \boldsymbol{z}_1^{(2)}$ and P_2 computes $\boldsymbol{z}_2 = \boldsymbol{x}_2 \cdot y_2 + \boldsymbol{z}_2^{(1)} + \boldsymbol{z}_2^{(2)}$. Then, P_1, P_2 hold $\langle \boldsymbol{z} \rangle = (\boldsymbol{z}_1, \boldsymbol{z}_2)$ where $\boldsymbol{z} = \boldsymbol{x} \cdot y$.

Figure 10: The protocol for a virtual party's multiplication.

The communication cost of Π_{Mult} is $O(n\kappa)$ bits. Now, the virtual parties are able to perform the local computations in Π_{VPcomp} with communication of $O(N \cdot n\kappa)$ bits for each group of gates.

6.4 Verifying Virtual Parties' Local Computations

Finally, we present how to ensure that enough virtual parties perform their local computations honestly. We also complete the verification of the Σ-sharings of wire labels for the virtual parties in this step. To verify the local computations of the virtual parties, we let all the parties open their "Watchlists" and let the members open their inputs and transcripts in Π_{VPcomp}. The parties can then check the local computations performed by the virtual parties on their "Watchlist". We give the protocol Π_{VPver} in Figure 11.

Protocol Π_{VPver}

1. **Verifying the Sharings.** Each party reconstructs $[\![\sigma]\!]_{\times\kappa}$ from the virtual parties' outputs and sends them to all the other parties. Then, all the parties check whether the results received from all the parties are the same and whether $[\![\sigma]\!]_{\times\kappa}$ is a valid $\Sigma_{\times\kappa}$-sharing. If not, the parties abort the protocol.
2. **Opening Watchlists.** The parties send their shares of $[\mathsf{Watch}_1]_t, \ldots, [\mathsf{Watch}_n]_t$ to all the parties. Each party checks whether they are all valid degree-t sharings. If not, the parties abort the protocol. Otherwise, each party reconstructs $\mathsf{Watch}_1, \ldots, \mathsf{Watch}_n$.
3. **Opening Transcripts.** For each virtual party $V_j \in \mathsf{Watch}_i$, the members $P_{j,1}, P_{j,2}$ send all the transcripts during the executions of Π_{Mult} for local multiplications of V_j to P_i.
4. **Opening Inputs.** For each virtual party $V_j \in \mathsf{Watch}_i$:
   (a) For each party P_α, the members $P_{j,1}, P_{j,2}$ send their shares of (the additive sharing of V_j's share of) each sharing in $\{[\![\boldsymbol{k}_{\mathbf{1}}^{(\alpha,\beta)} - \boldsymbol{k}_{\mathbf{0}}^{(\alpha,\beta)}]\!], [\![\boldsymbol{k}_{\mathbf{0}}^{(\alpha,\beta)}]\!]^{(2)}\}_{\beta=1}^{\kappa}$ (generated in Π_{VPinput} by P_α) for V_j's local computation to P_i. P_i also receives these shares from P_α and checks whether these shares match. If not, P_i aborts the protocol.
   (b) The parties send the degree-t sharings for $P_{j,1}, P_{j,2}$'s shares of each $\langle \boldsymbol{\Lambda}_i^{V_j} \rangle$ to P_i. P_i checked whether they are valid degree-t sharings. If not, P_i aborts the protocol.
   (c) The parties send the degree-t sharings for (the additive sharings for) V_j's shares of $[\![\boldsymbol{r}^{(1)}]\!]_{\times\kappa}, \ldots, [\![\boldsymbol{r}^{(n)}]\!]_{\times\kappa}$ to P_i. P_i checks whether they are valid degree-t sharings. If not, P_i aborts the protocol.
5. **Opening ROTs.** For each virtual party $V_j \in \mathsf{Watch}_i$, all the parties send their shares of the degree-t sharings of the ROT correlations prepared for V_j to P_i. P_i checks whether the received sharings are all valid. If not, P_i aborts the protocol.
6. **Checking the Computations.** Each party P_i checks whether each virtual party $V_j \in \mathsf{Watch}_i$ performs its computation honestly from the inputs for V_j's local computation (including checking the executions of Π_{Mult} and the computation process of V_j's share of $[\![\sigma]\!]_{\times\kappa}$). If not, P_i aborts the protocol.

Figure 11: Verification of the virtual parties' local computations.

7 Garbling and Evaluation

After the virtual parties perform their local computation, the parties are ready to generate the garbled circuit and let the evaluator evaluate it. We now show how the garbling and evaluation processes are performed.

7.1 Generating the Garbled Circuit

During the garbling process of the circuit, the parties compute their degree-$d = n-1$ packed secret sharings (PSSs) for the wire labels of each gate and encrypt their shares. We give the protocol in Figure 12.

Protocol Π_{Garble}

1. **Transforming to PSSs of Each Gate's Labels.** Recall that each virtual party holds its shares of $[\![\boldsymbol{k}_{\boldsymbol{\Lambda}_\beta}^{(i,\alpha)}]\!]^{(2)}$ for $i = 1, \ldots, n$, $\beta = 1, 2, 3, 4$, and $\alpha = 1, \ldots, \kappa$. For each wire w, let $\boldsymbol{k}_{w,i} = k_{w,i}^{(1)} \| \ldots \| k_{w,i}^{(n)}$ for each $i = 0, 1$. For each group of gates and $\beta = 1, 2, 3, 4$:
   (a) Each member of a virtual party V_j distributes random degree-d Shamir sharings of his shares of (the additive sharing of V_j's shares of) $[\![\boldsymbol{k}_{\boldsymbol{\Lambda}_\beta}^{(i,\alpha)}]\!]^{(2)}$ for $i = 1, \ldots, n$ and $\alpha = 1, \ldots, \kappa$ to all the parties.
   (b) The parties jointly compute $[\boldsymbol{k}_{c,\Lambda_{c,\beta}}]_d$ for each output wire c of a gate in this group. More concretely, the reconstruction algorithm $\Sigma^{(2)}.\mathsf{Rec}$ is a linear function. The parties simply add up the degree-d sharings generated by the members $P_{j,1}, P_{j,2}$ of each virtual member V_j and get the $\Sigma^{(2)}$-sharing of $[\boldsymbol{k}_{c,\Lambda_{c,\beta}}]_d$. Then, the parties apply the reconstruction algorithm on the degree-d shared data and get $[\boldsymbol{k}_{c,\Lambda_{c,\beta}}]_d$[1].
2. **Encrypting the Shares.** For each gate g with input wires a, b and c, the parties hold $[\boldsymbol{k}_{c,\Lambda_{c,i}}]_d$ for each $i = 1, 2, 3, 4$. Let P_j's share of $[\boldsymbol{k}_{c,\Lambda_{c,i}}]_d$ be $\mathsf{Sh}_{c,i}^{(j)}$. For each gate g in the circuit C, each party P_j computes
$$\begin{aligned}\mathsf{ct}_{g,2i_1+i_2}^{(j)} = \mathsf{Sh}_{c,2i_1+i_2}^{(j)} &\oplus \mathsf{PRF}(k_{a,i_1}^{(j)}, (2i_1+i_2)\|i_1\|j\|g) \\ &\oplus \mathsf{PRF}(k_{b,i_2}^{(j)}, (2i_1+i_2)\|i_2\|j\|g)\end{aligned}$$
   for each $i_1 = 0, 1$ and $i_2 = 0, 1$.
3. **Sending Ciphertexts.** Each party P_j sends $\mathsf{ct}_{g,1}^{(j)}, \mathsf{ct}_{g,2}^{(j)}, \mathsf{ct}_{g,3}^{(j)}, \mathsf{ct}_{g,4}^{(j)}$ for each gate g to P_{king}.

[1]Recall that $\boldsymbol{k}_{c,\Lambda_{c,\beta}} \in \mathbb{F}_{2^\kappa}^n$ while the secret size of a degree-d PSS is at most $(t+1)\kappa$, we actually need 2 degree-d PSSs for each $[\boldsymbol{k}_{c,\Lambda_{c,\beta}}]_d$. For simplicity, we write it as a single degree-d sharing.

Figure 12: The garbling phase of the main protocol.

7.2 Evaluation of the Garbled Circuit

The evaluation process is presented in Figure 13, where the evaluator P_{king} receives input labels, evaluates the garbled circuit, and sends the output labels to the parties. Finally, the parties obtain their outputs.

Protocol Π_{Eval}

The parties do the following:

1. **Computing Masked Inputs.**
   (a) For each input wire w attached to a party P_i, all the parties send their shares of $[\lambda_w]_t$ to P_i. P_i checks whether $[\lambda_w]_t$ is a valid degree-t Shamir sharing. If not, P_i aborts the protocol. Otherwise, P_i reconstructs λ_w and computes $v_w \oplus \lambda_w$ with his input v_w to this wire. Then, P_i sends $v_w \oplus \lambda_w$ to all the parties.
   (b) The parties invoke $\mathcal{F}_{\mathsf{Coin}}$ to get a random coin $s \in \mathbb{F}_{2^\kappa}$ and expand it using a PRG to obtain $(s_1, \ldots, s_{W_I}) \in \mathbb{F}_{2^\kappa}^{W_I}$, where W_I is the number of input wires of the circuit.
   (c) Let the input wires be $w_1, \ldots, w_{W_I}$, the parties compute
$$\tau = \sum_{i=1}^{W_I} s_i \cdot (v_{w_i} \oplus \lambda_{w_i})$$
   and send it to each other. Then these parties check whether the received τ is the same. If not, they abort the protocol.
2. **Sending Input Labels.** For each input wire w, each party P_i sends $k_{w,x_w \oplus \lambda_w}^{(i)}$ to P_{king}.
3. **Evaluating the Garbled Circuit.** P_{king} evaluates the garbled circuit gate by gate. Specifically speaking, for each gate with input wires a, b and output wire c, P_{king} holds $\boldsymbol{k}_{a,v_a \oplus \lambda_a}, \boldsymbol{k}_{b,v_b \oplus \lambda_b}$ and $v_a \oplus \lambda_a, v_b \oplus \lambda_b$. He uses them to decrypt all the parties' shares of $[\boldsymbol{k}_{c,v_c \oplus \lambda_c}]_d$. Then P_{king} reconstructs $\boldsymbol{k}_{c,v_c \oplus \lambda_c}$. Assume that

P_{king} is P_j. P_{king} compares $k^{(j)}_{c,v_c\oplus\lambda_c}$ obtained from $\boldsymbol{k}_{c,v_c\oplus\lambda_c}$ with $k^{(j)}_{c,0}, k^{(j)}_{c,1}$. If $k^{(j)}_{c,v_c\oplus\lambda_c}$ is not one of them, P_{king} aborts the protocol. If $k^{(j)}_{c,0} = k^{(j)}_{c,1}$, P_{king} also aborts the protocol. Otherwise, P_i gets $v_c \oplus \lambda_c$.

4. **Sending Output Labels.** After evaluating the garbled circuit, P_{king} gets $\boldsymbol{k}_{w,v_w\oplus\lambda_w} = k^{(1)}_{w,v_w\oplus\lambda_w} \| \dots \| k^{(n)}_{w,v_w\oplus\lambda_w}$ and $v_w \oplus \lambda_w$ for each output wire w of the circuit. Then, for each output wire w attached to each party P_i, P_{king} sends $k^{(i)}_{w,v_w\oplus\lambda_w}, v_w \oplus \lambda_w$ to P_i.
5. **Checking Output Labels.** For each output wire w attached to each party P_i, P_i checks whether $k^{(i)}_{w,v_w\oplus\lambda_w}$ received from P_{king} is among $k^{(i)}_{w,0}, k^{(i)}_{w,1}$. If not, P_i aborts the protocol. If $k^{(i)}_{w,0} = k^{(i)}_{w,1}$, P_i also aborts the protocol. Otherwise, P_i gets $v_w \oplus \lambda_w$.
6. **Computing Outputs.** For each output wire w attached to each party P_i, all the parties sends their shares of $[\lambda_w]_t$ to P_i. P_i checks whether $[\lambda_w]_t$ is a valid degree-t Shamir sharing. If not, P_i aborts the protocol. Otherwise, P_i reconstructs λ_w and computes $v_w = (v_w \oplus \lambda_w) \oplus \lambda_w$ as his output on this wire.

Figure 13: The evaluation phase of the main protocol.

8 Main Protocol

In this section, we summarize our main protocol Π_{Main} that puts all the things together. The protocol is presented in 14.

Protocol Π_{Main}

Public Values: All parties agree on $P_{j,1}, P_{j,2}$ who emulate the j-th virtual party V_j for $j = 1, \dots, N$, where $N = n^2 m$. Each pair of parties $(P_i, P_j) \in \mathcal{P}^2$ emulate $m = \Theta(\kappa/n^2)$ different virtual parties. Let Σ be an (N, T, K, ℓ)-LSSS over $\mathbb{F}_2$, where $K = \Theta(n)$, $T = N/3$, and ℓ is a constant. The multiplication of two Σ-sharings is a $\Sigma^{(2)}$-sharing (see Lemma 1). Let $d = n - 1$. All parties agree on an evaluator P_{king}.

1. **Preprocessing.** All the parties send (Prep, C) to $\mathcal{F}_{\mathsf{prep}}$ and receive the preprocessing data.
2. **Preparing ROTs.** All the parties send $(\mathsf{ROT}, 8\ell^2 Gm/K)$ to $\mathcal{F}_{\mathsf{ROT}}$ and receive the ROT correlations for virtual parties.
3. **Preparing Inputs for Virtual Parties.** The parties run Π_{VPinput}.
4. **Virtual Parties' Local Computation.** The parties and virtual parties run Π_{VPcomp}. More concretely, in the first step of Π_{VPcomp}, the members of each virtual party run Π_{Mult} $4\ell^2$ times for each group of K gates to compute $4\ell^2$ multiplications of a bit and a vector in $\mathbb{F}_2^{n\kappa}$. The other steps of virtual parties' local computations are performed by the members' local computations.
5. **Verifying the Computation.** The parties run Π_{VPver}.
6. **Garbling the Circuit.** The parties run Π_{Garble}.
7. **Evaluating the Circuit.** The parties run Π_{Eval}.

Figure 14: The main protocol.

We give the theorem of our main protocol below.

Theorem 2. Π_{Main} *t-securely realizes the MPC functionality* $\mathcal{F}$ *for circuit* C *against a fully malicious adversary in the* $\{\mathcal{F}_{\mathsf{Coin}}, \mathcal{F}_{\mathsf{prep}}, \mathcal{F}_{\mathsf{ROT}}\}$*-hybrid model with communication of* $O(|C|n\kappa + n^2\kappa^2 + n^4\kappa)$ *bits in constant rounds.*

From Lemma 4, Lemma 5, and Lemma 6, our main protocol can be instantiated in the plain model in a constant number of rounds with communication of $O(|C|n\kappa + n^2\kappa^2 + n^4\kappa)$ bits, which gives our main theorem.

Theorem 1. *Assuming one-way functions, there exists a computationally secure 26-round MPC protocol against a fully malicious static adversary corrupting up to* $(n-1)/2$ *parties with communication of* $O(|C|n\kappa+$

$n^2\kappa^2 + n^4\kappa)$, *where* n *is the number of parties,* $|C|$ *is the circuit size, and* κ *is the computational security parameter.*

We give the security proof of the main protocol in Section C and a detailed cost analysis in Section D. We also analyze the concrete round complexity in Section E.

Acknowledgement. J. Li and Y. Song were supported in part by the National Basic Research Program of China Grant 2011CBA00300, 2011CBA00301, the National Natural Science Foundation of China Grant 61033001, 61361136003, the Shanghai Qi Zhi Institute Innovation Program SQZ202313.

References

[ACGJ18] Prabhanjan Ananth, Arka Rai Choudhuri, Aarushi Goel, and Abhishek Jain. Round-optimal secure multiparty computation with honest majority. In *Advances in Cryptology – CRYPTO 2018: 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2018, Proceedings, Part II*, page 395–424, Berlin, Heidelberg, 2018. Springer-Verlag.

[ACGJ19] Prabhanjan Ananth, Arka Rai Choudhuri, Aarushi Goel, and Abhishek Jain. Two round information-theoretic mpc with malicious security. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2019*, pages 532–561, Cham, 2019. Springer International Publishing.

[ACGJ20] Prabhanjan Ananth, Arka Rai Choudhuri, Aarushi Goel, and Abhishek Jain. Towards efficiency-preserving round compression in mpc. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2020*, pages 181–212, Cham, 2020. Springer International Publishing.

[BCO+21] Aner Ben-Efraim, Kelong Cong, Eran Omri, Emmanuela Orsini, Nigel P. Smart, and Eduardo Soria-Vazquez. Large scale, actively secure computation from LPN and free-xor garbled circuits. In Anne Canteaut and Franccois-Xavier Standaert, editors, *Advances in Cryptology - EUROCRYPT 2021 - 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, October 17-21, 2021, Proceedings, Part III*, volume 12698 of *Lecture Notes in Computer Science*, pages 33–63. Springer, 2021.

[BFO12] Eli Ben-Sasson, Serge Fehr, and Rafail Ostrovsky. Near-linear unconditionally-secure multiparty computation with a dishonest minority. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology - CRYPTO 2012 - 32nd Annual Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2012. Proceedings*, volume 7417 of *Lecture Notes in Computer Science*, pages 663–680. Springer, 2012.

[BGH+23] Gabrielle Beck, Aarushi Goel, Aditya Hegde, Abhishek Jain, Zhengzhong Jin, and Gabriel Kaptchuk. Scalable multiparty garbling. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS 2023, Copenhagen, Denmark, November 26-30, 2023*, pages 2158–2172. ACM, 2023.

[BL18] Fabrice Benhamouda and Huijia Lin. k-round multiparty computation from k-round oblivious transfer via garbled interactive circuits. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018*, pages 500–532, Cham, 2018. Springer International Publishing.

[BLO16] Aner Ben-Efraim, Yehuda Lindell, and Eran Omri. Optimizing semi-honest secure multiparty computation for the internet. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*, pages 578–590. ACM, 2016.

[BLO17] Aner Ben-Efraim, Yehuda Lindell, and Eran Omri. Efficient scalable constant-round MPC via garbled circuits. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part II*, volume 10625 of *Lecture Notes in Computer Science*, pages 471–498. Springer, 2017.

[BMR90] Donald Beaver, Silvio Micali, and Phillip Rogaway. The round complexity of secure protocols (extended abstract). In Harriet Ortiz, editor, *Proceedings of the 22nd Annual ACM Symposium on Theory of Computing, May 13-17, 1990, Baltimore, Maryland, USA*, pages 503–513. ACM, 1990.

[Bra87] Gabriel Bracha. An o(log n) expected rounds randomized byzantine generals protocol. *J. ACM*, 34(4):910–920, 1987.

[Can00] Ran Canetti. Security and composition of multiparty cryptographic protocols. *J. Cryptol.*, 13(1):143–202, 2000.

[CC06] Hao Chen and Ronald Cramer. Algebraic geometric secret sharing schemes and secure multiparty computations over small fields. In Cynthia Dwork, editor, *Advances in Cryptology - CRYPTO 2006, 26th Annual International Cryptology Conference, Santa Barbara, California, USA, August 20-24, 2006, Proceedings*, volume 4117 of *Lecture Notes in Computer Science*, pages 521–536. Springer, 2006.

[CCXY18] Ignacio Cascudo, Ronald Cramer, Chaoping Xing, and Chen Yuan. Amortized complexity of information-theoretically secure MPC revisited. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology - CRYPTO 2018 - 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2018, Proceedings, Part III*, volume 10993 of *Lecture Notes in Computer Science*, pages 395–426. Springer, 2018.

[CGH+18] Koji Chida, Daniel Genkin, Koki Hamada, Dai Ikarashi, Ryo Kikuchi, Yehuda Lindell, and Ariel Nof. Fast large-scale honest-majority MPC for malicious adversaries. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology - CRYPTO 2018 - 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2018, Proceedings, Part III*, volume 10993 of *Lecture Notes in Computer Science*, pages 34–64. Springer, 2018.

[DI05] Ivan Damgård and Yuval Ishai. Constant-round multiparty computation using a black-box pseudorandom generator. In Victor Shoup, editor, *Advances in Cryptology - CRYPTO 2005: 25th Annual International Cryptology Conference, Santa Barbara, California, USA, August 14-18, 2005, Proceedings*, volume 3621 of *Lecture Notes in Computer Science*, pages 378–394. Springer, 2005.

[DIK+08] Ivan Damgård, Yuval Ishai, Mikkel Krøigaard, Jesper Buus Nielsen, and Adam Smith. Scalable multiparty computation with nearly optimal work and resilience. In David Wagner, editor, *Advances in Cryptology – CRYPTO 2008*, volume 5157 of *Lecture Notes in Computer Science*, pages 241–261, Santa Barbara, CA, USA, August 17–21, 2008. Springer Berlin Heidelberg, Germany.

[DN07] Ivan Damgård and Jesper Buus Nielsen. Scalable and unconditionally secure multiparty computation. In Alfred Menezes, editor, *Advances in Cryptology - CRYPTO 2007, 27th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2007, Proceedings*, volume 4622 of *Lecture Notes in Computer Science*, pages 572–590. Springer, 2007.

[EGP+23] Daniel Escudero, Vipul Goyal, Antigoni Polychroniadou, Yifan Song, and Chenkai Weng. Superpack: Dishonest majority mpc with constant online communication. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023*, pages 220–250, Cham, 2023. Springer Nature Switzerland.

[EGPS22] Daniel Escudero, Vipul Goyal, Antigoni Polychroniadou, and Yifan Song. Turbopack: Honest majority MPC with constant online communication. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*, pages 951–964. ACM, 2022.

[Gen09] Craig Gentry. Fully homomorphic encryption using ideal lattices. In Michael Mitzenmacher, editor, *Proceedings of the 41st Annual ACM Symposium on Theory of Computing, STOC 2009, Bethesda, MD, USA, May 31 - June 2, 2009*, pages 169–178. ACM, 2009.

[GLM+24] Vipul Goyal, Junru Li, Ankit Kumar Misra, Rafail Ostrovsky, Yifan Song, and Chenkai Weng. Dishonest majority constant-round MPC with linear communication from DDH. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part VI*, volume 15489 of *Lecture Notes in Computer Science*, pages 167–199. Springer, 2024.

[GLO+21] Vipul Goyal, Hanjun Li, Rafail Ostrovsky, Antigoni Polychroniadou, and Yifan Song. ATLAS: efficient and scalable MPC in the honest majority setting. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021, Proceedings, Part II*, volume 12826 of *Lecture Notes in Computer Science*, pages 244–274. Springer, 2021.

[GLOS25] Vipul Goyal, Junru Li, Rafail Ostrovsky, and Yifan Song. Towards building scalable constant-round MPC from minimal assumptions via round collapsing. In *Advances in Cryptology - CRYPTO 2025*, Lecture Notes in Computer Science. Springer, 2025.

[GMW87] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game or A completeness theorem for protocols with honest majority. In Alfred V. Aho, editor, *Proceedings of the 19th Annual ACM Symposium on Theory of Computing, 1987, New York, New York, USA*, pages 218–229. ACM, 1987.

[GPS21] Vipul Goyal, Antigoni Polychroniadou, and Yifan Song. Unconditional communication-efficient MPC via hall's marriage theorem. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021, Proceedings, Part II*, volume 12826 of *Lecture Notes in Computer Science*, pages 275–304. Springer, 2021.

[GPS22] Vipul Goyal, Antigoni Polychroniadou, and Yifan Song. Sharing transformation and dishonest majority MPC with packed secret sharing. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology - CRYPTO 2022 - 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15-18, 2022, Proceedings, Part IV*, volume 13510 of *Lecture Notes in Computer Science*, pages 3–32. Springer, 2022.

[GS18] Sanjam Garg and Akshayaram Srinivasan. Two-round multiparty secure computation from minimal assumptions. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology - EUROCRYPT 2018 - 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29 - May 3, 2018 Proceedings, Part II*, volume 10821 of *Lecture Notes in Computer Science*, pages 468–499. Springer, 2018.

[GS20] Vipul Goyal and Yifan Song. Malicious security comes free in honest-majority MPC. *IACR Cryptol. ePrint Arch.*, page 134, 2020.

[GSY21] S. Dov Gordon, Daniel Starin, and Arkady Yerukhimovich. The more the merrier: Reducing the cost of large scale MPC. In Anne Canteaut and Franccois-Xavier Standaert, editors, *Advances*

in Cryptology - EUROCRYPT 2021 - 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, October 17-21, 2021, Proceedings, Part II, volume 12697 of *Lecture Notes in Computer Science*, pages 694–723. Springer, 2021.

[HKN+25] David Heath, Vladimir Kolesnikov, Varun Narayanan, Rafail Ostrovsky, and Akash Shah. Multiparty garbling from OT with linear scaling and RAM support. Cryptology ePrint Archive, Paper 2025/444, 2025.

[HOSS18a] Carmit Hazay, Emmanuela Orsini, Peter Scholl, and Eduardo Soria-Vazquez. Concretely efficient large-scale MPC with active security (or, tinykeys for tinyot). In Thomas Peyrin and Steven D. Galbraith, editors, *Advances in Cryptology - ASIACRYPT 2018 - 24th International Conference on the Theory and Application of Cryptology and Information Security, Brisbane, QLD, Australia, December 2-6, 2018, Proceedings, Part III*, volume 11274 of *Lecture Notes in Computer Science*, pages 86–117. Springer, 2018.

[HOSS18b] Carmit Hazay, Emmanuela Orsini, Peter Scholl, and Eduardo Soria-Vazquez. Tinykeys: A new approach to efficient multi-party computation. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology - CRYPTO 2018 - 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2018, Proceedings, Part III*, volume 10993 of *Lecture Notes in Computer Science*, pages 3–33. Springer, 2018.

[IKNP03] Yuval Ishai, Joe Kilian, Kobbi Nissim, and Erez Petrank. Extending oblivious transfers efficiently. In Dan Boneh, editor, *Advances in Cryptology - CRYPTO 2003*, pages 145–161, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg.

[KOS15] Marcel Keller, Emmanuela Orsini, and Peter Scholl. Actively secure OT extension with optimal overhead. In Rosario Gennaro and Matthew Robshaw, editors, *Advances in Cryptology - CRYPTO 2015 - 35th Annual Cryptology Conference, Santa Barbara, CA, USA, August 16-20, 2015, Proceedings, Part I*, volume 9215 of *Lecture Notes in Computer Science*, pages 724–741. Springer, 2015.

[KS08] Vladimir Kolesnikov and Thomas Schneider. Improved garbled circuit: Free XOR gates and applications. In Luca Aceto, Ivan Damgård, Leslie Ann Goldberg, Magnús M. Halldórsson, Anna Ingólfsdóttir, and Igor Walukiewicz, editors, *Automata, Languages and Programming, 35th International Colloquium, ICALP 2008, Reykjavik, Iceland, July 7-11, 2008, Proceedings, Part II - Track B: Logic, Semantics, and Theory of Programming & Track C: Security and Cryptography Foundations*, volume 5126 of *Lecture Notes in Computer Science*, pages 486–498. Springer, 2008.

[LPSY15] Yehuda Lindell, Benny Pinkas, Nigel P. Smart, and Avishay Yanai. Efficient constant round multi-party computation combining BMR and SPDZ. In Rosario Gennaro and Matthew Robshaw, editors, *Advances in Cryptology - CRYPTO 2015 - 35th Annual Cryptology Conference, Santa Barbara, CA, USA, August 16-20, 2015, Proceedings, Part II*, volume 9216 of *Lecture Notes in Computer Science*, pages 319–338. Springer, 2015.

[Sha79] Adi Shamir. How to share a secret. *Commun. ACM*, 22(11):612–613, 1979.

[SY25] Yifan Song and Xiaxi Ye. Honest majority MPC with $\tilde{O}(|C|)$ communication in minicrypt. *Eurocrypt*, 2025.

[Yao82] Andrew Chi-Chih Yao. Protocols for secure computations (extended abstract). In *23rd Annual Symposium on Foundations of Computer Science, Chicago, Illinois, USA, 3-5 November 1982*, pages 160–164. IEEE Computer Society, 1982.

[Yao86] Andrew Chi-Chih Yao. How to generate and exchange secrets. In *27th Annual Symposium on Foundations of Computer Science (sfcs 1986)*, pages 162–167, 1986.

A Security Proof for Π_{Prep}

We prove the security of Π_{Prep} by constructing an ideal adversary Sim. Then we will show that the output in the ideal world is computationally indistinguishable from that in the real world using hybrid arguments. Without loss of generality, we assume that P_{king} is corrupted. The ideal adversary Sim is constructed as follows.

Simulator Sim

Sim sets $\mathsf{BitError} = \mathsf{RecError} = 0$ at the beginning. Then:

1. **Generating Random Bit Sharings.**
   (a) For each honest party P_i, Sim samples corrupted parties' shares of $[b_1^{(i)}]_t, \ldots, [b_{m_B/(n-t)}^{(i)}]_t, [b^{(i,1)}]_t, \ldots, [b^{(i,\kappa)}]_t$ and distributes them to the corrupted parties. For each corrupted party P_i, Sim receives honest parties' shares of $[b_1^{(i)}]_t, \ldots, [b_{m_B/(n-t)}^{(i)}]_t, [b^{(i,1)}]_t, \ldots, [b^{(i,\kappa)}]_t$. Sim checks whether these shares of $[b_1^{(i)}]_t, \ldots, [b_{m_B/(n-t)}^{(i)}]_t$ ($P_i \in \mathcal{C}$) are from valid degree-t sharings, and whether the secret of them are in $\mathbb{F}_2 \subseteq \mathbb{F}_{2^\kappa}$. If not, Sim sets $\mathsf{BitError} = 1$. Otherwise, Sim reconstructs the whole sharings $[b_1^{(i)}]_t, \ldots, [b_{m_B/(n-t)}^{(i)}]_t, \sum_{\beta\in\mathcal{C}}[b^{(\beta,j)}]_t$ and gets the corrupted parties' shares.
   (b) Sim emulates $\mathcal{F}_{\mathsf{Coin}}$ to receive $\mathsf{RandCoin}$ from all the corrupted parties. If abort_i is received, Sim sends abort_i to $\mathcal{F}_{\mathsf{prep}}$ and aborts the protocol on behalf of P_i. Then, Sim samples $r \in \mathbb{F}_{2^\kappa}$ randomly, emulates $\mathcal{F}_{\mathsf{Coin}}$ to send it to all the corrupted parties, and expands it to get $s_{\alpha,j}^{(\beta)}$ for $j = 1, \ldots, \kappa$, $\alpha = 1, \ldots, m_B/(n-t)$, and $\beta = 1, \ldots, n$.
   (c) Sim follows the protocol to compute the corrupted parties' shares of

$$[\sigma_j]_t = \sum_{\beta=1}^{n} \sum_{\alpha=1}^{m_B/(n-t)} s_{\alpha,j}^{(\beta)} [b_\alpha^{(\beta)}]_t + \sum_{\beta=1}^{n} [b^{(\beta,j)}]_t$$

   for each $j = 1, \ldots, \kappa$. Sim samples random bits $\sigma'_1, \ldots, \sigma'_\kappa$ and computes $\sigma_j = \sigma'_j + \sum_{\beta\in\mathcal{C}} \sum_{\alpha=1}^{m_B/(n-t)} s_{\alpha,j}^{(\beta)} b_\alpha^{(\beta)} + \sum_{\beta\in\mathcal{C}} b^{(\beta,j)}$ for each $j = 1, \ldots, \kappa$. Then, Sim samples the whole sharings $[\sigma_1]_t, \ldots, [\sigma_\kappa]_t$ based on the secrets and the corrupted parties' shares. Sim then sends each honest party's shares of these sharings to each corrupted party on behalf of the honest party.
   (d) Sim emulates each honest party to receive corrupted parties' shares of $[\sigma_1]_t, \ldots, [\sigma_\kappa]_t$ from them. Sim then follows the protocol to check whether each $[\sigma_j]_t$ is a valid degree-t Shamir sharings and $\sigma_j \in \mathbb{F}_2 \subseteq \mathbb{F}_{2^\kappa}$ on behalf of each honest party P_i. If not, Sim sends abort_i to $\mathcal{F}_{\mathsf{prep}}$ and aborts the protocol on behalf of P_i.
   (e) If $\mathsf{BitError} = 1$, Sim aborts the simulation.
   (f) Sim follows the protocol to compute corrupted parties' shares of the m_B random bit sharings and sends them to $\mathcal{F}_{\mathsf{prep}}$.
2. **Computing Sharings for Colored Bits.** Sim faithfully emulates $\mathcal{F}_{\mathsf{triple}}$ to receive (Triple, G_A) and corrupted parties' shares of the triples from the corrupted parties. If abort_i is received, Sim aborts the protocol on behalf of the honest parties. After the simulation terminates, Sim outputs what $\mathcal{A}$ outputs. For each AND gate with input wires a, b and output wire c:
   (a) Sim follows the protocol to compute corrupted parties' shares of $[\lambda_a - \alpha]_t, [\lambda_b - \beta]_t$. Then, Sim samples two random degree-t Shamir sharings as $[\lambda_a - \alpha]_t, [\lambda_b - \beta]_t$ based on the corrupted parties' shares and sends each honest party's shares of them to P_{king}.
   (b) Sim reconstructs $\lambda_a - \alpha, \lambda_b - \beta$ from all the parties' shares of $[\lambda_a - \alpha]_t, [\lambda_b - \beta]_t$. Sim receives $\overline{\lambda_a - \alpha}, \overline{\lambda_b - \beta}$ on behalf of each honest party. If $\overline{\lambda_a - \alpha} \neq \lambda_a - \alpha$ or $\overline{\lambda_b - \beta} \neq \lambda_b - \beta$, Sim sets $\mathsf{RecError} = 1$.

   After $[\lambda_a \cdot \lambda_b]_t$ for all the gates have been computed:

   (a) Sim emulates $\mathcal{F}_{\mathsf{Coin}}$ to receive $\mathsf{RandCoin}$ from all the corrupted parties. If abort_i is received, Sim sends abort to $\mathcal{F}_{\mathsf{prep}}$ and aborts the protocol on behalf of the honest parties. Then, Sim samples $s \in \mathbb{F}_{2^\kappa}$

randomly, emulates $\mathcal{F}_{\mathsf{Coin}}$ to send it to all the corrupted parties, and expands it to get $(s_1, \ldots, s_{2G_A}) \in \mathbb{F}_{2^\kappa}^{2G_A}$.

(b) Sim follows the protocol to compute all the parties' shares of $[\tau]_t$ and sends each honest party's share to all the corrupted parties on behalf of the honest party. Sim then receives the corrupted parties' shares from them. Sim follows the protocol to check the sharings and τ on behalf of each honest party P_i. If the check fails, Sim sends abort_i to $\mathcal{F}_{\mathsf{prep}}$ and aborts the protocol on behalf of P_i.

(c) If $\mathsf{RecError} = 1$, Sim aborts the simulation.

(d) Finally, Sim follows the protocol to compute corrupted parties' shares of $[\Lambda_{c,i}]_t$ for each $i = 1, 2, 3, 4$ and sends them to $\mathcal{F}_{\mathsf{prep}}$.

3. After the simulation terminates, Sim outputs what $\mathcal{A}$ outputs.

Figure 15: The simulator for Π_{Prep}.

We construct the following hybrids:

$\mathbf{Hyb}_0$: In this hybrid, Sim runs the protocol honestly. This corresponds to the real-world scenario.

$\mathbf{Hyb}_1$: In this hybrid, Sim additionally sets $\mathsf{BitError} = \mathsf{RecError} = 0$ at the beginning of the simulation. This doesn't affect the output distribution. Thus, $\mathbf{Hyb}_1$ and $\mathbf{Hyb}_0$ have the same output distribution.

$\mathbf{Hyb}_2$: In this hybrid, whenever an honest party or the functionality $\mathcal{F}_{\mathsf{triple}}$ generates a degree-t Shamir sharing, Sim first generates the corrupted parties' shares, and then randomly samples the honest parties' shares based on corrupted parties' shares and the secret. Since for all these sharings, the set of corrupted parties' shares is independent of the secret, we only change the order of generating the honest and corrupted parties' shares of the sharings. This doesn't change the output distribution. Thus, $\mathbf{Hyb}_2$ and $\mathbf{Hyb}_1$ have the same output distribution.

$\mathbf{Hyb}_3$: In this hybrid, Sim additionally sets $\mathsf{BitError} = 1$ if honest parties' shares of $[b_1^{(i)}]_t, \ldots, [b_{m_B/(n-t)}^{(i)}]_t$ for each corrupted P_i are not from valid degree-t sharings or the secrets of them are not in $\mathbb{F}_2 \subseteq \mathbb{F}_{2^\kappa}$. This doesn't affect the output distribution. Thus, $\mathbf{Hyb}_3$ and $\mathbf{Hyb}_2$ have the same output distribution.

$\mathbf{Hyb}_4$: In this hybrid, Sim doesn't follow the protocol to compute the honest parties' shares of each $[\sigma_j]_t$. Instead, Sim follows the protocol to compute corrupted parties' shares first. Let i' be the smallest index in $\mathcal{H}$. Sim computes honest parties' shares of

$$\sum_{\beta=1}^{n} \sum_{\alpha=1}^{m_B/(n-t)} s_{\alpha,j}^{(\beta)} [b_\alpha^{(\beta)}]_t + \sum_{\beta \neq i'} [b^{(\beta,j)}]_t$$

honestly and then samples $b^{(i',j)} \in \mathbb{F}_2$ randomly. Then, for each $j = 1, \ldots, \kappa$, Sim samples honest parties' shares of

$$[\sigma_j]_t = \sum_{\beta=1}^{n} \sum_{\alpha=1}^{m_B/(n-t)} s_{\alpha,j}^{(\beta)} [b_\alpha^{(\beta)}]_t + \sum_{\beta=1}^{n} [b^{(\beta,j)}]_t$$

based on the corrupted parties' shares and the secret $\sigma_j = \sum_{\beta=1}^{n} \sum_{\alpha=1}^{m_B/(n-t)} s_{\alpha,j}^{(\beta)} b_\alpha^{(\beta)} + \sum_{\beta=1}^{n} b^{(\beta,j)}$. Finally, Sim computes honest parties' shares of $[b^{(i',j)}]_t$ based on their shares of $[\sigma_j]_t$ and $\sum_{\beta=1}^{n} \sum_{\alpha=1}^{m_B/(n-t)} s_{\alpha,j}^{(\beta)} [b_\alpha^{(\beta)}]_t + \sum_{\beta \neq i'} [b^{(\beta,j)}]_t$.

Since honest parties' shares of $[b^{(i',j)}]_t$ are sampled based on the random secret $b^{(i',j)} \in \mathbb{F}_2$ and corrupted parties' shares are components of honest parties' shares of $[\sigma_j]_t$, sampling honest parties' shares of $[\sigma_j]_t$ based on corrupted parties' shares and the secret won't change the distribution of them. Thus, we only change the order of generating honest parties' shares of $[b^{(i',j)}]_t$ and $[\sigma_j]_t$ for each $j = 1, \ldots, \kappa$, which does not affect the output distribution. Thus, $\mathbf{Hyb}_4$ and $\mathbf{Hyb}_3$ have the same output distribution.

$\mathbf{Hyb}_5$: In this hybrid, Sim doesn't follow the protocol to sample $b^{(i',j)}$ for each $j = 1, \ldots \kappa$. Instead, Sim directly samples

$$\sigma'_j = \sum_{\beta \in \mathcal{H}} \sum_{\alpha=1}^{m_B/(n-t)} s_{\alpha,j}^{(\beta)} b_\alpha^{(\beta)} + \sum_{\beta \in \mathcal{H}} b^{(\beta,j)} \in \mathbb{F}_2 \subseteq \mathbb{F}_{2^\kappa}$$

for each $j = 1, \ldots, \kappa$ randomly. Then, σ_j is computed by adding σ' by

$$\sum_{\beta \in \mathcal{C}} \sum_{\alpha=1}^{m_B/(n-t)} s_{\alpha,j}^{(\beta)} b_\alpha^{(\beta)} + \sum_{\beta \in \mathcal{C}} b^{(\beta,j)} \in \mathbb{F}_2 \subseteq \mathbb{F}_{2^\kappa}.$$

$b^{(i',j)}$ is computed by $\sigma'_j - \sum_{\beta \in \mathcal{H} \setminus \{i'\}} \sum_{\alpha=1}^{m_B/(n-t)} s_{\alpha,j}^{(\beta)} b_\alpha^{(\beta)} - \sum_{\beta \in \mathcal{H} \setminus \{i'\}} b^{(\beta,j)} \in \mathbb{F}_2 \subseteq \mathbb{F}_{2^\kappa}$.

We only change the way of generating each σ_j and the order of generating σ'_j and $b^{(i',j)}$ for each $j = 1, \ldots, \kappa$. Since $b^{(i',j)}$ is a component of σ'_j, which is randomly sampled in $\mathbb{F}_2$, σ'_j is also a random bit. Therefore, directly sampling σ'_j and using it to compute $b^{(i',j)}$ won't change the output distribution. Thus, $\mathbf{Hyb}_5$ and $\mathbf{Hyb}_4$ have the same output distribution.

$\mathbf{Hyb}_6$: In this hybrid, Sim additionally aborts the simulation if the check of $[\sigma_1]_t, \ldots, [\sigma_\kappa]_t$ passes with $\mathsf{BitError} = 1$. Note that $\mathsf{BitError} = 1$ only happens when a sharing $[b_\alpha^{(i)}]_t$ generated by corrupted P_i is not a degree-t Shamir sharing for a bit. In this case, when $s_{\alpha',j}^{(\beta)}$ for all $\alpha' \neq \alpha$ or $\beta \neq i$ are fixed, at most one of $[\sigma_j]_t$ corresponding to $s_{\alpha,j}^{(i)} = 0$ or 1 can be a valid sharing of a bit since their subtraction is $[b_\alpha^{(i)}]_t$ that is invalid. Therefore, if $s_{\alpha,j}^{(\beta)}$ for $j = 1, \ldots, \kappa$, $\alpha = 1, \ldots, m_B/(n-t)$, and $\beta = 1, \ldots, n$ are truly random, each $[\sigma_j]_t$ can be a valid degree-t sharing for a bit for at most $1/2$ probability. In this case, the check can pass with at most $2^{-\kappa}$ probability, which is negligible.

Thus, if there is a non-negligible probability that $[\sigma_1]_t, \ldots, [\sigma_\kappa]_t$ are all valid degree-t sharings for a bit, then the truly random bits $s_{\alpha,j}^{(\beta)}$ and the pseudorandom ones expanded from a PRG can be distinguished by computing $[\sigma_1]_t, \ldots, [\sigma_\kappa]_t$ with a non-negligible probability, which contradicts the definition of a PRG, so there probability that the check passes with $\mathsf{BitError} = 1$ is negligible. Therefore, the distribution only changes with negligible probability. Thus, the distributions of $\mathbf{Hyb}_6$ and $\mathbf{Hyb}_5$ are computationally indistinguishable.

$\mathbf{Hyb}_7$: In this hybrid, for each AND gate, Sim doesn't follow the protocol to sample honest parties' shares of $[\alpha]_t, [\beta]_t$ first and use them to compute honest parties' shares of $[\lambda_a - \alpha]_t, [\lambda_b - \beta]_t$. Instead, Sim directly samples honest parties' shares of $[\lambda_a - \alpha]_t, [\lambda_b - \beta]_t$ based on the corrupted parties' shares. Then, Sim computes honest parties' shares of $[\alpha]_t, [\beta]_t$ based on their shares of $[\lambda_a - \alpha]_t, [\lambda_b - \beta]_t$ and $[\lambda_a]_t, [\lambda_b]_t$. With the corrupted parties' shares fixed, the honest parties' shares of $[\alpha]_t, [\beta]_t$ are random (based on the corrupted parties' shares), so are their shares of $[\lambda_a - \alpha]_t, [\lambda_b - \beta]_t$. Therefore, changing the order of generating $[\alpha]_t, [\beta]_t$ and $[\lambda_a - \alpha]_t, [\lambda_b - \beta]_t$ doesn't affect the output distribution. Thus, $\mathbf{Hyb}_7$ and $\mathbf{Hyb}_6$ have the same output distribution.

$\mathbf{Hyb}_8$: In this hybrid, Sim additionally sets $\mathsf{RecError} = 1$ if for some AND gate, the values $\lambda_a - \alpha, \lambda_b - \beta$ reconstructed from all the parties' shares are different from $\overline{\lambda_a - \alpha}, \overline{\lambda_b - \beta}$ received from P_{king}. This doesn't affect the output distribution. Thus, $\mathbf{Hyb}_8$ and $\mathbf{Hyb}_7$ have the same output distribution.

$\mathbf{Hyb}_9$: In this hybrid, after checking the sharing $[\tau]_t$, Sim aborts simulation if the check passes with $\mathsf{RecError} = 1$. This only happens when some $\overline{e_\alpha}$ (or $\overline{d_\alpha}$, we assume that e_α is not correctly sent here without loss of generality) received by an honest party P_j is not equal to the e_α determined by the honest parties' shares of $[e_\alpha]_t$. We suppose $\overline{e_i} - e_i = \delta_i$ and $\overline{d_i} - d_i = \delta^{(i)}$. Then, since τ is determined by the honest parties' shares of $[\tau]_t$, the check only passes when $\sum_{i=1}^{G_A} (s_i \cdot \delta_i + s_{G_A+i} \cdot \delta^{(i)}) = 0$. Note that $\delta_\alpha \neq 0$, there is only one $s_\alpha \in \mathbb{F}_{2^\kappa}$ that can make $\sum_{i=1}^{G_A} (s_i \cdot \delta_i + s_{G_A+i} \cdot \delta^{(i)}) = 0$ hold when $s_\beta, \beta \in \{1, \ldots, 2G_A\} \setminus \{\alpha\}$ are fixed. Therefore, in this case, if $s_\beta, \beta \in \{1, \ldots, G_A\}$ are truly random, the check passes for at most $2^{-\kappa}$ probability, which is negligible.

Thus, if there is a non-negligible probability that the check passes, then the truly random elements $s_\beta, \beta \in \{1, \ldots, G_A\} \setminus \{\alpha\}$ and the pseudorandom ones expanded from a PRG can be distinguished by computing $[\tau]_t$ with a non-negligible probability, which contradicts the definition of a PRG, so the probability that the check passes with $\mathsf{RecError} = 1$ is negligible. Therefore, the distribution only changes with negligible probability. Thus, the distributions of $\mathbf{Hyb}_9$ and $\mathbf{Hyb}_8$ are computationally indistinguishable.

$\mathbf{Hyb}_{10}$: In this hybrid, Sim directly samples honest parties' outputs based on the corrupted parties' shares instead of computing them from honest parties' shares of $[b_1^{(i)}]_t, \ldots, [b_{m_B/(n-t)}^{(i)}]_t$ for each honest P_i

and each $[c]_t$ of each triple. Then, Sim computes these shares based on the honest parties' outputs. More concretely, honest parties' shares of $[c]_t$ is computed based on their shares of $[a]_t, [b]_t$ and each multiplication result, i.e. $[\lambda_a \cdot \lambda_b]_t$ for each AND gate. For $[b_1^{(i)}]_t, \ldots, [b_{m_B/(n-t)}^{(i)}]_t$, $[b^{(i,1)}]_t, \ldots, [b^{(i,\kappa)}]_t$, we will introduce the generation process of them below.

Note that sampling the random bit sharings is equivalent to sampling the a-fold interleaved sharings $[u_1]_{\times a}^b, [u_2]_{\times a}^b, \ldots, [u_{n-t}]_{\times a}^b$ for the process of generating batch of $a(n-t)$ random bit sharings. Let $\mathcal{S}_\mathcal{C}$ be a set of t indices such that $\mathcal{C} \subseteq \mathcal{S}_\mathcal{C}$ and $\mathcal{S}_\mathcal{H}$ be $\mathcal{P} \backslash \mathcal{S}_\mathcal{C}$. Then let $\mathcal{N}_\mathcal{C}$ denote the $n \times t$ sub-matrix of $\mathcal{N}$ containing columns with indices in $\mathcal{S}_\mathcal{C}$, and $\mathcal{N}_\mathcal{H}$ denote the sub-matrix containing columns with indices in $\mathcal{S}_\mathcal{H}$. We have

$$\begin{pmatrix} [u_1]_{\times a}^b \\ [u_2]_{\times a}^b \\ \vdots \\ [u_{n-t}]_{\times a}^b \end{pmatrix} = \mathcal{N} \cdot \begin{pmatrix} [u^{(1)}]_{\times a}^b \\ [u^{(2)}]_{\times a}^b \\ \vdots \\ [u^{(n)}]_{\times a}^b \end{pmatrix} = \mathcal{N}_\mathcal{C} \cdot \left([u^{(j)}]_{\times a}^b\right)_{j \in \mathcal{S}_\mathcal{C}} + \mathcal{N}_\mathcal{H} \cdot \left([u^{(j)}]_{\times a}^b\right)_{j \in \mathcal{S}_\mathcal{H}}.$$

Note that $\mathcal{N}_\mathcal{H}$ is a $(n-t) \times (n-t)$ submatrix of an $(n-t) \times n$ Vandermonde matrix, which is invertible. Sim samples honest parties' shares of $[u^{(j)}]_{\times a}^b$ for each $j \in \mathcal{S}_\mathcal{C} \backslash \mathcal{C}$ randomly and then computes $\left([u^{(j)}]_{\times a}^b\right)_{j \in \mathcal{S}_\mathcal{H}}$ by

$$\mathcal{N}_\mathcal{H}^{-1} \cdot \left(\begin{pmatrix} [u_1]_{\times a}^b \\ [u_2]_{\times a}^b \\ \vdots \\ [u_{n-t}]_{\times a}^b \end{pmatrix} - \mathcal{N}_\mathcal{C} \cdot \left([u^{(j)}]_{\times a}^b\right)_{j \in \mathcal{S}_\mathcal{C}} \right).$$

Then, $[b_1^{(i)}]_t, \ldots, [b_{m_B/(n-t)}^{(i)}]_t$, $[b^{(i,1)}]_t, \ldots, [b^{(i,\kappa)}]_t$ are computed from each $[u^{(i)}]_{\times a}^b$.

Since honest parties' shares of each $[c]_t$ in a triple are sampled based on corrupted parties' shares, and they are components of each $[\lambda_a \cdot \lambda_b]_t$, which is added into the result of $[\Lambda_{c,i}]_t$ for each AND gate. Thus, honest parties' shares of $[\Lambda_{c,i}]_t$ are random based on the corrupted parties' shares, and so are $[\lambda_a \cdot \lambda_b]_t$ and $[c]_t$. We only change the order of generating them without changing their distributions. For $[b_1^{(i)}]_t, \ldots, [b_{m_B/(n-t)}^{(i)}]_t$, $[b^{(i,1)}]_t, \ldots, [b^{(i,\kappa)}]_t$ that are also sampled randomly based on corrupted parties' shares, sampling them are equivalent to sampling shares of $[u^{(i)}]_{\times a}^b$. Since $\mathcal{N}_\mathcal{H}$ is invertible, there is a bijective map from honest parties' shares of $\left([u^{(j)}]_{\times a}^b\right)_{j \in \mathcal{S}_\mathcal{H}}$ and the output shares for the honest parties when $\left([u^{(j)}]_{\times a}^b\right)_{j \in \mathcal{S}_\mathcal{C}}$ are fixed, and we do not change the generation process of $\left([u^{(j)}]_{\times a}^b\right)_{j \in \mathcal{S}_\mathcal{C}}$. Thus, we only change the order of sampling honest parties' shares of $\left([u^{(j)}]_{\times a}^b\right)_{j \in \mathcal{S}_\mathcal{H}}$ and the output bit sharings without changing their distributions. Thus, $\mathbf{Hyb}_{10}$ and $\mathbf{Hyb}_9$ have the same output distribution.

$\mathbf{Hyb}_{11}$: In this hybrid, Sim doesn't generate the honest parties' shares of sharings $[b_1^{(i)}]_t, \ldots, [b_{m_B/(n-t)}^{(i)}]_t$, $[b^{(i,1)}]_t, \ldots, [b^{(i,\kappa)}]_t$ for each honest P_i and each triple $([a]_c, [b]_t, [c]_t)$. Since these shares are not used in the simulation, not generating them does not affect the output distribution. Thus, $\mathbf{Hyb}_{11}$ and $\mathbf{Hyb}_{10}$ have the same output distribution.

$\mathbf{Hyb}_{12}$: In this hybrid, honest parties' outputs are not sampled by Sim. Instead, they directly get the outputs from $\mathcal{F}_{\mathsf{prep}}$. Since the way of generating these outputs is the same in both hybrids, we do not change the output distribution. Thus, $\mathbf{Hyb}_{12}$ and $\mathbf{Hyb}_{11}$ have the same output distribution.

Note that $\mathbf{Hyb}_{12}$ is the ideal-world scenario, Π_{Prep} computes $\mathcal{F}_{\mathsf{prep}}$ with computational security in the $\{\mathcal{F}_{\mathsf{Coin}}, \mathcal{F}_{\mathsf{triple}}\}$-hybrid model.

B Cost Analysis for Π_{Prep}

We give the analysis for the communication cost of Π_{prep} as follows.

1. **Generating Random Bit Sharings.** In Step (a), each party generates $m_B/(n-t)+\kappa = O(m_B/n+\kappa)$ degree-t Shamir sharings of $O(n\kappa)$ bits. Thus, the communication cost of Step (a) is $O(m_B n\kappa + n^2\kappa^2 +$

$N \cdot n\kappa) = O(|C|n\kappa + n^2\kappa^2 + n^3\kappa)$. Step (b) only contains an invocation of $\mathcal{F}_{\mathsf{Coin}}$ and local computations. In Step (c), the parties send κ degree-t Shamir sharing of $O(n\kappa)$ bits to all the parties, which requires communication of $O(n^2\kappa^2)$ bits. Steps (d) and (e) only contain local computations. Therefore, the total communication complexity is $O(|C|n\kappa + n^2\kappa^2 + n^3\kappa)$ bits.

2. **Computing Sharings for Colored Bits.** For each AND gate, the parties need to send two degree-t Shamir sharings to P_{king} and receive the secret from him, which requires communication of $O(n\kappa)$ bits. Finally, the parties need to send a degree-t Shamir sharing to all the parties, which requires communication of $O(n^2\kappa)$ bits. Thus, the total communication complexity is $O(G_A n\kappa + n^2\kappa) = O(|C|n\kappa + n^2\kappa)$.

Realizing $\mathcal{F}_{\mathsf{Coin}}$ requires communication of $O(n^2\kappa)$ bits, and realizing $\mathcal{F}_{\mathsf{triple}}$ requires communication of $O(n^2\kappa + nG_A\kappa) = O(|C|n\kappa + n^2\kappa)$ bits. Thus, the total communication cost of realizing $\mathcal{F}_{\mathsf{prep}}$ in the plain model is $O(|C|n\kappa + n^2\kappa^2 + n^3\kappa)$ bits.

C Security Proof of the Main Protocol

We prove the security of Π_{Main} by constructing an ideal adversary Sim. Then we will show that the output in the ideal world is computationally indistinguishable from that in the real world using hybrid arguments. Without loss of generality, we assume that P_{king} is corrupted. The ideal adversary Sim is constructed as follows.

Simulator Sim

We regard a virtual party V_j as corrupted if its members $P_{j,1}, P_{j,2}$ are both corrupted. The other virtual parties are honest virtual parties. At the beginning of the simulation, Sim sets $\mathsf{Corr}_i = \mathsf{Comp}_i = 0$ for each honest virtual party V_i. Sim sets $\mathsf{CompCheck} = 0$. When Sim aborts the protocol on behalf of an honest party P_i, he also sends abort_i to $\mathcal{F}$.

1. **Preprocessing.** Sim faithfully emulates $\mathcal{F}_{\mathsf{prep}}$ to receive (Prep, C) from all the corrupted parties and receive corrupted parties' outputs from $\mathcal{A}$. If abort_i is received, Sim aborts the protocol on behalf of P_i.
2. **Preparing ROTs.** Sim faithfully emulates $\mathcal{F}_{\mathsf{ROT}}$ to receive $(\mathsf{ROT}, 8\ell^2 Gm/K)$ from all the corrupted parties and receive corrupted parties' outputs from $\mathcal{A}$. If abort_i is received, Sim aborts the protocol on behalf of P_i.
3. **Preparing Inputs for Virtual Parties.** Sim runs $\mathsf{Sim}_{\mathsf{VPinput}}$ (Figure 17) to simulate Π_{VPinput}.
4. **Virtual Parties' Local Computation.** Sim runs $\mathsf{Sim}_{\mathsf{VPcomp}}$ (Figure 18) to simulate the virtual parties' local computation.
5. **Verifying the Computation.** Sim runs $\mathsf{Sim}_{\mathsf{VPver}}$ (Figure 19) to simulate Π_{VPver}.
6. **Garbling the Circuit.** Sim runs $\mathsf{Sim}_{\mathsf{Garble}}$ (Figure 20) to simulate Π_{Garble}.
7. **Evaluating the Circuit.** Sim runs $\mathsf{Sim}_{\mathsf{Eval}}$ (Figure 21) to simulate Π_{Eval}.
8. After the simulation terminates, Sim outputs what $\mathcal{A}$ outputs.

Figure 16: The simulator for the main protocol Π_{Main}.

Simulator $\mathsf{Sim}_{\mathsf{VPinput}}$

1. **Committing Watchlists.** For each honest party P_i, Sim follows the protocol to sample Watch_i. Then, Sim samples the corrupted parties' shares of $[\mathsf{Watch}_i]_t$ randomly and distributes them on behalf of P_i. For each corrupted party P_i, Sim receives honest parties' shares of Watch_i from P_i. Sim then reconstructs Watch_i from the received shares (if they are from a valid degree-t sharing). For each $j \in \mathsf{Watch}_i$, Sim sets $\mathsf{Corr}_j = 1$.
2. **Sharing the Wire Labels.** For each group of K gates $g_1, \ldots, g_K$ with output wires $c_1, \ldots, c_K$, for each honest party P_i, Sim samples the corrupted members' shares of (the additive sharings of) the virtual

parties' shares of $[\![\boldsymbol{k}_{\mathbf{1}}^{(i,\alpha)} - \boldsymbol{k}_{\mathbf{0}}^{(i,\alpha)}]\!]$ and $[\![\boldsymbol{k}_{\mathbf{0}}^{(i,\alpha)}]\!]^{(2)}$ for $\alpha = 1, \ldots, \kappa$ randomly and distributes them on behalf of P_i. For each virtual party V_j with $\mathsf{Corr}_j = 1$, Sim also samples the honest members' shares of these sharings based on corrupted parties' shares. For each corrupted party P_i, Sim receives the honest members' shares of the additive sharings of the virtual parties' shares of $[\![\boldsymbol{k}_{\mathbf{1}}^{(i,\alpha)} - \boldsymbol{k}_{\mathbf{0}}^{(i,\alpha)}]\!]$ and $[\![\boldsymbol{k}_{\mathbf{0}}^{(i,\alpha)}]\!]^{(2)}$ for $\alpha = 1, \ldots, \kappa$.

3. **Sharing Colored Bits to Virtual Parties.** For each group of K gates $g_1, \ldots, g_K$ with output wires $c_1, \ldots, c_K$ and each $i = 1, 2, 3, 4$:
   (a) Sim follows the protocol to compute the Σ-sharings (together with the additive sharings of the shares) generated by each corrupted party.
   (b) For each corrupted member of a virtual party and each honest member of a virtual party V_j with $\mathsf{Corr}_j = 1$, Sim samples his degree-t sharings of his share of $\langle \boldsymbol{\Lambda}_i^{V_j} \rangle$ based on the shares generated by corrupted parties (computed in the last step). For each corrupted member among them, Sim sends the shares generated by honest parties to him on behalf of the honest parties. Then, Sim reconstructed $\langle \boldsymbol{\Lambda}_i^{V_j} \rangle$ for each corrupted virtual party V_j and each honest virtual party V_j with $\mathsf{Corr}_j = 1$.
   (c) Sim emulates each honest member of a virtual party to receive the shares of his degree-t sharings of his share of $\langle \boldsymbol{\Lambda}_i^{V_j} \rangle$ generated by corrupted parties. Sim then checks whether they are correctly sent. If not, Sim aborts the protocol on behalf of the honest member.
4. **Preparing the Verification of Sharings.** For each honest party P_i, Sim samples the corrupted members' shares of the additive sharings of the virtual members' shares (of the additive sharing) of $[\![\boldsymbol{r}^{(i)}]\!]$ randomly and samples the degree-t Shamir sharings of them. For each virtual party V_j with $\mathsf{Corr}_j = 1$, Sim also samples the honest members' shares of these sharings based on corrupted parties' shares and samples the degree-t sharings for them. For other members' shares, Sim randomly samples the corrupted parties' shares of the degree-t sharings for these shares. Finally, Sim distributes all these corrupted parties' shares of degree-t sharings among the corrupted parties. For each corrupted party P_i, Sim receives honest parties' shares of all the degree-t sharings generated by P_i in this step and reconstructs the secrets of them (if valid). Then, Sim obtains all the virtual parties' shares of $[\![\boldsymbol{r}^{(i)}]\!]$ together with the additive sharings of them.

Figure 17: The simulator for Π_{VPinput}.

Simulator $\mathsf{Sim}_{\mathsf{VPcomp}}$

1. **Computing Multiplications.** For simplicity, we use the same notation as in Π_{Mult} here.
   – For each execution of Π_{Mult} executed by an honest party P_α (who acts as P_1 in Π_{Mult}) and a corrupted party P_β (who acts as P_2 in Π_{Mult}) while emulating a virtual party V_j with $\mathsf{Corr}_j = 0$:
     (a) Sim samples a random string in $\mathbb{F}_2^{n\kappa}$ as $\boldsymbol{u}_1^{(1)} + \boldsymbol{u}_0^{(1)} + \boldsymbol{x}_1$ and a random bit as $y_1 + b^{(2)}$. Sim sends them to P_β on behalf of P_α. Sim then receives $y_2 + b^{(1)}, \boldsymbol{u}_1^{(2)} + \boldsymbol{u}_0^{(2)} + \boldsymbol{x}_2$ from P_β.
     (b) Sim checks whether $y_2 + b^{(1)}, \boldsymbol{u}_1^{(2)} + \boldsymbol{u}_0^{(2)} + \boldsymbol{x}_2$ are computed correctly from $b^{(1)}, k_1^{(2)}, k_0^{(2)}$, the corrupted members' shares of $\{[\![\boldsymbol{k}_{\mathbf{1}}^{(i,\gamma)} - \boldsymbol{k}_{\mathbf{0}}^{(i,\gamma)}]\!]\}_{\gamma=1}^{\kappa}$, and their shares of $\{[\![\boldsymbol{\Lambda}_\beta]\!]\}_{\beta=1}^{4}$ for each honest party P_i. If not, Sim sets $\mathsf{Comp}_j = 1$. For each corrupted party P_i, Sim computes P_β's share of the additive sharing for V_j's shares of $\{[\![\boldsymbol{k}_{\mathbf{1}}^{(i,\gamma)} - \boldsymbol{k}_{\mathbf{0}}^{(i,\gamma)}]\!]\}_{\gamma=1}^{\kappa}$ based on $k_1^{(2)}, k_0^{(2)}$ and $\boldsymbol{u}_1^{(2)} + \boldsymbol{u}_0^{(2)} + \boldsymbol{x}_2$. Then, Sim checks whether each $y_2 + b^{(1)}$ is computed correctly from $b^{(1)}$ and the corrupted members' shares of $\{[\![\boldsymbol{\Lambda}_\beta]\!]\}_{\beta=1}^{4}$. If not, Sim sets $\mathsf{Comp}_j = 1$.

   If there are more than $N/24$ virtual parties V_j with $\mathsf{Comp}_j = 1$, Sim sets $\mathsf{CompCheck} = 1$. Otherwise, for each virtual party V_j with $\mathsf{Comp}_j = 1$, Sim samples V_j's shares of $[\![\boldsymbol{k}_{\mathbf{1}}^{(i,\alpha)} - \boldsymbol{k}_{\mathbf{0}}^{(i,\alpha)}]\!]$ and $[\![\boldsymbol{k}_{\mathbf{0}}^{(i,\alpha)}]\!]^{(2)}$ for $\alpha = 1, \ldots, \kappa$ and each honest P_i randomly. Then, Sim computes the honest member's shares accordingly.
   – For each execution of Π_{Mult} run by two members (honest P_α acts as P_1 and corrupted P_β acts as P_2) emulating a virtual party V_j with $\mathsf{Corr}_j = 1$:
     (a) Sim samples $k_{b^{(1)}\oplus 1}^{(1)} \in \mathbb{F}_{2^\kappa}$ and $b^{(2)} \in \mathbb{F}_2$ randomly based. Then, Sim samples the degree-t sharings for $k_0^{(1)}, k_1^{(1)}$ and $b^{(2)} \in \mathbb{F}_2, k_{b^{(2)}}^{(2)}$ randomly based on the secrets and corrupted parties' shares.
     (b) Sim follows the protocol to emulate P_α and interact with P_β during the execution of Π_{Mult}.
   – For each execution of Π_{Mult} run by two honest members of a virtual party V_j with $\mathsf{Corr}_j = 1$, Sim honestly samples all the ROT correlations.

2. **Verification of the Sharings.**
   (a) For each degree-t sharing for a corrupted member's share of $[\![\boldsymbol{r}^{(i)}]\!]$ for each honest party P_i, Sim samples the honest parties' shares of the sharing and sends them to the member on behalf of the honest parties. For each honest member's share of the degree-t sharing $[\![\boldsymbol{r}^{(i)}]\!]$ for each party P_i, Sim receives the corrupted parties' shares of the sharing from the corrupted parties. If P_i is honest, Sim checks whether the shares are correctly sent. If P_i is corrupted, Sim follows the protocol to check whether the shares are from a valid degree-t sharing. If not, Sim aborts the protocol on behalf of the honest receiver.
   (b) Sim emulates $\mathcal{F}_{\mathsf{Coin}}$ to receive $\mathsf{RandCoin}$ from all the corrupted parties. If abort_i is received, Sim aborts the protocol on behalf of P_i. Then, Sim samples $r \in \mathbb{F}_{2^\kappa}$ randomly, emulates $\mathcal{F}_{\mathsf{Coin}}$ to send it to all the corrupted parties, and expands it to get $(r_1, \ldots, r_{k_1}) \in \mathbb{F}_{2^\kappa}^{k_1}$.
   (c) Sim follows the protocol to compute $[\![\sigma]\!]_{\times\kappa}$ and sends it to all the corrupted parties on behalf of each honest party. Sim then receives $[\![\sigma]\!]_{\times\kappa}$ from the corrupted parties.

Finally, Sim follows the protocol to compute the share for each corrupted virtual party V_j and each honest virtual party V_j with $\mathsf{Corr}_j = 1$ or Comp_j of $[\![\boldsymbol{\Lambda}_\beta]\!]^{(2)}$ and $[\![\boldsymbol{k}_{\boldsymbol{\Lambda}_\beta}^{(i,\alpha)}]\!]^{(2)}$ for $\alpha = 1, \ldots, \kappa$ and $i = 1, \ldots, n$.

Figure 18: The simulator for virtual parties' local computations.

Simulator $\mathsf{Sim}_{\mathsf{VPver}}$

1. **Verifying the Sharings.** Sim follows the protocol to do the verification on behalf of each honest party.
2. **Opening Watchlists.** Sim sends honest parties' shares of each $[\mathsf{Watch}_i]_t$ to the corrupted parties. Then, Sim receives corrupted parties' shares of each $[\mathsf{Watch}_i]_t$ on behalf of each honest party. Then, Sim emulates each honest party to follow the protocol to check whether the sharings are valid degree-t sharings. If not, Sim aborts the protocol on behalf of the honest party.
3. **Opening Transcripts.** For each corrupted party P_i and each honest virtual party V_j with $j \in \mathsf{Watch}_i$, Sim follows the protocol to send the transcripts during the executions of Π_{Mult} (when performing V_j';s local computation) to P_i on behalf of the honest member emulating V_j. For each honest party P_i and each corrupted member of a virtual party V_j with $j \in \mathsf{Watch}_i$, Sim receives the transcripts during the executions of Π_{Mult} (when performing V_j's local computation) from the corrupted member.
4. **Opening Inputs.** For each corrupted party P_i and each virtual party V_j with $j \in \mathsf{Watch}_i$:
   (a) For each party P_α and each honest member of V_j, Sim follows the protocol to send the member's share of (the additive sharing of V_j's share of) each sharing in $\{[\![\boldsymbol{k}_{\boldsymbol{1}}^{(\alpha,\beta)} - \boldsymbol{k}_{\boldsymbol{0}}^{(\alpha,\beta)}]\!], [\![\boldsymbol{k}_{\boldsymbol{0}}^{(\alpha,\beta)}]\!]^{(2)}\}_{\beta=1}^{\kappa}$ to P_i on behalf of the member. If P_α is honest, Sim also sends these shares on behalf of P_α.
   (b) Sim sends the honest parties' shares of the degree-t sharings for $P_{j,1}, P_{j,2}$'s shares of each $\langle \boldsymbol{\Lambda}_i^{V_j} \rangle$ to P_i on behalf of the honest parties.
   (c) Sim sends honest parties' shares of the degree-t sharings for (the additive sharing for) V_j's share of $[\![\boldsymbol{r}^{(1)}]\!]_{\times\kappa}, \ldots, [\![\boldsymbol{r}^{(n)}]\!]_{\times\kappa}$ to P_i on behalf of the honest parties.

   For each honest party P_i and virtual party V_j with $j \in \mathsf{Watch}_i$:
   (a) For each party P_α and each corrupted member of V_j, Sim receives the corrupted member's share of (the additive sharing of V_j's share of) each sharing in $\{[\![\boldsymbol{k}_{\boldsymbol{1}}^{(\alpha,\beta)} - \boldsymbol{k}_{\boldsymbol{0}}^{(\alpha,\beta)}]\!], [\![\boldsymbol{k}_{\boldsymbol{0}}^{(\alpha,\beta)}]\!]^{(2)}\}_{\beta=1}^{\kappa}$ from the member. If P_α is corrupted, Sim also receives these shares from P_α and checks whether the received shares are the same. If not, Sim aborts the protocol on behalf of P_i. If P_α is honest, Sim checks whether the corrupted member sends the shares correctly. If not, Sim aborts the protocol on behalf of P_i.
   (b) Sim receives the corrupted parties' shares of the degree-t sharings for $P_{j,1}, P_{j,2}$'s share of each $\langle \boldsymbol{\Lambda}_i^{V_j} \rangle$ from the corrupted parties and checks whether they are correctly sent. If not, Sim aborts the protocol on behalf of P_i.
   (c) Sim receives the corrupted parties' shares of the degree-t sharings for (the additive sharing for) V_j's shares of $[\![\boldsymbol{r}^{(1)}]\!]_{\times\kappa}, \ldots, [\![\boldsymbol{r}^{(n)}]\!]_{\times\kappa}$ and checks whether they are correctly sent. If not, Sim aborts the protocol on behalf of P_i.
5. **Opening ROTs.** For each corrupted party P_i and each virtual party V_j with $j \in \mathsf{Watch}_i$, Sim samples the honest parties' shares of all the degree-t sharings for the ROT correlations between $P_{j,1}, P_{j,2}$ based on

corrupted parties' shares and the secrets, and then Sim sends them to P_i on behalf of the honest parties. For each honest party P_i and each virtual party V_j with $j \in \mathsf{Watch}_i$, Sim receives the corrupted parties' shares of all the degree-t sharings for the ROT correlations between $P_{j,1}, P_{j,2}$ and checks whether they are correctly sent. If not, Sim aborts the protocol on behalf of P_i.

6. **Checking the Computations.** Sim follows the protocol to check the computation of virtual parties on each honest party P_i's watchlist. If the computation is not correctly performed, Sim aborts the protocol on behalf of P_i.
7. If $\mathsf{CompCheck} = 1$, Sim aborts the simulation. Otherwise, Sim sets $\mathcal{H}_{\mathsf{vir}}$ to be the set of honest virtual parties V_j with $\mathsf{Corr}_j = \mathsf{Comp}_j = 0$ and $\mathcal{C}_{\mathsf{vir}}$ to be the set of the other virtual parties. Then, if all the verifications pass for an honest party, but the shares for virtual parties in $\mathcal{H}_{\mathsf{vir}}$ of some checked Σ-sharing in the first step generated by a corrupted party are not from a valid Σ-sharing, Sim aborts the simulation.

Figure 19: The simulator for Π_{VPver}.

Simulator $\mathsf{Sim}_{\mathsf{Garble}}$

1. **Transforming to PSSs of Each Gate's Labels.** For each honest member of a virtual party, Sim samples the corrupted parties' shares of the degree-d Shamir sharings generated by this honest member randomly and distributes them to the corrupted parties on behalf of the honest member. For the honest members of virtual parties in $\mathcal{C}_{\mathsf{vir}}$ among them, Sim generates the whole sharings based on the secrets and corrupted parties' shares. For each corrupted member of a virtual party, Sim receives honest parties' shares of the degree-d sharings from this corrupted member.
2. **Encrypting the Shares.**
   (a) For each wire w excepting input wires attached to a corrupted party, Sim samples a random bit as $v_w \oplus \lambda_w$. For each wire w and each honest party P_i, Sim samples $k^{(i)}_{w,v_w \oplus \lambda_w} \in \mathbb{F}_{2^\kappa}$ randomly.
   (b) For each group of gates, $\beta = 1, 2, 3, 4$, and each virtual party V_j in $\mathcal{C}_{\mathsf{vir}}$, Sim follows the protocol to compute V_j's share of $[\![\boldsymbol{k}^{(i,\alpha)}_{\boldsymbol{\Lambda}_\beta}]\!]^{(2)}$ for each honest P_i and $\alpha = 1, \ldots, \kappa$. For each corrupted party P_i, Sim samples random Σ-sharings as $[\![\boldsymbol{k}^{(i,\alpha)}_{\mathbf{1}} - \boldsymbol{k}^{(i,\alpha)}_{\mathbf{0}}]\!]$ and random $\Sigma^{(2)}$-sharings as $[\![\boldsymbol{k}^{(i,\alpha)}_{\mathbf{0}}]\!]^{(2)}$ for $\alpha = 1, \ldots, \kappa$ (including the additive sharing of each virtual party's share) from the shares for virtual parties in $\mathcal{H}_{\mathsf{vir}}$. These sharings are regarded as the sharings that should be shared by the corrupted parties. Using them, Sim reconstructs $k^{(i)}_{c,0}, k^{(i)}_{c,1}$ for each output wire c of a gate and each corrupted party P_i.
   (c) For each group of gates with input wires $a_1, \ldots, a_K, b_1, \ldots, b_K$, output wires $c_1, \ldots, c_K$, and $\beta = 1, 2, 3, 4$, Sim follows the protocol to sample corrupted parties' shares of the degree-d sharings generated by members of virtual parties in $\mathcal{C}_{\mathsf{vir}}$ (where the shares generated by honest members of them have already been sampled) and corrupted members of virtual parties in $\mathcal{H}_{\mathsf{vir}}$ and then compute the honest parties' shares based on them. Sim compares them with the received shares (including those generated for honest members of virtual parties in $\mathcal{C}_{\mathsf{vir}}$) and then computes the additive errors on these shares. Then, Sim applies the coefficients of the linear function $\Sigma^{(2)}.\mathsf{Rec}$ on them to compute the the additive error $\delta^{(i)}_{c_j,\mathcal{C}}$ brought from these shares to each honest party P_i's share of $[\boldsymbol{k}_{c_j,\Lambda_{c_j,\beta}}]_d$ for each gate with $\beta = 2(v_{a_j} \oplus \lambda_{a_j}) + (v_{b_j} \oplus \lambda_{b_j})$.
   (d) For each group of gates with input wires $a_1, \ldots, a_K, b_1, \ldots, b_K$, output wires $c_1, \ldots, c_K$, and $\beta = 1, 2, 3, 4$, for virtual parties in $\mathcal{H}_{\mathsf{vir}}$, Sim compares the honest members' shares of (the additive sharing of the shares of) $[\![\boldsymbol{k}^{(i,\alpha)}_{\mathbf{1}} - \boldsymbol{k}^{(i,\alpha)}_{\mathbf{0}}]\!]$ and $[\![\boldsymbol{k}^{(i,\alpha)}_{\mathbf{0}}]\!]^{(2)}$ for $\alpha = 1, \ldots, \kappa$ received from each corrupted P_i and the shares that should be shared generated by Sim. Sim computes the additive error on these shares and applies the sharing algorithm (the algorithm takes the secret and the shares for a fixed set of t parties containing all the corrupted parties as inputs, which is a $\mathbb{F}_{2^\kappa}$-linear and deterministic algorithm) of a degree-d sharing on them to compute the additive errors on the honest parties' shares of the degree-d sharings generated by honest members of virtual parties in $\mathcal{H}_{\mathsf{vir}}$. Applying the coefficients of the linear function $\Sigma^{(2)}.\mathsf{Rec}$ on them, Sim computes the additive error $\delta^{(i)}_{c_j,\mathcal{H}}$ brought from these shares to each honest party P_i's share of $[\boldsymbol{k}_{c_j,\Lambda_{c_j,\beta}}]_d$ for each gate with $\beta = 2(v_{a_j} \oplus \lambda_{a_j}) + (v_{b_j} \oplus \lambda_{b_j})$.
   (e) For each group of gates with input wires $a_1, \ldots, a_K, b_1, \ldots, b_K$, output wires $c_1, \ldots, c_K$, and $\beta = 1, 2, 3, 4$, Sim follows the protocol to compute corrupted parties' shares of share of $[\boldsymbol{k}_{c_j,\Lambda_{c_j,\beta}}]_d$ for

each gate with $\beta = 2(v_{a_j} \oplus \lambda_{a_j}) + (v_{b_j} \oplus \lambda_{b_j})$ from all their shares of degree-d sharings. Then, Sim samples honest parties' shares of $[\boldsymbol{k}_{c_j,\Lambda_{c_j},\beta}]_d$ for each gate with $\beta = 2(v_{a_j} \oplus \lambda_{a_j}) + (v_{b_j} \oplus \lambda_{b_j})$ based on the corrupted parties' shares and $k^{(i)}_{c_j,0}, k^{(i)}_{c_j,1}$ for each corrupted party P_i. Sim then add the additive error $\delta^{(\alpha)}_{c_j,\mathcal{H}} + \delta^{(\alpha)}_{c_j,\mathcal{C}}$ to each honest party P_α's share of it.

(f) For each honest party P_j and each gate g with input wires a, b and c, Sim computes

$$\begin{aligned}\mathsf{ct}^{(j)}_{g,2(v_a\oplus\lambda_a)+(v_b\oplus\lambda_b)} &= \mathsf{Sh}^{(j)}_{c,2(v_a\oplus\lambda_a)+(v_b\oplus\lambda_b)} \\ &\oplus \mathsf{PRF}(k^{(j)}_{a,v_a\oplus\lambda_a}, (2(v_a \oplus \lambda_a) + (v_b \oplus \lambda_b))\|(v_a \oplus \lambda_a)\|j\|g) \\ &\oplus \mathsf{PRF}(k^{(j)}_{b,v_b\oplus\lambda_b}, (2(v_a \oplus \lambda_a) + (v_b \oplus \lambda_b))\|(v_b \oplus \lambda_b)\|j\|g).\end{aligned}$$

Then, Sim samples three random strings with the same length as the three other ciphertexts for this gate.

3. **Sending Ciphertexts.** Sim sends $\mathsf{ct}^{(j)}_{g,1}, \mathsf{ct}^{(j)}_{g,2}, \mathsf{ct}^{(j)}_{g,3}, \mathsf{ct}^{(j)}_{g,4}$ for each gate g to P_{king} on behalf of each honest party P_j.

Figure 20: The simulator for Π_{Garble}.

Simulator $\mathsf{Sim}_{\mathsf{Eval}}$

1. **Computing Masked Inputs.**
   (a) For each input wire w attached to a corrupted party P_i, Sim samples the honest parties' shares of $[\lambda_w]_t$ based on the corrupted parties' shares and sends them to P_i on behalf of the honest parties. For each input wire w attached to a corrupted party P_i, Sim receives the corrupted parties' shares of $[\lambda_w]_t$ from them and checks whether they are correctly sent. If not, Sim aborts the protocol on behalf of P_i. Then, for each input wire w attached to a corrupted party P_i, Sim receives $v_w \oplus \lambda_w$ from P_i. For each input wire w attached to an honest party P_i, Sim sends $v_w \oplus \lambda_w$ to all the corrupted parties on behalf of P_i.
   (b) Sim emulates $\mathcal{F}_{\mathsf{Coin}}$ to receive $\mathsf{RandCoin}$ from all the corrupted parties. If abort_i is received, Sim aborts the protocol on behalf of P_i. Then, Sim samples $s \in \mathbb{F}_{2^\kappa}$ randomly, emulates $\mathcal{F}_{\mathsf{Coin}}$ to send it to all the corrupted parties, and expands it to get $(s_1, \ldots, s_{G_I}) \in \mathbb{F}^{G_I}_{2^\kappa}$.
   (c) Sim follows the protocol to compute τ and sends it to all the corrupted parties on behalf of each honest party. Sim then receives τ from the corrupted parties and then follows the protocol to check whether all these τ are all the same on behalf of each honest party. If not, Sim aborts the protocol on behalf of the honest party.
   (d) If the check passes but for an input wire w attached to a corrupted party P_i, the received $v_w \oplus \lambda_w$ by all the honest parties are not the same, Sim aborts the simulation.
   (e) For each input wire w attached to a corrupted party, Sim sends $v_w = (v_w \oplus \lambda_w) \oplus \lambda_w$ as the circuit input on wire w to $\mathcal{F}$.
2. **Sending Input Labels.** For each input wire w, Sim sends $k^{(i)}_{w,x_w\oplus\lambda_w}$ to P_{king} on behalf of each honest party P_i.
3. **Sending Output Labels.** For each output wire w attached to each honest party P_i, Sim receives $k^{(i)}_{w,v_w\oplus\lambda_w}, v_w \oplus \lambda_w$ from P_{king}.
4. **Checking Output Labels.** For each output wire w attached to each honest party P_i, Sim checks $k^{(i)}_{w,v_w\oplus\lambda_w}, v_w \oplus \lambda_w$ match what he generates by himself. If not, Sim aborts the protocol on behalf of P_i. After the simulation terminates, Sim outputs what $\mathcal{A}$ outputs.
5. **Computing Outputs.** For each output wire w attached to an honest party P_i, Sim receives corrupted parties' shares of $[\lambda_w]_t$ from the corrupted parties and checks whether they are correctly sent. If not, Sim aborts the protocol on behalf of P_i. For each output wire w attached to a corrupted party P_i, Sim obtain v_w from $\mathcal{F}$ and computes λ_w by $(v_w \oplus \lambda_w) \oplus v_w$. Then Sim samples the honest parties' shares of $[\lambda_w]_t$ based on the secret and corrupted parties' shares. Finally, Sim sends the honest parties' shares to P_i on behalf of the honest parties.

Figure 21: The simulator for Π_{Eval}.

We construct the following hybrids:

$\mathbf{Hyb}_0$: In this hybrid, Sim learns all the honest parties' inputs and runs the protocol honestly. This corresponds to the real-world scenario.

$\mathbf{Hyb}_1$: In this hybrid, Sim additionally sets $\mathsf{Corr}_i = \mathsf{Comp}_i = 0$ for each honest virtual party V_i and $\mathsf{CompCheck} = 0$ at the beginning of the simulation. This doesn't affect the output distribution. Thus, $\mathbf{Hyb}_1$ and $\mathbf{Hyb}_0$ have the same output distribution.

$\mathbf{Hyb}_2$: In this hybrid, whenever an honest party generates a sharing (including degree-t Shamir sharing, degree-d Shamir sharing, $\Sigma, \Sigma^{(2)}$-sharing, and additive sharing), Sim first generates the corrupted parties' (or corrupted virtual parties') shares, and then randomly samples the honest parties' (or honest virtual paries') shares based on corrupted parties' (or corrupted virtual parties') shares and the secret. Since for all these sharings, the set of corrupted parties' (or corrupted virtual parties') shares is independent of the secret, we only change the order of generating the honest and corrupted parties' (or virtual parties') shares of the sharings. This doesn't change the output distribution. Thus, $\mathbf{Hyb}_2$ and $\mathbf{Hyb}_1$ have the same output distribution.

$\mathbf{Hyb}_3$: In this hybrid, for each corrupted P_i, Sim reconstructs Watch_i from honest parties' shares of $[\mathsf{Watch}_i]_t$ (if valid). For each $j \in \mathsf{Watch}_i$, Sim additionally sets $\mathsf{Corr}_j = 1$. This doesn't affect the output distribution. Thus, $\mathbf{Hyb}_3$ and $\mathbf{Hyb}_2$ have the same output distribution.

$\mathbf{Hyb}_4$: In this hybrid, whenever an honest party generates a $\Sigma, \Sigma^{(2)}$-sharing, after generating corrupted virtual parties' shares, Sim first sample the shares for those virtual parties V_j with $\mathsf{Corr}_j = 1$ based on the corrupted parties' shares and then samples the shares for other virtual parties based on the secret and the sampled shares. Among these virtual parties V_j with $\mathsf{Corr}_j = 1$, Sim samples the additive sharings (generated by honest parties) among the members based on corrupted members' shares and the secrets. Since there are at most $t \cdot N/12n < N/24$ virtual parties V_j with $\mathsf{Corr}_j = 1$ and the number of corrupted virtual parties is $N/4$, we have $N/24 + N/4 = 7N/24 < N/3$, which means we can sample these virtual parties' shares of each $\Sigma, \Sigma^{(2)}$-sharing based on the corrupted virtual parties' shares without the secret (see Remark 1). Therefore, we only change the order of generating the honest virtual parties' shares of these sharings. This doesn't change the output distribution. Thus, $\mathbf{Hyb}_4$ and $\mathbf{Hyb}_3$ have the same output distribution.

$\mathbf{Hyb}_5$: In this hybrid, while sharing the colored bits to virtual parties, we change the way Sim checks the degree-t sharing for each honest member (of V_j)'s share of $\langle \mathbf{\Lambda}_i^{V_j} \rangle$ for each group of gates and $i = 1, 2, 3, 4$. Instead of checking whether the received shares are from a valid degree-t Shamir sharing, Sim directly checks whether corrupted parties send their shares correctly computed from their shares of $[\Lambda_{c_j,i}]_t$ for $j = 1, \ldots, K$, $[b_1]_t, \ldots, [b_{N\ell}]_t$, and $[b'_1]_t, \ldots, [b'_{N\ell}]_t$. Since the $\geq n - t$ shares generated by honest parties uniquely determine the degree-t sharing, if corrupted parties don't send the shares correctly, the honest member must find that the shares are not from a valid degree-t sharing. Therefore, the two ways of verification lead to the same result. Thus, $\mathbf{Hyb}_5$ and $\mathbf{Hyb}_4$ have the same output distribution.

$\mathbf{Hyb}_6$: In this hybrid, while performing the local computations of virtual parties, during each execution of Π_{Mult} executed by an honest party P_α (who acts as P_1 in Π_{Mult}) and a corrupted party P_β (who acts as P_2 in Π_{Mult}) while emulating a virtual party V_j with $\mathsf{Corr}_j = 0$, Sim doesn't follow the protocol to compute $\boldsymbol{u}_1^{(1)} + \boldsymbol{u}_0^{(1)} + \boldsymbol{u}_1$ and $y_1 + b^{(2)}$. Instead, Sim samples a random string in $\mathbb{F}_2^{n\kappa}$ as $\boldsymbol{u}_1^{(1)} + \boldsymbol{u}_0^{(1)} + \boldsymbol{x}_1$ and a random bit as $y_1 + b^{(2)}$. Note that $k^{(1)}_{b^{(1)} \oplus 1}, b^{(2)}$ are not used in the computation, Sim do not generate them in future hybrids.

For $\boldsymbol{u}_1^{(1)} + \boldsymbol{u}_0^{(1)} + \boldsymbol{x}_1$, $\boldsymbol{u}^{(1)}_{b^{(1)} \oplus 1}$ is expanded from $k^{(1)}_{b^{(1)} \oplus 1}$, and $k^{(1)}_{b^{(1)} \oplus 1}$ is a randomly sampled PRG seed in $\mathbb{F}_2^\kappa$. Therefore, $\boldsymbol{u}^{(1)}_{b^{(1)} \oplus 1}$ is computationally indistinguishable from a uniformly random string in $\mathbb{F}_2^{n\kappa}$. Thus, $\boldsymbol{u}_1^{(1)} + \boldsymbol{u}_0^{(1)} + \boldsymbol{x}_1$ is also computationally indistinguishable from a uniformly random string in $\mathbb{F}_2^{n\kappa}$. In addition, $b^{(2)}$ is a randomly sampled bit, so $y_1 + b^{(2)}$ is also uniformly random. Then, if a distinguisher can distinguish between the two hybrids, we can let the distribution of $\mathbf{Hyb}_6$ be represented as a function output whose input is a uniformly random string $\boldsymbol{u}^{(1)}_{b^{(1)} \oplus 1}$, and the distribution of $\mathbf{Hyb}_6$ is the output of the same function where $\boldsymbol{u}^{(1)}_{b^{(1)} \oplus 1}$ is computed by the PRG with seed $k^{(1)}_{b^{(1)} \oplus 1}$. In this way, we can apply the distinguisher

and distinguish $\boldsymbol{u}^{(1)}_{b^{(1)}\oplus 1}$ from $\mathsf{PRG}(k^{(1)}_{b^{(1)}\oplus 1})$ in polynomial time, which leads to a contradiction. Thus, the distributions of $\mathbf{Hyb}_6$ and $\mathbf{Hyb}_5$ are computationally indistinguishable.

$\mathbf{Hyb}_7$: In this hybrid, while performing the local computations of virtual parties, during each execution of Π_{Mult} executed by an honest party P_α (who acts as P_1 in Π_{Mult}) and a corrupted party P_β (who acts as P_2 in Π_{Mult}) while emulating a virtual party V_j with $\mathsf{Corr}_j = 0$, Sim additionally checks whether $y_2 + b^{(1)}, \boldsymbol{u}^{(2)}_1 + \boldsymbol{u}^{(2)}_0 + \boldsymbol{x}_2$ received from P_β are computed correctly from $b^{(1)}, k^{(2)}_1, k^{(2)}_0$, the corrupted members' shares of $\{[\![\boldsymbol{k}^{(i,\gamma)}_{\boldsymbol{1}} - \boldsymbol{k}^{(i,\gamma)}_{\boldsymbol{0}}]\!]\}_{\gamma\in\{1,\ldots,\kappa\}}$, and the corrupted members' shares of $[\![\boldsymbol{\Lambda}_1]\!], [\![\boldsymbol{\Lambda}_2]\!], [\![\boldsymbol{\Lambda}_3]\!], [\![\boldsymbol{\Lambda}_4]\!]$ for each honest party P_i. Sim also checks whether $y_2 + b^{(1)}$ received from P_β are computed correctly from $b^{(1)}$ and the corrupted members' shares of $[\![\boldsymbol{\Lambda}_1]\!], [\![\boldsymbol{\Lambda}_2]\!], [\![\boldsymbol{\Lambda}_3]\!], [\![\boldsymbol{\Lambda}_4]\!]$ for each corrupted party P_i. If not, Sim sets $\mathsf{Comp}_j = 1$. Then, for each corrupted party P_i, Sim computes P_β's share of the additive sharing for V_j's shares of $\{[\![\boldsymbol{k}^{(i,\gamma)}_{\boldsymbol{1}} - \boldsymbol{k}^{(i,\gamma)}_{\boldsymbol{0}}]\!]\}^{\kappa}_{\gamma=1}$ based on $k^{(2)}_1, k^{(2)}_0$ and $\boldsymbol{u}^{(2)}_1 + \boldsymbol{u}^{(2)}_0 + \boldsymbol{x}_2$. This doesn't affect the output distribution. Thus, $\mathbf{Hyb}_7$ and $\mathbf{Hyb}_6$ have the same output distribution.

$\mathbf{Hyb}_8$: In this hybrid, while reconstructing each degree-t sharing for an honest member's share of $[\![\boldsymbol{r}^{(i)}]\!]$ for each honest P_i during the verification of Σ-sharings, Sim does not follow the protocol to check the validity of corrupted parties' shares. Instead, Sim directly checks whether corrupted parties correctly sent these shares. If not, Sim aborts the protocol on behalf of the honest member. Since the $\geq n - t$ shares generated by honest parties uniquely determine the degree-t sharing, if corrupted parties don't send the shares correctly, the honest member must find that the shares are not from a valid degree-t sharing. Therefore, the two ways of verification lead to the same result. Thus, $\mathbf{Hyb}_8$ and $\mathbf{Hyb}_7$ have the same output distribution.

$\mathbf{Hyb}_9$: In this hybrid, after performing the local computations of virtual parties, if there are more than $N/24$ virtual parties V_j with $\mathsf{Comp}_j = 1$, Sim sets $\mathsf{CompCheck} = 1$. Otherwise, for each virtual party V_j with $\mathsf{Comp}_j = 1$, Sim samples V_j's shares of $[\![\boldsymbol{k}^{(i,\alpha)}_{\boldsymbol{1}} - \boldsymbol{k}^{(i,\alpha)}_{\boldsymbol{0}}]\!]$ and $[\![\boldsymbol{k}^{(i,\alpha)}_{\boldsymbol{0}}]\!]^{(2)}$ for $\alpha = 1, \ldots, \kappa$ and each honest P_i randomly based on the shares for corrupted virtual parties and honest virtual parties V_j with $\mathsf{Corr}_j = 1$. The shares for each V_j with $\mathsf{Corr}_j = \mathsf{Comp}_j = 0$ are sampled based on other virtual parties' shares finally. The sampling process is delayed until this step since these shares are not used in the previous simulation. Note that the number of corrupted virtual parties and honest virtual parties with $\mathsf{Corr}_j = 1$ or $\mathsf{Comp}_j = 1$ is no more than $N/4 + N/24 + N/24 = N/3 = T$. We can sample the share for virtual parties V_j with $\mathsf{Comp}_j = 1$ of each $\Sigma, \Sigma^{(2)}$-sharing based on the corrupted virtual parties' shares and the shares for honest virtual parties V_j with $\mathsf{Corr}_j = 1$ without the secret (see Remark 1). Therefore, we only change the order of generating these virtual parties' shares of these sharings. This doesn't change the output distribution. Thus, $\mathbf{Hyb}_9$ and $\mathbf{Hyb}_8$ have the same output distribution.

$\mathbf{Hyb}_{10}$: In this hybrid, while simulating Π_{VPver}, for each honest party P_i and virtual party V_j with $j \in \mathsf{Watch}_i$, Sim does not follow the protocol to check the degree-t sharings for $\langle \boldsymbol{\Lambda}^{V_j}_i \rangle$ and the ROT correlations prepared for $P_{j,1}, P_{j,2}$. Instead, Sim directly checks whether corrupted parties' shares are correctly sent. Since the $\geq n - t$ shares generated by honest parties uniquely determine the degree-t sharing, if corrupted parties don't send the shares correctly, the honest member must find that the shares are not from a valid degree-t sharing. Therefore, the two ways of verification lead to the same result. Thus, $\mathbf{Hyb}_{10}$ and $\mathbf{Hyb}_9$ have the same output distribution.

$\mathbf{Hyb}_{11}$: In this hybrid, after checking the computations of all the virtual parties on the honest parties' watchlist, Sim aborts the simulation if $\mathsf{CompCheck} = 1$, and sets $\mathcal{H}_{\mathsf{vir}}$ to be the set of honest virtual parties V_j with $\mathsf{Corr}_j = \mathsf{Comp}_j = 0$ and $\mathcal{C}_{\mathsf{vir}}$ to be the set of the other virtual parties otherwise. Note that setting $\mathcal{H}_{\mathsf{vir}}$ and $\mathcal{C}_{\mathsf{vir}}$ does not affect the output distribution. The distribution only changes when the check of all the honest parties passes with $\mathsf{CompCheck} = 1$, where $\mathsf{CompCheck} = 1$ only happens when at least $N/24$ virtual parties does not perform their local multiplications correctly, which can be detected if the virtual parties are on an honest party's watchlist. The probability that each honest party doesn't choose these virtual parties on his watchlist is

$$\frac{\frac{23N}{24}}{N} \cdot \frac{\frac{23N}{24} - 1}{N - 1} \cdot \cdots \cdot \frac{\frac{23N}{24} - \frac{N}{24n} + 1}{N - \frac{N}{24n} + 1} > \left(1 - \frac{1}{24}\right)^{\frac{N}{24n}}.$$

Thus, the probability that the $N/24$ virtual parties are not chosen by all the honest parties is at most $(23/24)^{(N/48)}$. Recall that $N = \Theta(n^2 + \kappa)$, the probability is negligible. Thus, the distributions of $\mathbf{Hyb}_{11}$

and $\mathbf{Hyb}_{10}$ are statistically close.

$\mathbf{Hyb}_{12}$: In this hybrid, after checking the computations of all the virtual parties on the honest parties' watchlist, Sim aborts the simulation if $\mathsf{CompCheck} = 0$ but the shares for virtual parties in $\mathcal{H}_{\mathsf{vir}}$ of some checked Σ-sharing in the first step generated by a corrupted party are not from a valid Σ-sharing.

Assume that the shares for virtual parties in $\mathcal{H}_{\mathsf{vir}}$ of a Σ-sharing are not valid, and the random coefficient on this sharing in $[\![\sigma]\!]_{\times\kappa} = \sum_{j=1}^{k_1} r_j \cdot [\![\boldsymbol{x}_j]\!]_{\times\kappa} + \sum_{i=1}^{n} [\![\boldsymbol{r}^{(i)}]\!]_{\times\kappa}$ is $r_j \in \mathbb{F}_{2^\kappa}$. If $r_1, \ldots, r_{k_1} \in \mathbb{F}_{2^{\kappa'}}$ are all truly random, we can sample r_j after the invalid sharing is fixed. If there exists $r'_j \neq r_j \in \mathbb{F}_{2^{\kappa'}}$ such that both r_j and r'_j (serve as the coefficient on the invalid sharing) lead to a valid $[\![\sigma]\!]_{\times\kappa}$, then the invalid sharing (which has been embedded in a $\Sigma_{\times\kappa}$-sharing) is $(r'_j - r_j)^{-1}$ times a valid $\Sigma_{\times\kappa}$-sharing, which must be a valid $\Sigma_{\times\kappa}$-sharing, and this leads to a contradiction. Thus, there is only one element $r_j \in \mathbb{F}_{2^\kappa}$ that can make $[\![\sigma]\!]_{\times\kappa}$ pass the check. The probability is $2^{-\kappa}$. Thus, if there is a non-negligible probability that $[\![\sigma]\!]_{\times\kappa}$ is valid, then the truly random field elements and the pseudo-random values $r_1, \ldots, r_{k_1} \in \mathbb{F}_{2^\kappa}$ can be distinguished by computing $[\![\sigma]\!]_{\times\kappa}$ with a non-negligible probability, which contradicts the definition of a PRG, so the probability that $[\![\sigma]\!]_\kappa$ is valid is negligible. Thus, the distributions of $\mathbf{Hyb}_{12}$ and $\mathbf{Hyb}_{11}$ are statistically close.

$\mathbf{Hyb}_{13}$: In this hybrid, Sim doesn't follow the protocol to generate λ_w for each wire w of the circuit except the input wires attached to corrupted parties. Instead, Sim directly samples a random bit as $v_w \oplus \lambda_w$ for these wires. While simulating Π_{Eval}, after getting all the input values from $v_w \oplus \lambda_w$ received from corrupted parties for those input wires attached to them, Sim uses corrupted parties' inputs (i.e., v_w of these wires) to evaluate the whole circuit. For each wire w except those input wires attached to corrupted parties, Sim obtains v_w from the evaluation and then computes λ_w by $(v_w \oplus \lambda_w) \oplus v_w$. Sim also delays the generation of honest parties' shares of $[\lambda_w]_t$ for these wires until the corrupted parties' inputs are determined.

Since λ_w is a random bit for each wire w that is not an input wire attached to a corrupted party, so is $v_w \oplus \lambda_w$. We only change the order of sampling λ_w and $v_w \oplus \lambda_w$ without changing their distributions. In addition, λ_w and the honest parties' shares of $[\lambda_w]_t$ for these wires are not used in the simulation before the corrupted parties' inputs are determined. Delaying the generation of them does not affect the output distribution. Thus, $\mathbf{Hyb}_{13}$ and $\mathbf{Hyb}_{12}$ have the same output distribution.

$\mathbf{Hyb}_{14}$: In this hybrid, during the evaluation, after receiving $k^{(i)}_{w,v_w\oplus\lambda_w}, v_w \oplus \lambda_w$ for each output wire w attached to an honest party, Sim doesn't follow the protocol to check them. Instead, Sim checks whether they matches $v_w \oplus \lambda_w$ and $k^{(i)}_{w,v_w\oplus\lambda_w}$ generated by him. This only changes the output distribution when Sim receives $k^{(i)}_{w,v_w\oplus\lambda_w\oplus 1}, v_w \oplus \lambda_w \oplus 1$ from P_{king} or $k^{(i)}_{w,0} = k^{(i)}_{w,1}$. Note that $k^{(i)}_{w,v_w\oplus\lambda_w\oplus 1}$ is a random string in $\mathbb{F}_2^\kappa$ and it is not used in computing any message sent to the corrupted parties, we can sample it after $k^{(i)}_{w,v_w\oplus\lambda_w}, v_w \oplus \lambda_w$ are received. Thus, the probability that the received message happens to match $k^{(i)}_{w,v_w\oplus\lambda_w\oplus 1}$ is $2^{-\kappa}$, which is negligible. The probability that $k^{(i)}_{w,0} = k^{(i)}_{w,1}$ is also $2^{-\kappa}$, which is negligible. Therefore, the output distribution only changes with a negligible probability. Thus, the distributions of $\mathbf{Hyb}_{14}$ and $\mathbf{Hyb}_{13}$ are statistically close.

$\mathbf{Hyb}_{15}$: In this hybrid, Sim additionally does some computation. Sim samples random Σ-sharings as $[\![\boldsymbol{k}^{(i,\alpha)}_{\boldsymbol{1}} - \boldsymbol{k}^{(i,\alpha)}_{\boldsymbol{0}}]\!]$ and random $\Sigma^{(2)}$-sharings as $[\![\boldsymbol{k}^{(i,\alpha)}_{\boldsymbol{0}}]\!]^{(2)}$ for $\alpha = 1, \ldots, \kappa$ (including the additive sharing of each virtual party's share) based on the shares for virtual parties in $\mathcal{H}_{\mathsf{vir}}$. These sharings are regarded as the sharings that should be shared by the corrupted parties. Using them, Sim reconstructs $k^{(i)}_{c,0}, k^{(i)}_{c,1}$ for each output wire c of a gate and each corrupted party P_i. For each group of gates with input wires $a_1, \ldots, a_K, b_1, \ldots, b_K$, output wires $c_1, \ldots, c_K$, and $\beta = 1, 2, 3, 4$, Sim follows the protocol to sample corrupted parties' shares of the degree-d sharings generated by members of virtual parties in $\mathcal{C}_{\mathsf{vir}}$ (where the shares generated by honest members of them have already been sampled) and corrupted members of virtual parties in $\mathcal{H}_{\mathsf{vir}}$ and then compute the honest parties' shares based on them. Sim compares them with the received shares (including those generated for honest members of virtual parties in $\mathcal{C}_{\mathsf{vir}}$) and then computes the additive errors on these shares. Then, Sim applies the coefficients of the linear function $\Sigma^{(2)}.\mathsf{Rec}$ on them to compute the the additive error $\delta^{(i)}_{c_j,\mathcal{C}}$ brought from these shares to each honest party P_i's share of $[\boldsymbol{k}_{c_j,\Lambda_{c_j},\beta}]_d$ for each gate with $\beta = 2(v_{a_j} \oplus \lambda_{a_j}) + (v_{b_j} \oplus \lambda_{b_j})$. Moreover, for virtual parties in $\mathcal{H}_{\mathsf{vir}}$, Sim compares the honest

members' shares of (the additive sharing of the shares of) $[\![\boldsymbol{k}_{\mathbf{1}}^{(i,\alpha)} - \boldsymbol{k}_{\mathbf{0}}^{(i,\alpha)}]\!]$ and $[\![\boldsymbol{k}_{\mathbf{0}}^{(i,\alpha)}]\!]^{(2)}$ for $\alpha = 1, \ldots, \kappa$ received from each corrupted P_i and the shares that should be shared generated by Sim. Sim computes the additive error on these shares and applies the sharing algorithm of a degree-d sharing on them to compute the additive errors on the honest parties' shares of the degree-d sharings generated by honest members of virtual parties in $\mathcal{H}_{\mathsf{vir}}$. Applying the coefficients of the linear function $\Sigma^{(2)}.\mathsf{Rec}$ on them, Sim computes the additive error $\delta^{(i)}_{c_j,\mathcal{H}}$ brought from these shares to each honest party P_i's share of $[\boldsymbol{k}_{c_j,\Lambda_{c_j,\beta}}]_d$ for each gate with $\beta = 2(v_{a_j} \oplus \lambda_{a_j}) + (v_{b_j} \oplus \lambda_{b_j})$. This does not affect the output distribution. Thus, $\mathbf{Hyb}_{15}$ and $\mathbf{Hyb}_{14}$ have the same output distribution.

$\mathbf{Hyb}_{16}$: In this hybrid, Sim does not follow the protocol to compute honest parties' shares of each $[\boldsymbol{k}_{c,\Lambda_{c,2(v_a\oplus\lambda_a)+(v_b\oplus\lambda_b)}}]_d$. Instead, Sim follows the protocol to compute the corrupted parties' shares and samples the honest parties' shares based on the corrupted parties' shares and $k^{(i)}_{c,0}, k^{(i)}_{c,1}$ for each corrupted party P_i. Finally, Sim adds the additive errors to them. In addition, for each gate, Sim doesn't follow the protocol to encrypt the ciphertexts except $\mathsf{ct}^{(j)}_{g,2(v_a\oplus\lambda_a)+(v_b\oplus\lambda_b)}$ on behalf of each honest party P_j. Instead, Sim samples random strings as the other 3 ciphertexts. For each virtual party V_j in $\mathcal{H}_{\mathsf{vir}}$ and honest party P_i, Sim does not sample V_j's share of $[\![\boldsymbol{k}_{\mathbf{1}}^{(i,\alpha)} - \boldsymbol{k}_{\mathbf{0}}^{(i,\alpha)}]\!]$ and $[\![\boldsymbol{k}_{\mathbf{0}}^{(i,\alpha)}]\!]^{(2)}$ and then computes the honest members' shares of the additive sharing of them in future hybrids. Sim also does not sample honest parties' shares of the degree-d sharings generated by honest members of virtual parties in $\mathcal{H}_{\mathsf{vir}}$ in future hybrids.

To show that the distributions of $\mathbf{Hyb}_{16}$ and $\mathbf{Hyb}_{15}$ are computationally indistinguishable, we construct the following hybrids between $\mathbf{Hyb}_{15}$ and $\mathbf{Hyb}_{16}$.

$\mathbf{Hyb}_{16.1.1}$: In this hybrid, for each gate in the first layer in C with input wires a, b and output wire c, Sim does not follow the protocol to compute honest parties' shares of $[\boldsymbol{k}_{c,\Lambda_{c,2(v_a\oplus\lambda_a\oplus1)+(v_b\oplus\lambda_b)}}]_d$. Instead, Sim first follows the protocol to compute what these shares should be by replacing what honest parties received with the messages computed by Sim via emulating corrupted parties to run the protocol honestly. Finally, Sim computes the additive errors on them, where the additive errors are computed by the subtraction of the messages the additive errors on the parties/virtual parties' shares of $\Sigma, \Sigma^{(2)}$-sharings and degree-d Shamir sharings (as we computed in the last hybrid). Finally, Sim adds them up.

Since the errors in honest parties' shares of $[\boldsymbol{k}_{c,\Lambda_{c,2(v_a\oplus\lambda_a\oplus1)+(v_b\oplus\lambda_b)}}]_d$ can be computed deterministically and linearly from the additive errors on the parties/virtual parties' shares of $\Sigma, \Sigma^{(2)}$-sharings and degree-d Shamir sharings, the distribution of the honest parties' shares of $[\boldsymbol{k}_{c,\Lambda_{c,2(v_a\oplus\lambda_a\oplus1)+(v_b\oplus\lambda_b)}}]_d$ for all the gates generated in both hybrids is the same. Thus, $\mathbf{Hyb}_{16.1.1}$ and $\mathbf{Hyb}_{15}$ have the same output distribution.

$\mathbf{Hyb}_{16.1.2}$: In this hybrid, for each gate in the first layer in C with input wires a, b and output wire c, Sim doesn't simulate the whole process of computing honest parties' shares of $[\boldsymbol{k}_{c,\Lambda_{c,2(v_a\oplus\lambda_a\oplus1)+(v_b\oplus\lambda_b)}}]_d$ (before adding the additive errors) via emulating corrupted parties to run the protocol honestly. Instead, Sim directly samples them based on $k^{(i)}_{c,0}, k^{(i)}_{c,1}$ for each corrupted P_i and the corrupted parties' shares. Then, for the group of gates containing c, the honest parties' shares of degree-d sharings generated by honest members of virtual parties in $\mathcal{H}_{\mathsf{vir}}$ are sampled based on their shares of $[\boldsymbol{k}_{c,\Lambda_{c,2(v_a\oplus\lambda_a\oplus1)+(v_b\oplus\lambda_b)}}]_d$, the corrupted parties' shares, and the secrets. For $[\boldsymbol{k}_{c,\Lambda_{c,\beta}}]_d$ with $\beta \neq 2(v_a \oplus \lambda_a \oplus 1) + (v_b \oplus \lambda_b)$, Sim still follows the protocol to compute honest parties' shares.

Note that for each group of gates, honest parties' shares of the degree-d sharings generated by honest members of virtual parties in $\mathcal{H}_{\mathsf{vir}}$ are randomly sampled based on the secret and corrupted parties' shares. If corrupted parties all run the protocol honestly, the honest parties' shares of $[\boldsymbol{k}_{c,2(v_a\oplus\lambda_a\oplus1)+(v_b\oplus\lambda_b)}]_d$ should be random based on corrupted parties' shares and the secret. This is because when we consider the $\Sigma^{(2)}$-sharing of honest parties' shares of $[\boldsymbol{k}_{c,\Lambda_{c,2(v_a\oplus\lambda_a\oplus1)+(v_b\oplus\lambda_b)}}]_d$, we know that the secret of a $\Sigma^{(2)}$-sharing is independent of the shares for $\leq N/3$ virtual parties in $\mathcal{C}_{\mathsf{vir}}$, and the shares for virtual parties in $\mathcal{H}_{\mathsf{vir}}$ are random (based on corrupted parties' shares and the secret of the degree-d sharings). Note that $k^{(i)}_{c,0}, k^{(i)}_{c,1}$ for each honest party P_i are not used in the simulation before this step. We can delay sampling them here. Thus, the honest parties' shares of $[\boldsymbol{k}_{c,\Lambda_{c,2(v_a\oplus\lambda_a\oplus1)+(v_b\oplus\lambda_b)}}]_d$ are random based on corrupted parties' shares and $k^{(i)}_{c,0}, k^{(i)}_{c,1}$ for each corrupted party P_i. Thus, $\mathbf{Hyb}_{16.1.2}$ and $\mathbf{Hyb}_{16.1.1}$ have the same output distribution.

$\mathbf{Hyb}_{16.1.3}$: In this hybrid, for each gate g with input wires a, b and output wire c in the first layer

of the circuit, for each honest party P_i, Sim does not follow the protocol to compute the ciphertext $\mathsf{ct}^{(i)}_{g,2(v_a\oplus\lambda_a\oplus 1)+(v_b\oplus\lambda_b)}$. Instead, Sim samples a random string as it.

Note that the computation of $[\boldsymbol{k}_{c,2(v_a\oplus\lambda_a\oplus 1)+(v_b\oplus\lambda_b\oplus 1)}]_d$ doesn't affect the encryption of $[\boldsymbol{k}_{c,2(v_a\oplus\lambda_a\oplus 1)+(v_b\oplus\lambda_b)}]_d$, we can delay the generation and encryption of $[\boldsymbol{k}_{c,2(v_a\oplus\lambda_a\oplus 1)+(v_b\oplus\lambda_b\oplus 1)}]_d$ after $\mathsf{ct}^{(i)}_{g,2(v_a\oplus\lambda_a\oplus 1)+(v_b\oplus\lambda_b)}$ for each honest party P_i is computed. Then, $k^{(i)}_{v_a\oplus\lambda_a\oplus 1}$ is not used in the simulation until encrypting P_i's share of $[\boldsymbol{k}_{c,2(v_a\oplus\lambda_a\oplus 1)+(v_b\oplus\lambda_b\oplus 1)}]_d$. Thus, we can delay the generation of them until this step. Since $k^{(i)}_{v_a\oplus\lambda_a\oplus 1}$ is randomly sampled in $\mathbb{F}_2^\kappa$, the PRF result computed with key $k^{(i)}_{v_a\oplus\lambda_a\oplus 1}$ is computationally indistinguishable from random strings. Thus, the ciphertext is also computationally indistinguishable from a random string. If there exists a PPT distinguisher that can distinguish the distribution of the two hybrids, we can construct an algorithm to compute the two hybrids from this ciphertext and apply the distinguisher. Then, by inputting the 3 ciphertexts and random strings to the algorithm respectively, we can distinguish the ciphertexts from random strings, which leads to a contradiction. Thus, the distributions of $\mathbf{Hyb}_{16.1.3}$ and $\mathbf{Hyb}_{16.1.2}$ are computationally indistinguishable.

Now, for each gate in the first layer in C with input wires a, b and output wire c, we don't need to generate honest parties' shares of $[\boldsymbol{k}_{c,2(v_a\oplus\lambda_a\oplus 1)+(v_b\oplus\lambda_b\oplus 1)}]_d$ as they are not used in the simulation. Sim doesn't generate them in future hybrids.

$\mathbf{Hyb}_{16.1.4}$: In this hybrid, for each gate in the first layer in C with input wires a, b and output wire c, Sim does not follow the protocol to compute honest parties' shares of $[\boldsymbol{k}_{c,\Lambda_{c,2(v_a\oplus\lambda_a)+(v_b\oplus\lambda_b\oplus 1)}}]_d$. Instead, Sim first follows the protocol to compute what these shares should be by replacing what honest parties received with the messages computed by Sim via emulating corrupted parties to run the protocol honestly. Finally, Sim computes the additive errors on them, where the additive errors are computed by the subtractions of the messages the additive errors on the parties/virtual parties' shares of $\Sigma, \Sigma^{(2)}$-sharings and degree-d Shamir sharings. Finally, Sim adds them up. For the same reason as in $\mathbf{Hyb}_{16.1.1}$, $\mathbf{Hyb}_{16.1.4}$ and $\mathbf{Hyb}_{16.1.3}$ have the same output distribution.

$\mathbf{Hyb}_{16.1.5}$: In this hybrid, for each gate in the first layer in C with input wires a, b and output wire c, Sim doesn't simulate the whole process of computing honest parties' shares of $[\boldsymbol{k}_{c,\Lambda_{c,2(v_a\oplus\lambda_a)+(v_b\oplus\lambda_b\oplus 1)}}]_d$ (before adding the additive errors) via emulating corrupted parties to run the protocol honestly. Instead, Sim directly samples them based on $k^{(i)}_{c,0}, k^{(i)}_{c,1}$ for each corrupted P_i and the corrupted parties' shares. Then, for the group of gates containing c, the honest parties' shares of degree-d sharings generated by honest members of virtual parties in $\mathcal{H}_{\mathsf{vir}}$ are sampled based on their shares of $[\boldsymbol{k}_{c,\Lambda_{c,2(v_a\oplus\lambda_a)+(v_b\oplus\lambda_b\oplus 1)}}]_d$, the corrupted parties' shares, and the secrets. For $[\boldsymbol{k}_{c,\Lambda_{c,\beta}}]_d$ with $\beta \neq 2(v_a\oplus\lambda_a\oplus 1)+(v_b\oplus\lambda_b)$ and $\beta \neq 2(v_a\oplus\lambda_a)+(v_b\oplus\lambda_b\oplus 1)$, Sim still follows the protocol to compute honest parties' shares. For the same reason as in $\mathbf{Hyb}_{16.1.2}$, $\mathbf{Hyb}_{16.1.5}$ and $\mathbf{Hyb}_{16.1.4}$ have the same output distribution.

$\mathbf{Hyb}_{16.1.6}$: In this hybrid, for each gate g with input wires a, b and output wire c in the first layer of the circuit, for each honest party P_i, Sim does not follow the protocol to compute the ciphertext $\mathsf{ct}^{(i)}_{g,2(v_a\oplus\lambda_a)+(v_b\oplus\lambda_b\oplus 1)}$. Instead, Sim samples a random string as it. For the same reason as in $\mathbf{Hyb}_{16.1.3}$, $\mathbf{Hyb}_{16.1.6}$ and $\mathbf{Hyb}_{16.1.5}$ have the same output distribution.

$\mathbf{Hyb}_{16.1.7}$: In this hybrid, for each gate in the first layer in C with input wires a, b and output wire c, Sim does not follow the protocol to compute honest parties' shares of $[\boldsymbol{k}_{c,\Lambda_{c,2(v_a\oplus\lambda_a\oplus 1)+(v_b\oplus\lambda_b\oplus 1)}}]_d$. Instead, Sim first follows the protocol to compute what these shares should be by replacing what honest parties received with the messages computed by Sim via emulating corrupted parties to run the protocol honestly. Finally, Sim computes the additive errors on them, where the additive errors are computed by the subtractions of the messages the additive errors on the parties/virtual parties' shares of $\Sigma, \Sigma^{(2)}$-sharings and degree-d Shamir sharings. Finally, Sim adds them up. For the same reason as in $\mathbf{Hyb}_{16.1.1}$, $\mathbf{Hyb}_{16.1.7}$ and $\mathbf{Hyb}_{16.1.6}$ have the same output distribution.

$\mathbf{Hyb}_{16.1.8}$: In this hybrid, for each gate in the first layer in C with input wires a, b and output wire c, Sim doesn't simulate the whole process of computing honest parties' shares of $[\boldsymbol{k}_{c,\Lambda_{c,2(v_a\oplus\lambda_a\oplus 1)+(v_b\oplus\lambda_b\oplus 1)}}]_d$ (before adding the additive errors) via emulating corrupted parties to run the protocol honestly. Instead, Sim directly samples them based on $k^{(i)}_{c,0}, k^{(i)}_{c,1}$ for each corrupted P_i and the corrupted parties' shares. Then, for the group

of gates containing c, the honest parties' shares of degree-d sharings generated by honest members of virtual parties in $\mathcal{H}_{\mathsf{vir}}$ are sampled based on their shares of $[\boldsymbol{k}_{c,\Lambda_{c,2(v_a\oplus\lambda_a\oplus 1)+(v_b\oplus\lambda_b\oplus 1)}}]_d$, the corrupted parties' shares, and the secrets. For $[\boldsymbol{k}_{c,\Lambda_{c,2(v_a\oplus\lambda_a)+(v_b\oplus\lambda_b)}}]_d$, Sim still follows the protocol to compute honest parties' shares. For the same reason as in $\mathbf{Hyb}_{16.1.2}$, $\mathbf{Hyb}_{16.1.8}$ and $\mathbf{Hyb}_{16.1.7}$ have the same output distribution.

$\mathbf{Hyb}_{16.1.9}$: In this hybrid, for each gate g with input wires a, b and output wire c in the first layer of the circuit, for each honest party P_i, Sim does not follow the protocol to compute the ciphertext $\mathsf{ct}^{(i)}_{g,2(v_a\oplus\lambda_a\oplus 1)+(v_b\oplus\lambda_b\oplus 1)}$. Instead, Sim samples a random string as it. For the same reason as in $\mathbf{Hyb}_{16.1.3}$, $\mathbf{Hyb}_{16.1.9}$ and $\mathbf{Hyb}_{16.1.8}$ have the same output distribution.

$\mathbf{Hyb}_{16.1.10}$: In this hybrid, for each gate in the first layer in C with input wires a, b and output wire c, Sim does not follow the protocol to compute honest parties' shares of $[\boldsymbol{k}_{c,\Lambda_{c,2(v_a\oplus\lambda_a)+(v_b\oplus\lambda_b)}}]_d$. Instead, Sim first follows the protocol to compute what these shares should be by replacing what honest parties received with the messages computed by Sim via emulating corrupted parties to run the protocol honestly. Finally, Sim computes the additive errors on them, where the additive errors are computed by the subtractions of the messages the additive errors on the parties/virtual parties' shares of $\Sigma, \Sigma^{(2)}$-sharings and degree-d Shamir sharings. Finally, Sim adds them up. For the same reason as in $\mathbf{Hyb}_{16.1.1}$, $\mathbf{Hyb}_{16.1.10}$ and $\mathbf{Hyb}_{16.1.9}$ have the same output distribution.

$\mathbf{Hyb}_{16.1.11}$: In this hybrid, for each gate in the first layer in C with input wires a, b and output wire c, Sim doesn't simulate the whole process of computing honest parties' shares of $[\boldsymbol{k}_{c,\Lambda_{c,2(v_a\oplus\lambda_a)+(v_b\oplus\lambda_b)}}]_d$ (before adding the additive errors) via emulating corrupted parties to run the protocol honestly. Instead, Sim directly samples them based on $k^{(i)}_{c,0}, k^{(i)}_{c,1}$ for each corrupted P_i and the corrupted parties' shares. Then, for the group of gates containing c, the honest parties' shares of degree-d sharings generated by honest members of virtual parties in $\mathcal{H}_{\mathsf{vir}}$ are sampled based on their shares of $[\boldsymbol{k}_{c,\Lambda_{c,2(v_a\oplus\lambda_a)+(v_b\oplus\lambda_b)}}]_d$, the corrupted parties' shares, and the secrets. For the same reason as in $\mathbf{Hyb}_{16.1.2}$, $\mathbf{Hyb}_{16.1.11}$ and $\mathbf{Hyb}_{16.1.10}$ have the same output distribution.

Now, we can delay the generation of $k^{(i)}_{v_w\oplus\lambda_w\oplus 1}$ for each wire w and honest party P_i after generating the ciphertexts for the gates in the first layer.

Then, we construct $\mathbf{Hyb}_{16.\gamma.1}, \ldots, \mathbf{Hyb}_{16.\gamma.10}$ for $\gamma = 2, \ldots, \mathsf{depth}(C)$ as $\mathbf{Hyb}_{16.1.1}, \ldots, \mathbf{Hyb}_{16.1.10}$ by replacing the gates in the first layer with the gates in the γ-th layer. For the same reasons as in $\mathbf{Hyb}_{16.1.1}, \ldots, \mathbf{Hyb}_{16.1.10}$, we conclude that $\mathbf{Hyb}_{16.\gamma.1}$ and $\mathbf{Hyb}_{16.\gamma-1.11}$ have the same output distribution, and $\mathbf{Hyb}_{16.\gamma.i}$ and $\mathbf{Hyb}_{16.\gamma-1.i-1}$ also have the same output distribution for each $\gamma = 2, \ldots, \mathsf{depth}(C)$ and $i = 2, \ldots, 11$.

$\mathbf{Hyb}_{16.4}$: Note that in $\mathbf{Hyb}_{16.\mathsf{depth}(C).11}$, the honest parties' shares of each $[\boldsymbol{k}_{c,\Lambda_{c,\beta}}]_d$ for $\beta \neq 2(v_a \oplus \lambda_a) + (v_b \oplus \lambda_b)$ are not used in the simulation. Sim no longer generates them. Moreover, for each virtual party V_j in $\mathcal{H}_{\mathsf{vir}}$ and honest party P_i, Sim does not sample V_j's share of $[\![\boldsymbol{k}^{(i,\alpha)}_{\mathbf{1}} - \boldsymbol{k}^{(i,\alpha)}_{\mathbf{0}}]\!]$ and $[\![\boldsymbol{k}^{(i,\alpha)}_{\mathbf{0}}]\!]^{(2)}$ and then computes the honest members' shares of the additive sharing of them in future hybrids. Since they are also not used in the simulation, not generating them won't affect the output distribution. Thus, $\mathbf{Hyb}_{16.4}$ and $\mathbf{Hyb}_{16.\mathsf{depth}(C).11}$ have the same output distribution.

Note that $\mathbf{Hyb}_{16.4}$ is the same as $\mathbf{Hyb}_{16}$, we conclude that $\mathbf{Hyb}_{16}$ and $\mathbf{Hyb}_{15}$ have the same output distribution.

$\mathbf{Hyb}_{17}$: In this hybrid, during the evaluation, after receiving corrupted parties' shares of $[\lambda_w]_t$ for each input wire w attached to an honest party, Sim doesn't follow the protocol to check the validity of the sharing. Instead, Sim directly checks whether the corrupted parties send these shares correctly. Since the $\geq n - t$ shares generated by honest parties uniquely determine the degree-t sharing, if corrupted parties don't send the shares correctly, the honest member must find that the shares are not from a valid degree-t sharing. Therefore, the two ways of verification lead to the same result. Thus, $\mathbf{Hyb}_{17}$ and $\mathbf{Hyb}_{16}$ have the same output distribution.

$\mathbf{Hyb}_{18}$: In this hybrid, after checking τ, Sim aborts the simulation if the check passes with two honest parties receiving different $v_{w_j} \oplus \lambda_{w_j}$ for an input wire w_j attached to a corrupted party. We assume that δ_i is the subtraction of $v_{w_i} \oplus \lambda_{w_i}$ received by the two honest parties, with $\delta_j \neq 0$. Then, the check passes only when $\sum_{i=1}^{G_I} s_i \cdot \delta_i = 0$. Note that $\delta_j \neq 0$, there is only one $s_j \in \mathbb{F}_{2^\kappa}$ that can make $\sum_{i=1}^{G_I} s_i \cdot \delta_i = 0$ hold when $s_i, i \in \{1, \ldots, G_I\} \backslash \{j\}$ are fixed. Therefore, in this case, if $s_i, i \in \{1, \ldots, G_I\}$ are truly random, the check passes for at most $2^{-\kappa}$ probability, which is negligible.

Thus, if there is a non-negligible probability that the check passes, then the truly random elements $s_i, i \in \{1, \ldots, G_I\} \backslash \{j\}$ and the pseudorandom ones expanded from a PRG can be distinguished by computing the outputs of the two hybrids with the same function from truly random and psuedorandom $s_i, i \in \{1, \ldots, G_I\}$, with a non-negligible probability. This contradicts the definition of a PRG, so the probability that the check passes in this case is negligible. Therefore, the distribution only changes with negligible probability. Thus, the distributions of $\mathbf{Hyb}_{18}$ and $\mathbf{Hyb}_{17}$ are computationally indistinguishable.

$\mathbf{Hyb}_{19}$: In this hybrid, during the evaluation, after receiving corrupted parties' shares of $[\lambda_w]_t$ for each output wire w attached to an honest party, Sim doesn't follow the protocol to check the validity of the sharing. Instead, Sim directly checks whether the corrupted parties send these shares correctly. Since the $\geq n - t$ shares generated by honest parties uniquely determine the degree-t sharing, if corrupted parties don't send the shares correctly, the honest member must find that the shares are not from a valid degree-t sharing. Therefore, the two ways of verification lead to the same result. Thus, $\mathbf{Hyb}_{19}$ and $\mathbf{Hyb}_{18}$ have the same output distribution.

$\mathbf{Hyb}_{20}$: In this hybrid, the honest parties do not compute their outputs by themselves. Instead, they obtain their output from $\mathcal{F}$. In addition, Sim does not compute corrupted parties' outputs by evaluating the circuit. Instead, Sim gets corrupted parties outputs from $\mathcal{F}$. Since the evaluation process of $\mathcal{F}$ and Sim is the same, the outputs computed in the two different ways are the same. Thus, $\mathbf{Hyb}_{20}$ and $\mathbf{Hyb}_{19}$ have the same output distribution.

$\mathbf{Hyb}_{21}$: In this hybrid, Sim does not generate honest parties' shares of $[\lambda_w]_t$ for each wire w except input/output wires attached to corrupted parties, auxiliary random bit sharings, degree-t sharings for ROT relations prepared for the members of virtual parties that are not on any corrupted party's watchlist, honest parties' shares of the degree-t sharings of $[\![\boldsymbol{r}^{(1)}]\!], \ldots, [\![\boldsymbol{r}^{(n)}]\!]$ (used for checking the Σ-sharings) for honest virtual parties that are not on any corrupted party's watchlist, and the degree-d sharing $[\boldsymbol{k}_{c,v_w \oplus \lambda_w \oplus 1}]_d$ for each output wire c of a gate. In addition, Sim does not sample $k^{(j)}_{c,v_w \oplus \lambda_w \oplus 1}$ for each honest party P_j and each wire w. Since these values are not used in the simulation, not generating them won't affect the output distribution. Thus, $\mathbf{Hyb}_{21}$ and $\mathbf{Hyb}_{20}$ have the same output distribution.

Note that $\mathbf{Hyb}_{21}$ is the ideal-world scenario, Π_{Main} computes $\mathcal{F}$ with computational security in the $\{\mathcal{F}_{\mathsf{Coin}}, \mathcal{F}_{\mathsf{prep}}, \mathcal{F}_{\mathsf{ROT}}\}$-hybrid model.

D Cost Analysis for the Main Protocol

We give the analysis for the communication cost of the main protocol as follows.

Preprocessing. The preprocessing only involves an invocation of $\mathcal{F}_{\mathsf{prep}}$. From Lemma 5, the preprocessing requires communication of $O(|C|n\kappa + n^2\kappa^2 + n^3\kappa)$ bits in the plain model.

Preparing ROTs. This step only involves an invocation of $\mathcal{F}_{\mathsf{ROT}}$. From Lemma 6, this step requires communication of $O(n^3\kappa \cdot 8\ell^2 G\lceil \kappa/n^2 \rceil / K) = O(|C|n\kappa)$ bits in the plain model.

Preparing Inputs for Virtual Parties. We analyze the cost of Π_{VPinput} as follows:

1. **Committing Watchlists.** Each party's watchlist is a set of $N/12n = O(n + \kappa/n)$ virtual parties, which requires $O((n + \kappa/n) \cdot \log N) = O((n + \kappa/n) \cdot \log n)$ bits. Each watchlist is distributed to all the parties via degree-t Shamir sharings, so the size of all these sharings is $O((n + \kappa/n) \cdot (\log n) \cdot n^2) = O(n^3 \log n + n\kappa \log n)$ bits. Thus, this step requires communication of $O(n^3 \log n + n\kappa \log n)$ bits.

2. **Sharing the Wire Labels.** In this step, for each group of $K = O(N)$ gates, each party distributes $O(\kappa)$ $\Sigma, \Sigma^{(2)}$-sharings, where each of the sharings have size of $O(N)$ bits. Thus, this step requires communication of $O(GNn\kappa/K) = O(|C|n\kappa + Nn\kappa) = O(|C|n\kappa + n^3\kappa + n\kappa^2)$ bits.

3. **Sharing Colored Bits to Virtual Parties.** In this step, the parties need to send a degree-t sharing of a Σ-sharing for each group of $K = O(N)$ gates, which requires communication of $O(GNn\kappa/K) = O(|C|n\kappa + n^3\kappa + n\kappa^2)$ bits.

4. **Preparing the Verification of Sharings.** In this step, the parties distributes degree-t Shamir sharings of n $\Sigma_{\times\kappa}$-sharings, which requires communication of $O(n^2N\kappa) = O(n^4\kappa + n^2\kappa^2)$ bits.

In conclusion, the communication cost of Π_{VPinput} is $O(|C|n\kappa + n^4\kappa + n^2\kappa^2)$ bits.

Virtual Parties' Local Computation. We analyze the cost of Π_{VPcomp} as follows:

1. **Computing Multiplications.** During the first step, the members of each virtual party need to execute Π_{Mult} $4\ell^2 = O(1)$ times for each group of $K = O(N)$ gates, where each execution of Π_{Mult} requires communication of $O(n\kappa)$ bits. Thus, this step requires communication of $O(GNn\kappa/K) = O(|C|n\kappa + n^3\kappa + n\kappa^2)$ bits.

2. **Verification of the Sharings.** During this step, the parties invoke $\mathcal{F}_{\mathsf{Coin}}$ once. Moreover, the degree-t sharing of n $\Sigma_{\times\kappa}$-sharings are reconstructed, and the parties send a $\Sigma_{\times\kappa}$-sharing to all the parties. The instantiation of $\mathcal{F}_{\mathsf{Coin}}$ requires communication of $O(n^2\kappa)$ bits, the reconstruction requires communication of $O(n^2N\kappa) = O(n^4\kappa+n^2\kappa^2)$ bits, and sending the sharings requires communication of $O(nN\kappa) = O(n^3\kappa + n\kappa^2)$ bits. Thus, this step requires communication of $O(n^2N\kappa) = O(n^4\kappa + n^2\kappa^2)$ bits.

In conclusion, the communication cost of Π_{VPcomp} is $O(|C|n\kappa + n^4\kappa + n^2\kappa^2)$ bits.

Verifying the Computation. We analyze the cost of Π_{VPver} as follows:

1. **Verifying the Sharings.** This step only involves local computation.

2. **Opening Watchlists.** In this step, the degree-t sharings for the watchlist are reconstructed to all the parties, so the cost is n times the total size of the degree-t sharings for the watchlist. Thus, this step requires communication of $O(n^4 \log n + n^2\kappa \log n)$ bits.

3. **Opening Transcripts.** There are $O(N)$ virtual parties' transcripts that are required to be sent to a party. Thus, this step has the same communication complexity as the step of performing virtual parties' local computation, which is $O(|C|n\kappa + n^3\kappa + n\kappa^2)$ bits.

4. **Opening Inputs.** This step requires sending a constant fraction of the virtual parties' shares for labels and their degree-t sharings for their shares of shares for colored bits, which requires the same communication complexity as the sum of the communication complexity of Steps 2 and 3 of Π_{VPinput}, i.e. $O(|C|n\kappa+n^3\kappa+n\kappa^2)$ bits. This step also requires reconstruction of degree-t sharings for a constant fraction of n $\Sigma_{\times\kappa}$-sharings, which requires communication of $O(n^2N\kappa) = O(n^4\kappa + n^2\kappa^2)$ bits. Thus, this step requires communication of $O(n^2N\kappa) = O(|C|n\kappa + n^4\kappa + n^2\kappa^2)$ bits.

5. **Opening ROTs.** In this step, the parties send their shares for the ROT correlations generated for the parties on the watchlists to the holder of the watchlist. Thus, the communication complexity of this step is the same as the size of outputs from $\mathcal{F}_{\mathsf{ROT}}$, which is $O(|C|n\kappa)$ bits.

6. **Checking the Computations.** This step only involves local computations of the parties.

In conclusion, the communication cost of Π_{VPver} is $O(|C|n\kappa + n^4\kappa + n^2\kappa^2)$ bits.

Garbling the Circuit. We analyze the cost of Π_{Garble} as follows:

1. **Transforming to PSSs of Each Gate's Labels.** In this step, the parties jointly send a $\Sigma^{(2)}$-sharing of $[\boldsymbol{k}_{c,\Lambda_{c,i}}]_d$ for each output wire c of a gate and $i = 1, 2, 3, 4$. Each $[\boldsymbol{k}_{c,\Lambda_{c,i}}]_d$ has size $O(n\kappa)$, thus the cost for each gate is $O(n\kappa)$ bits. This step is performed in groups of $K = O(N)$ gates, so the total communication cost of this step is $O(|C|n\kappa + Nn\kappa) = O(|C|n\kappa + n^3\kappa + n\kappa^2)$ bits.

2. **Encrypting the Shares.** This step only involves local computations of the parties.

3. **Sending Ciphertexts.** There are 4 ciphertexts that needs to be sent from each party to P_{king} for each gate, where each ciphertext has size $O(\kappa)$. Thus, this step requires communication of $O(|C|n\kappa)$.

In conclusion, the communication cost of Π_{Garble} is $O(|C|n\kappa + n^3\kappa + n\kappa^2)$ bits.

Evaluating the Circuit. We analyze the cost of Π_{Eval} as follows:

1. **Computing Masked Inputs.** In this step, for each input wire w, each party needs to send a share of $[\lambda_w]_t$ to the input holder and receives a bit $v_w \oplus \lambda_w$ from him. Then, the parties will invoke $\mathcal{F}_{\mathsf{Coin}}$, which requires communication of $O(n^2\kappa)$ bits to be instantiated in the plain model (see Lemma 4). Finally, the parties send an element $\tau \in \mathbb{F}_{2^\kappa}$ to each other, which requires communication of Thus, this step requires communication of $O(W_I n\kappa + n^2\kappa)$ bits.

2. **Sending Input Labels.** In this step, the parties send input wire labels to P_{king}, where each input wire label is of size $O(n\kappa)$. Thus, this step requires communication of $O(W_I n\kappa)$ bits.

3. **Evaluating the Garbled Circuit.** This step only involves local computation of P_{king}.

4. **Sending Output Labels.** In this step, P_{king} sends output wire labels to the output holders, where each output wire label is of size $O(n\kappa)$. Thus, this step requires communication of $O(W_O n\kappa)$ bits, where W_O is the output size of the circuit.

5. **Checking Output Labels.** This step only involves local computation of the parties.

6. **Computing Outputs.** In this step, each party needs to send a share of $[\lambda_w]_t$ for each output wire w, which requires communication of $O(W_O n\kappa)$ bits.

Taking W_I, W_O bounded by $|C|$, the communication cost of Π_{Eval} is $O(|C|n\kappa)$.

Adding up the cost of all the steps, the main protocol requires communication of $O(|C|n\kappa + n^4\kappa + n^2\kappa^2)$ bits.

E Analysis of Round Complexity

We analyze the round complexity of our main protocol in this section.

Preprocessing. The first step of Π_{Prep} requires 2 rounds of communication and an invocation of $\mathcal{F}_{\mathsf{Coin}}$, where the instantiation of $\mathcal{F}_{\mathsf{Coin}}$ requires 2 rounds. The first step can be executed in parallel with the invocation of $\mathcal{F}_{\mathsf{triple}}$ in the second step. $\mathcal{F}_{\mathsf{triple}}$ can be realized via the MPC protocol of [CGH+18] in 12 rounds. Apart from the invocation of $\mathcal{F}_{\mathsf{triple}}$, the second step requires 3 extra rounds of communication and an invocation of $\mathcal{F}_{\mathsf{Coin}}$ that can be instantiated in 2 rounds. Thus, the preprocessing requires 17 rounds.

Preparing ROTs. The invocation of $\mathcal{F}_{\mathsf{ROT}}$ can be instantiated with Π_{ROT}. Using the MPC protocol from [CGH+18] to instantiate $\mathcal{F}_{\mathsf{MPC}}$, $\mathcal{F}_{\mathsf{ROT}}$ can be realized in 12 rounds. Note that this step can be executed in parallel with the preprocessing, we don't need any extra rounds in this step.

Preparing Inputs for Virtual Parties. The steps of Π_{VPinput} can be executed in parallel, which only involves one extra round of distributing the sharings.

Virtual Parties' Local Computation. The virtual parties' local computation contains parallel computations of Π_{Mult} that requires one extra round. Afterwards, the parties invoke $\mathcal{F}_{\mathsf{Coin}}$ and reconstruct degree-t sharings in parallel, which requires 2 extra rounds. Finally, the virtual parties send their shares of a $\Sigma_{\times^\kappa}$ to all the parties, which requires another round. Thus, the execution of Π_{VPcomp} requires 4 extra rounds.

Verifying the Computation. The first 5 steps of Π_{VPver} can be performed in parallel, and the last step only contains local computations. Thus, the execution of Π_{VPver} only requires one extra round.

Garbling the Circuit. The first step of Π_{Garble} requires one extra round of communication, and the second step requires another round. Thus, the execution of Π_{Garble} requires 2 extra rounds.

Evaluating the Circuit. The first step of Π_{Eval} requires 3 rounds of communication and an invocation of $\mathcal{F}_{\mathsf{Coin}}$, where the instantiation of $\mathcal{F}_{\mathsf{Coin}}$ requires 2 rounds. The second step requires another round. The 6 rounds can be run in parallel with $\Pi_{\mathsf{VPcomp}}, \Pi_{\mathsf{VPver}}$, and Π_{Garble}. Steps 4 and 6 can be executed in parallel in one round. Steps 3 and 5 only contain local computations. Thus, the execution of Π_{Eval} requires one extra round.

In conclusion, our main protocol requires 26 rounds of communication.

If we only ensure semi-honest security, the instantiation of $\mathcal{F}_{\mathsf{ROT}}$ and $\mathcal{F}_{\mathsf{triple}}$ only requires 3 rounds, we don't need the coins for the checks of sharings, and virtual parties' computations also do not need to be checked. In this case, we only need 5 rounds for the preparation, 1 extra round for Π_{VPinput}, 1 extra round for Π_{VPcomp}, 2 extra rounds for Π_{Garble}, and another round for Π_{Eval}. As a result, we require 10 rounds to achieve semi-honest security.